Chapter 1

OUTLINE

Unit 1: System Development Fundamentals(9 Hrs.)

a.System Development Environment

Introduction; A Modern Approach to Systems Analysis and Design;Information System and its type, Developing Information Systems and the Systems Development Life Cycle; The Heart of the Systems Development Process and Traditional Waterfall SDLC; Approaches of Improving Development,CASE Tools

Rapid Application Development; Service Oriented Architecture(SOA),Agile Methodology,eXtreme Programming,Object Oriented Analysis and Design

b.Origin of Software:

Introduction,System *Acquision,reuse*

c.*Managing the Information Systems Project*:

Introduction; Managing the Information Systems Project; Representing and Scheduling Project Plans; Using Project Management Software

System Analysis and Design

Introduction : Data, Information, and Knowledge

Data: raw facts
Information: collection of facts organized in such a way that they have value beyond the facts themselves.
Knowledge: awareness and understanding of a set of information and ways that information can be made useful to support a specific task or reach a decision

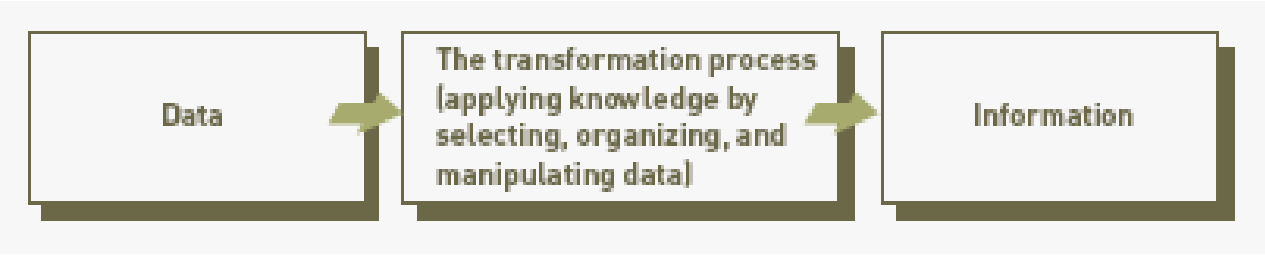
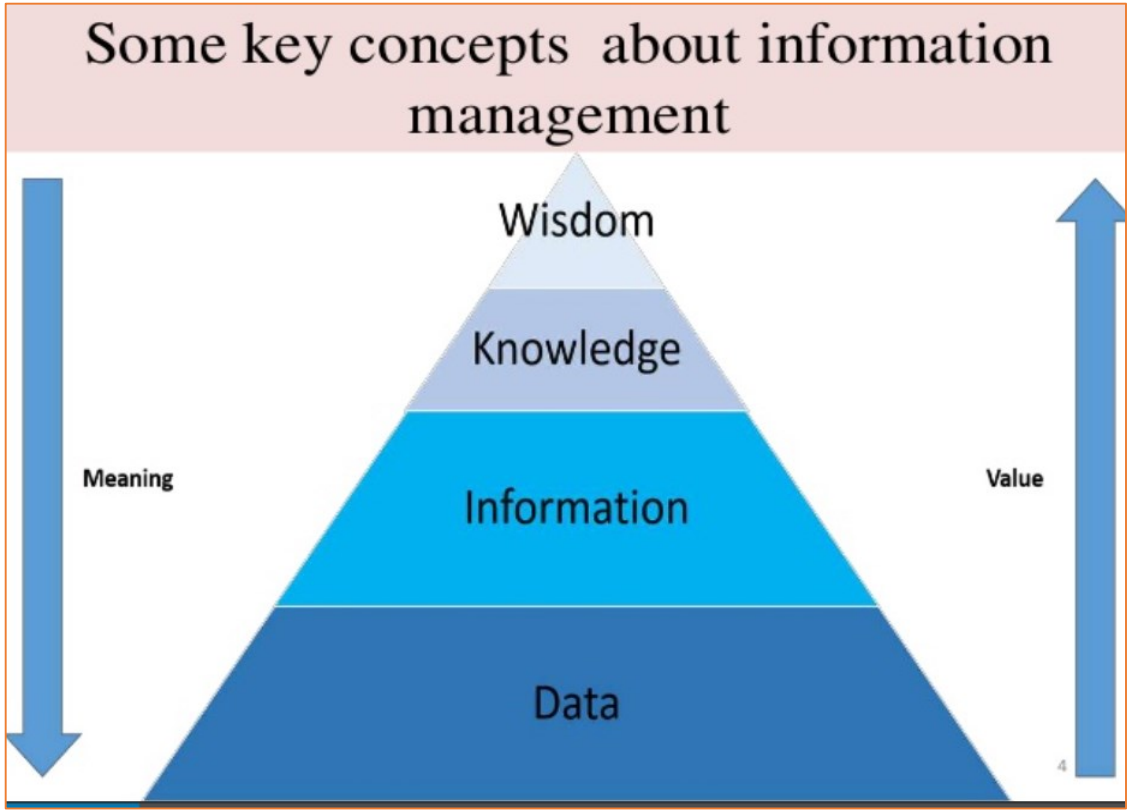


Figure : The Process of Transforming Data into Information



Information management

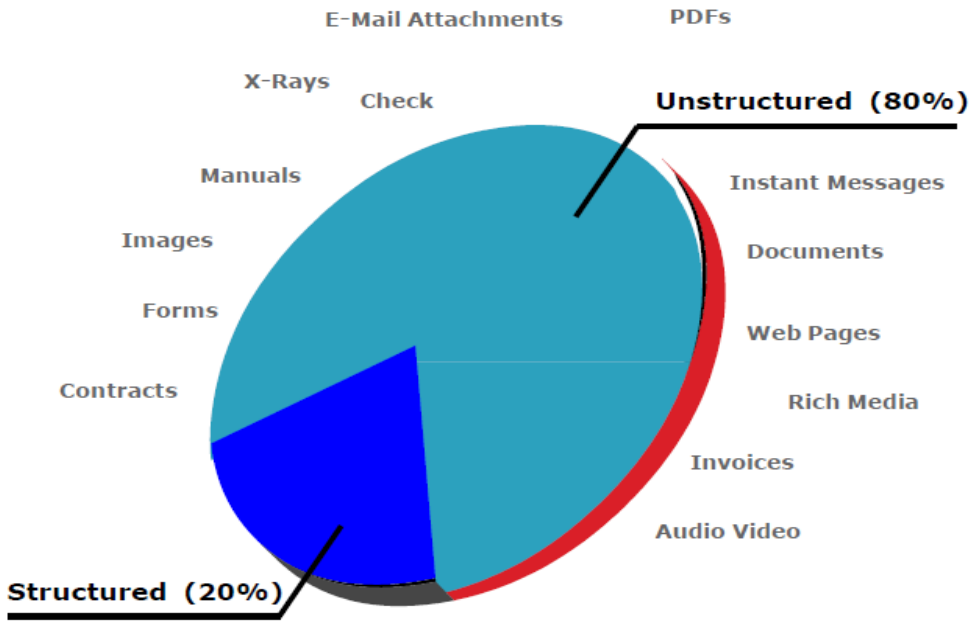
- Data: Unorganized and unprocessed facts, raw numbers, figures, images, words, sounds, derived from observations or measurements. Data is limitless and present everywhere in the universe Most data is being converted into a digital format.
- Who creates data?
  - Individuals
  - Businesses

CATEGORIES OF DATA:

Data can be categorized as either structured or unstructured data

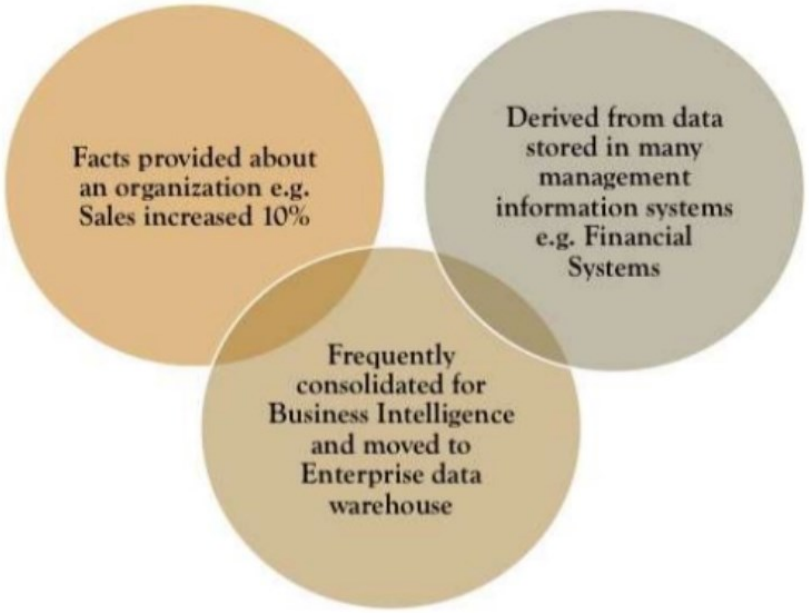
- **Structured**
  - Data Bases
  - Spread Sheets
- **Unstructured**
  - Forms
  - Images
  - Audio
  - Movies

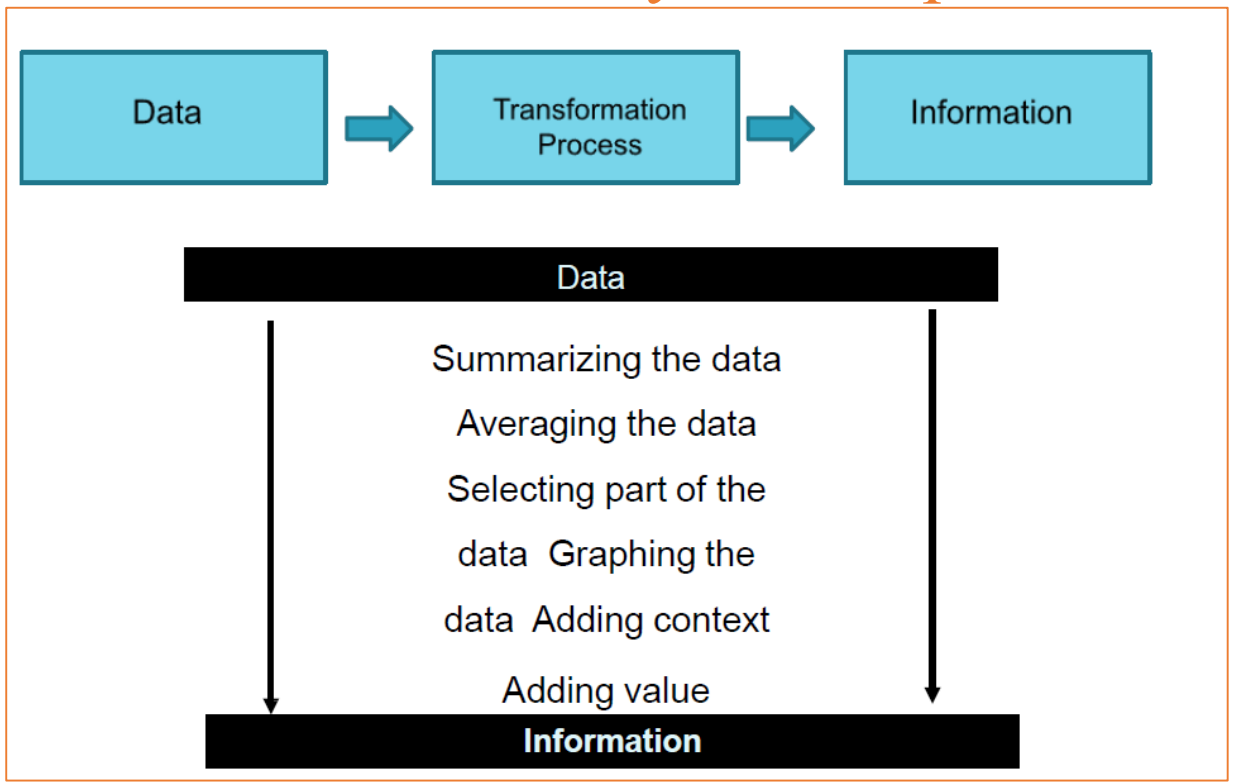
Over 80% of Information is unstructured



Information

- Organized form of data is known as information
- Definitions:
  - data that have been processed so that they are meaningful;
  - data that have been processed for a purpose;
  - data that have been interpreted and understood by the recipient.





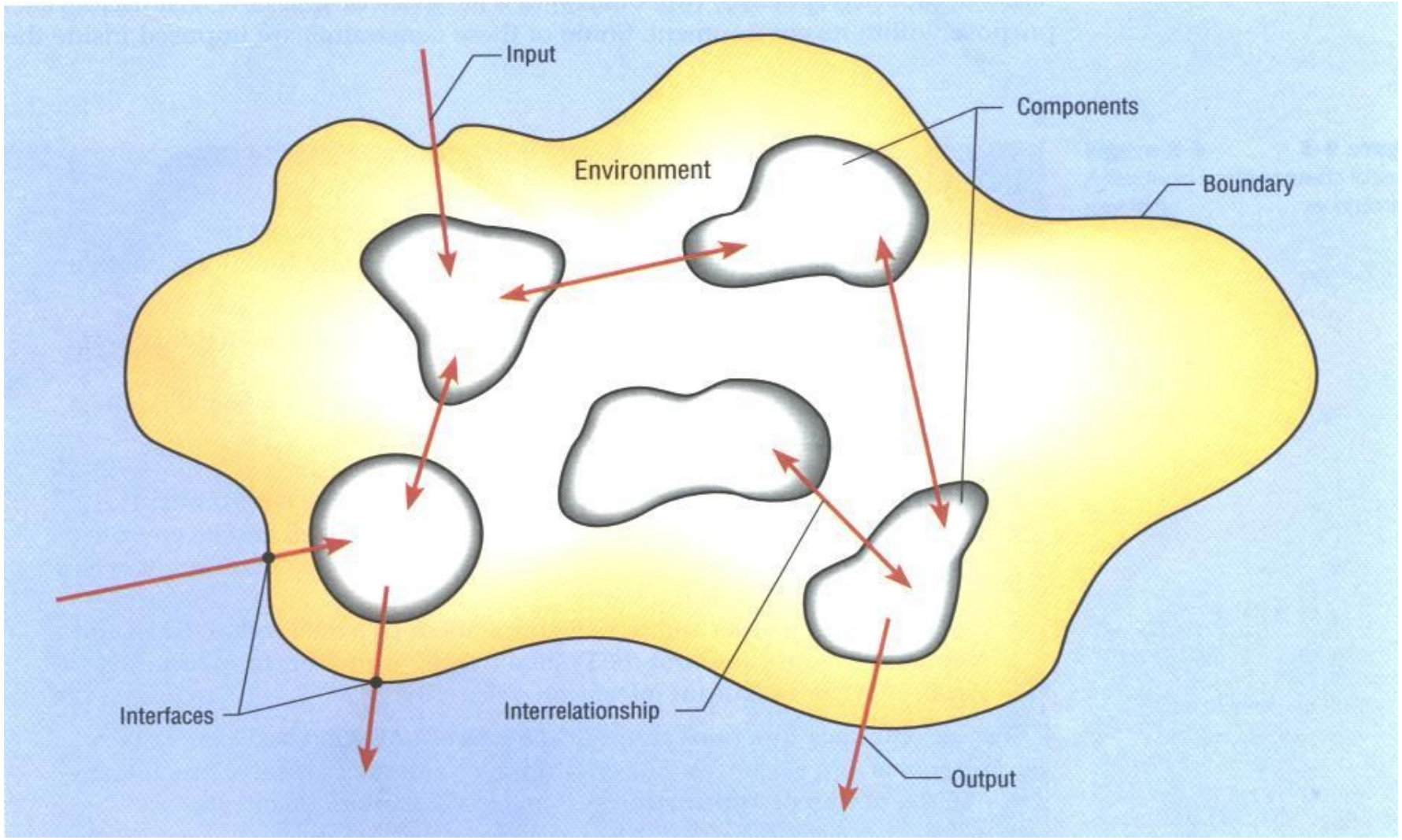
Knowledge

According to Wikipedia..Knowledge is familiarity, awareness or understanding of something such as facts, information, description or skills, which is acquired through experience or education by perceiving, discovering or learning.

Defining System :

- System is an interrelated set of components with an identifiable boundary working together for some purpose.
- It is also defined as ,a purposeful collection of inter-related components working together some common objectives.
- A system exists within an environment
- A boundary separates a system from its environment
- A system mainly includes
  - software ,mechanical , electrical and electronic hardware and be operated by people.
- System components are dependent on other system components.

Defining System :



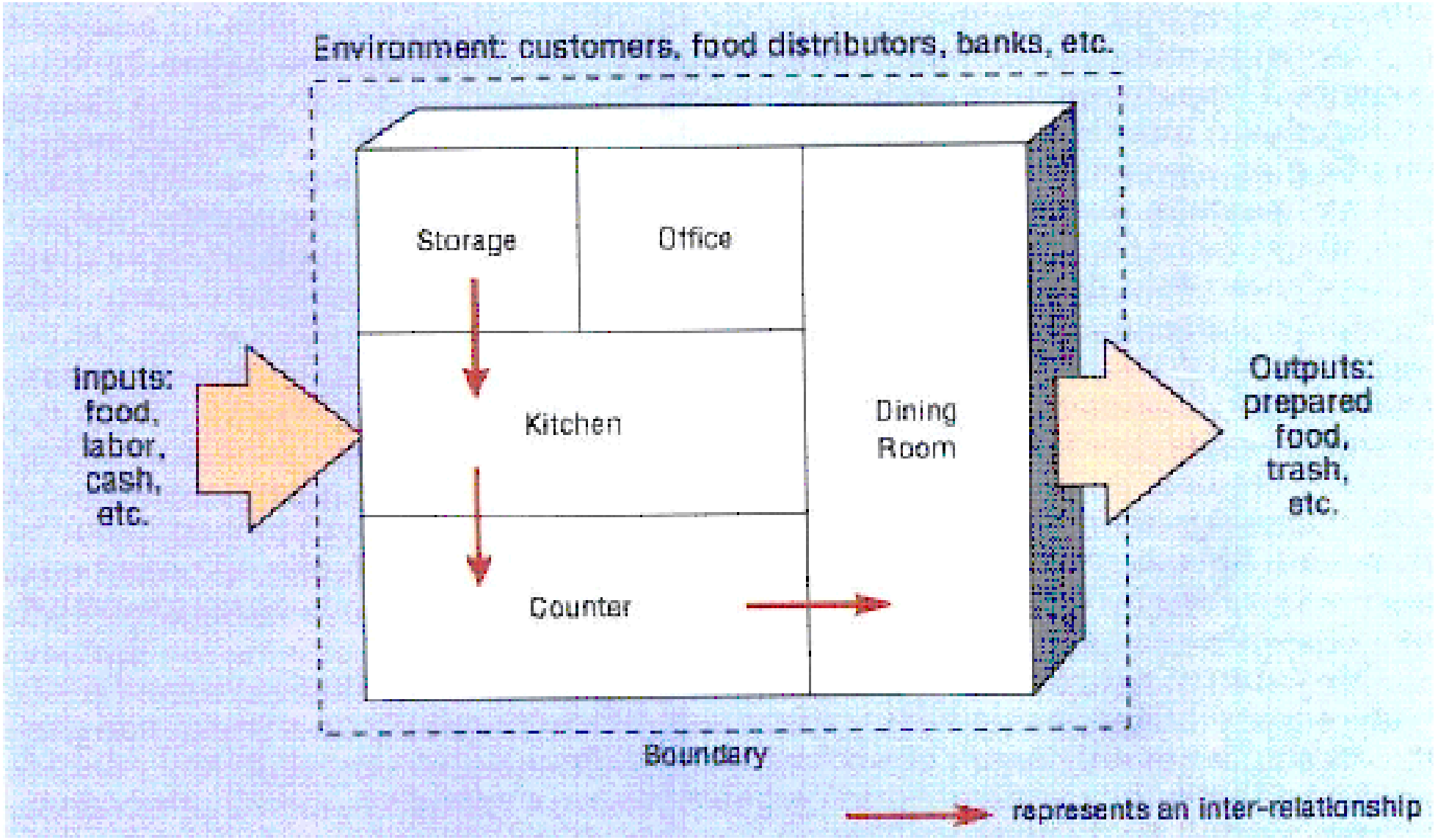
Components of a system

- Input
- Processing
- Output
- Feedback

System Characteristics :

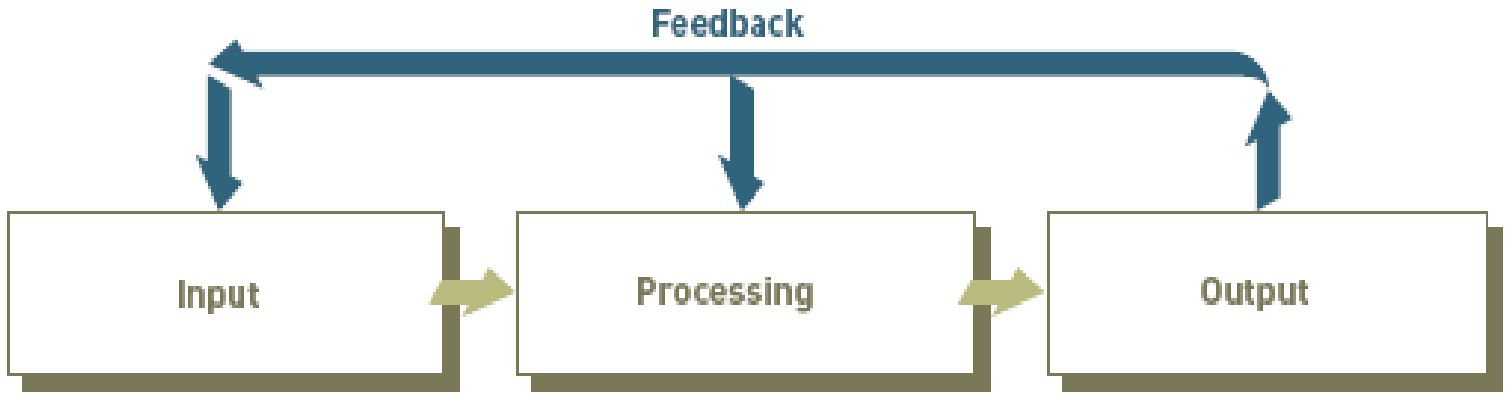
- *Components :* An irreducible part or aggregate of parts that make up a system ,also called subsystem
- Interrelationships between components
- *Boundary:* The line that marks the inside and outside of a system and that sets off the system from its environment
- *Purpose:* The overall goal or function of a system
- *Environment:* Everything external to a system that interacts with the system.
- *Interfaces:* Point of contact where a system meets its environment or where subsystems meet each other.
- *Input:* Whatever a system takes from its environments in order to fulfill its purpose
- *Output:* Whatever a system returns to its environments in order to fulfill its purpose
- *Constraints:* A limit to what a system can accomplish

A fast food restaurant as a system



Information System

- An information system is an arrangement of people, data, process ,communication and information technology that interacts to collect, process ,store, and provide as output the information needed to support to improve day to day operations in a business ,as well as to support the problem solving and decision making needs of management and users



The Components of an Information System

An Information System

- A set of interrelated components that collect, manipulate, and disseminate data and information, and provide feedback to meet an objective.
- System users, business managers, and information systems professionals must work together to build a successful information system.
- Example: A payroll system, for example, collects information on employees and their work, processes and stores that information, and then produces paychecks and payroll reports for the organization.

Computer-based information system (CBIS)

- Computer-based Information System (CBIS): An information system that uses computer technology to perform some or all of its intended tasks.
- Examples: Airline reservation systems;

Components of Information System :

- An information system includes various components.
- Among which some importants components are given below:
- **People, process and data:**
  - Information is produced and used by people in an organization .
  - Process determines a set of steps how things are done with data as it enters and passes through the system.
  - Process must support the tasks carried out by each person as well as an interaction between persons.
  - Data is the core information actually carried by an IS by receiving raw level and then resulting fine in the abstract.
  - Many equipments used to store the data ,move it around the organization and carryout any computation in data which involves the computer itself, their disks drives, I/O devices , interfaces and communication device.

Components of Information System :

- **Database:**
  - This term is often used to refer to the data stored in a computer. It not only includes the structure records but can also includes forms, letters, and notes that people may keep.
  - In the past ,database used to be mainly the structured record such as accounts or orders , now it increasingly includes texts documents ,images and other kinds of artifacts an objects process by a computer.

- **Computer System:**
  - System is a collection of components that work together to realize an objectives . Computer based information system is very simple system which supports the person keep track of their records using personal computer, laptops ,PDA. The Computer System may be:
    - Personal Computer System
    - Centralized System
    - Distributed System

Types of Information System :

- Several different people in an organization can be involved in developing information systems. Given the broad range of people and interest represented in system development , it could take different types of IS to satisfy all an organizations IS needs and classified as below:

  - Transaction Processing System :
  - Management Information System
  - Decision Support System
  - Expert System
  - Office Automation System

Types of Information System :

- **Transaction Processing Systems (TPS)**
  Automate handling of data about business activities (transactions)
  Process orientation
- **Management Information Systems (MIS)**
  Converts raw data from transaction processing system into meaningful form
  Data orientation
- **Decision Support Systems (DSS)**
  Designed to help decision makers
  Provides interactive environment for decision making
  Involves data warehouses, executive information systems (EIS)
  Database, model base, user dialogue

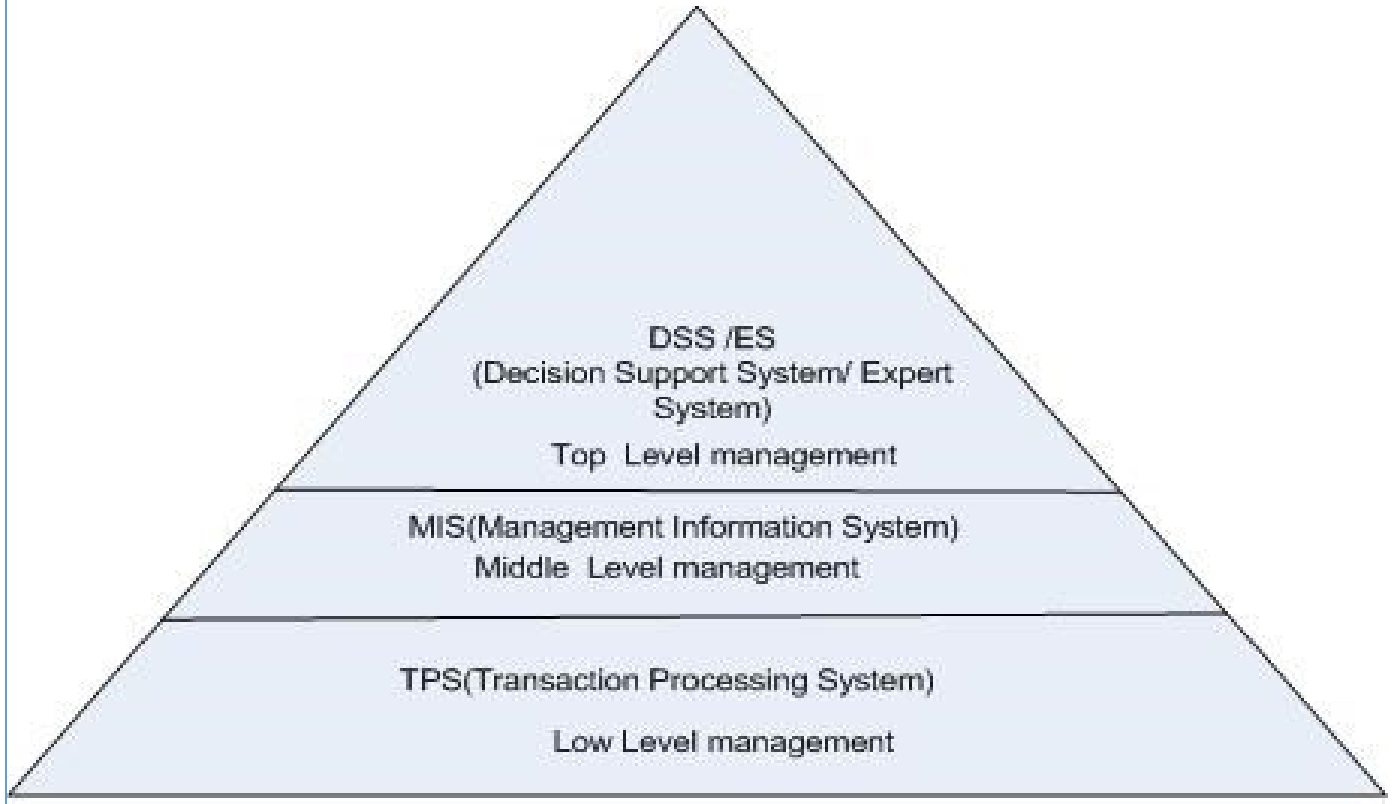


Fig: Placement of Information System according to Level of Managment

Types of Information System :

- **Transaction Processing System (TPS) :**
  - A TPS collects and stores information about transactions and controls some aspects of transactions .
  - A transaction is an event of interest to the organization.
  - It is an information system that capture and process data generated during the day – to- day transactions of an organizations .
  - For example A sale at a store, Deposits ,payments ,orders or reservations.
- **Management Information System (MIS):**
  - MIS takes the relatively raw data available through the TPS and converts them into a meaningful aggregated form that managers need to conduct their responsibilities.
  - MIS is an information system that generates accurate, timely and organized information so that managers **can** make decisions, solve problems, supervise activities and track progress.
  - It condenses and converts TPS data into information for monitoring performance and managing an organization.
  - Transactions recorded in a TPS are analyzed and reported by an MIS . They have large quantities of input data and they produce summary reports as output.
  - Used by middle level managers. For example : Annual Budgeting System.

Types of Information System :

- **Decision Support System (DSS) :**
  - DSS is an system designed to support strategic management staff to make decisions by providing information , models or analysis tools. It is used for analytical work, rather than general office support.
  - DSS uses data from internal/external sources. For example: 5 year operating plan.
- **Expert System(ES) :**
  - ES is an information system that captures and stores the knowledge of human experts and then imitates human reasoning and decision making process for those who have less expertise. ES attempts to codify and manipulate knowledge rather than information. Typically user communicate with an ES through an interactive dialog. The ES ask questions that can expert would ask and the end user replies the answer .The answer then used to determine which rules apply and ES provides a recommendations based on these rules. For example ,Artificial Intelligence(AI) the application of human intelligence to computer (Speech recognition system, Medical Diagnosis System(MYCIN)).

Types of Information System :

- **Office Automation System(OAS)**
  - OAS provides individual effective ways to process personal and organizational data, perform calculations and create documents. E.g. Word processing, spreadsheets, files management , personal calendars, presentation packages.
  - They are used for increasing personal productivity and reducing “paper warfare”

Types of Information System :

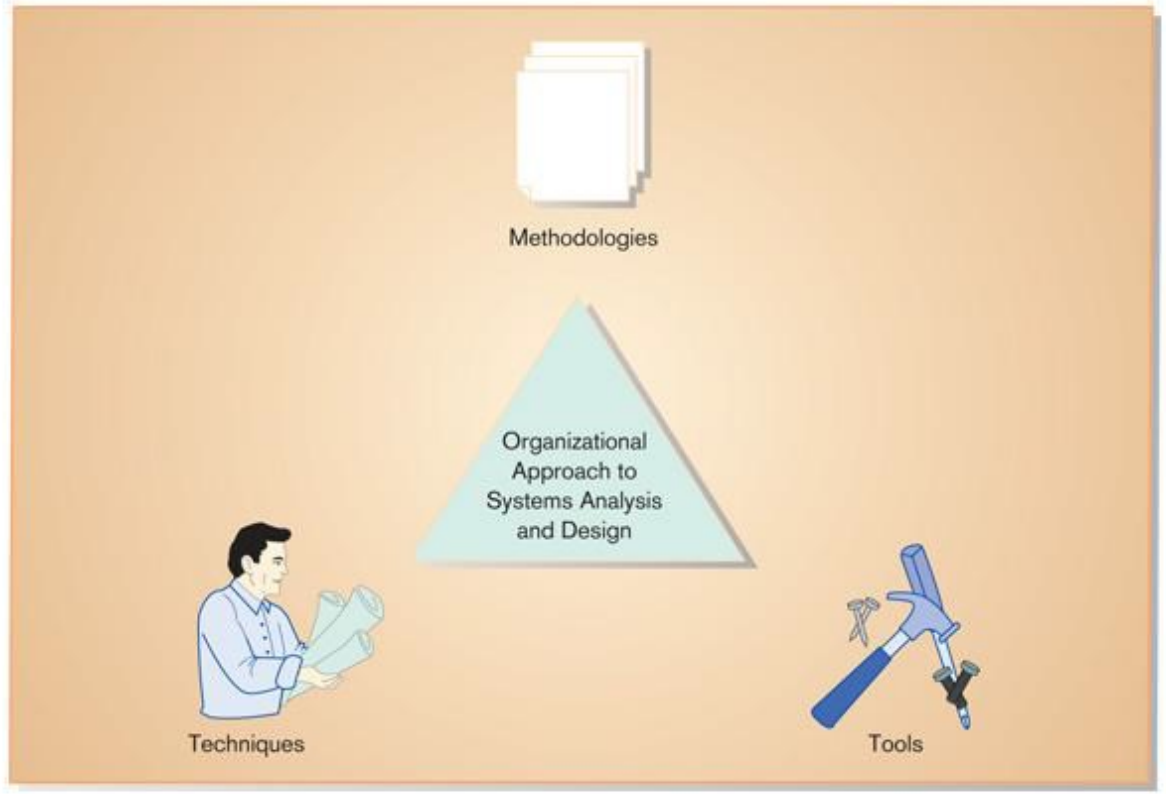
Table 1-1 Systems Development for Different IS Types

IS Type	*IS Characteristics*	*Systems Development Methods*
Transaction processing system	High-volume, data capture focus; goal is efficiency of data movement and processing and interfacing different TPSs	Process orientation; concern with capturing, validating, and storing data and with moving data between each required step
Management information system	Draws on diverse yet predictable data resources to aggregate and summarize data; may involve forecasting future data from historical trends and business knowledge	Data orientation; concern with understanding relationships among data so data can be accessed and summarized in a variety of ways; builds a model of data that supports a variety of uses
Decision support system	Provides guidance in identifying problems, finding and evaluating alternative solutions, and selecting or comparing alternatives; potentially involves groups of decision makers; often involves semi-structured problems and the need to access data at different levels of detail	Data and decision logic orientations; design of user dialogue; group communication may also be key, and access to unpredictable data may be necessary; nature of systems requires iterative development and almost constant updating

Information Systems Analysis and Design

- system analysis and design is the complex, challenging organizational process whereby systems are built and re-built for organizational benefits
- Used to develop and maintain computer-based information systems
- Used by a team of business and systems professionals
- The result of system analysis and design is application software that support organizational function and process.
- Along with application software, the information system includes the hardware and system software, people using software , documentation and materials.

Information System Development



An organizational approach to systems analysis and design is driven by methodologies, techniques, and tools

An output of system analysis and design is **application software**.

- **Methodologies** are comprehensive, multiple-step approaches to system development. Methodologies uses different techniques.
- **Techniques** are particular processes that you as an analyst, will follow to ensure your work is well thought out, complete, and comprehensive to others on your project team.

Eg: conducting interviews to determine what your system should do, diagramming the system's logic, designing the reports your system will generate.

Tools are computer programs that make it easy to use and benefit from techniques

System Analysis and Design

- System Analysis and Design refers to **the process of examining a business situation with the intent of improving it through better procedures and methods**.
- System analysis and design relates to shaping organizations, improving performance and achieving objectives for profitability and growth
- **System analysis**, then, is the process of gathering and interpreting facts, diagnosing problems, and using the information to recommend improvements to the system
- **System design** is the process of planning a new business system or one to replace or complement an existing system. But before this planning can be done, we must thoroughly understand the old system and determine how computers can best be used to make its operation more effective.

System Analysis and Design(SAD)

- **System Analysis:**
  - System analysis is a problem solving technique that decomposes a system into its components pieces for the purpose of studying how well these components part work and interact to accomplish their purpose.
  - System Analysis addresses the data, process and interface building block from system owner's perspectives.

- **System Design:**
  - It is a complementary problem solving technique to system analysis that reassembles a system components pieces back into a complete system.
  - The system is hoped an improved which may including, adding, updating, deleting pieces relative to the original system.
  - Hence system is integration of submit which are combined together for performing certain specific task.
  - System in our relevance is the information system.

System Analysis and Design(SAD)

- Hence **system analysis and design** is the complex, challenging organizational process whereby systems are built and re-built for organizational benefits.
- The result of system analysis and design is application software that support organizational function and process.
- Along with application software, the information system includes the hardware and system software, people using software , documentation and materials.

Stakeholders of IS(Who is involved for building IS)

- A stakeholder is any person who has an interest in an existing or new information system .
- He/She can be technical or non- technical person.
- What is an information worker?
  - “any person whose job involves creating, collecting, processing, distributing, and using information”
- **The information system stakeholders are :**
- **System Owner:** System owner pay for the system to be built and maintained. He owns the system, sets priorities for the system and determines policies for its use. In some cases system owner may also be the system user.
- **System User:** System user is the person who actually uses the system to perform or support work to be completed. System user defines the business requirements and performs expectations for the system to be built.

Stakeholders of IS(Who is involved for building IS)

- **System Analyst:** System analyst is a person who facilitates the development of IS and computer application by bridging the communication gap that exist between non-technical system owner, user and technical team ,designers and builders.
- **System Designers:** System designer is a technical specialist who designs the system to meet the user requirements.
- **System Builders:** System Builder is a technical specialist who develop (Construct, test) and delivers the system into operations.

- **IT Vendors and Consultants:** Who sales hardware, software and services to business for incorporation into there IS.

Stakeholder of IS

- Any person that is interested or affected by an information system.
  - Often include both technical and non-technical people
  - Often include internal workers and external workers or individuals
- 5 Main Groups of IS Stakeholders:
  - System Owners
  - System Users
  - System Designers
  - System Builders
  - System Analysts & Project Managers
- These can all be generally classed as information workers!

Stakeholder of IS: System Owner

- Own the system!
- Upper levels of Management
- Mainly interested in the business objectives of the system
- What will the system cost?
- What strategic benefits will the system bring to our business?
- How do we measure the value of the system to the business?

Stakeholder of IS: System User

- System Users are the majority of information workers affected by IS
- Less concerned with the business and strategic imperatives underpinning IS
- More concerned with the functionality of the IS
  - Does it let them do what they want to do?
  - How easy is the system to use?
- Internal & External System Users

Stakeholder of IS: System User

- Internal System Users:
  - Employees of the business that owns the IS
  - Clerical and Service Workers - Day to day transaction processing. Inputting information, processing payments, writing letters etc
  - Knowledge Workers – Business specialists who perform highly skilled and specialized work, e.g. lawyers, accountants, engineers etc. IS provide them with data analysis and information to do their work
  - Supervisors, Middle Managers, Executive Managers – IS provides access to all the information needed to support their problem solving and decision making activities.

Stakeholder of IS: System User

- External System Users:
  - Customers and Other Businesses
- Customers – Often interact with IS of a business when purchasing products or services. E.g. Aer Lingus, Amazon, tickets.ie
- Suppliers – Contemporary supply chain management solutions are IS based allowing suppliers to interact with businesses
- Partners – When companies partner with others to create or improve services or build new products.
- Employees – Remote access employees such as sales reps

Stakeholder of IS: System Designers

- Technology Specialists for Information Systems
- Translate business requirements into a feasible technical solution
- Interested in IT choices such as:
  - Databases
  - Networking
  - Security
  - Programming Languages

Stakeholder of IS: System Builders

- Another layer of technology specialists but are different from System Designers in that they construct the IS based on the design specifications!
- In small organisations the system designers and system builders are often the same but in larger organisations they are not. Why?

Stakeholder of IS: System Analysts

- Each of the other stakeholders have very different perspectives on IS.
- For instance, the System Owner will likely have very different perspectives on an IS than that of a System Designer.
- Some are only concerned with technical issues
- Others only with business issues
- A communications gap exists between people who need the IS and those that understand IT
- A key role of System Analyst is to bridge this gap!

Stakeholder of IS: System Analysts

- Their role overlaps all the other roles of the other stakeholders. They need, at the least, a general understand of each of the other stakeholder concerns.
- Facilitates interaction between groups of stakeholders.
- Analyse current and possible future IS for a business. This is not just a technology issue but concerns people, process, and technology
- Role of System Analyst will be discussed in greater detail later in the year!

Example of Stakeholders

- System Owner: Library Managers.
  - Want an IS to reduce costs of running library. Currently very labour intensive; implementation of new IS will remove a great deal of admin work through automationSystem
- User: One group of main users are front end desk staff.
  - The process for checking out a book is long, tedious, and laborious.
  - Difficult to find student information in log book, incomplete data entered in log books etc
- System Designer:
  - Design technical solution from business requirements.
  - Create a design of database to store information on students and books. Based on the Users issues they include specific details on what data must be entered into the system such as book return date!
- **System Builder:**
  - Builds the technical solution using software code, databases, networks and implements security

Example of Stakeholders

- System Analyst:
  - Conveys business requirements from System Owners and System Users to technical stakeholders such as System Designers and Builders and vice versa! Bridges the Gap!!!
  - E.g. System User to System Analyst: I want a system that will reduce the amount of time to check out a book for a student.
  - Currently we must check a “Students” log book to ensure the student has no fines.
  - We must then enter the book details into a the “Book Checkout” log book.
- System Analyst to System Designer:
  - Need to store student information and book information in a system.
  - Checking out a book requires examining a students record first for fines so having a quick and easy student information lookup is essential.
  - Also, linking the student information to checked out books must be included in the design

Organizational Roles/Responsibilities in System Developments:

- IS manager
- System Analyst
- Programmers
- Business managers
- Other IS managers/ Technicians
  - Database Administrator
  - Network and Telecommunication Experts
  - Human Factors specialists
  - Internal Audits

System Analyst :

- System analyst is a key person in the system development process. It is a organizational role most responsible for the analysis and design of information systems.
- System analyst is a specialist who studies the problems and needs of an organization to determine how people ,data, process and information technology can be best accomplish improvements for the business. System analyst is the person who guides through the development of an information system .
- In performing these tasks the system analyst must always match the information systems objectives with the goals of the organization.
- System analyst performs analysis and design based upon :
- Understanding of organization’s objectives ,structures and process
- Knowledge of how to exploit information technology for advantage

Roles of System Analyst:

- The main role of the system analyst is to perform system analysis and to prepare analysis model which clearly describes how the system works and how a requirements model must do for the new system.
- In order to perform these roles system analyst must need to discuss with user what they required from the system.
- **Review the specification :** The analyst reviews and refines the specifications used to develop or write computer software. The analyst is expected to maintain an expert level of knowledge about the software programs, so they can provide advice and guidance about what is possible and how long it will take.

Roles of System Analyst:

- **Review Test and Document:** System analyst reviews the test , System testing forms a large part of an information systems analyst's daily work. There are two types of testing functional and quality. And Parallel ,system analyst prepares proper documentations. Documentation is the process of writing down the steps for using the software. Technical documentation typically includes a list of all the data tables accessed , methodology used and a quick summary of the program logic.
- **System analysis , design and Developing :** It includes system's study in order to get facts about business activity. It is about getting information and determining requirements. Here the responsibility includes requirement determination and also the design of the system .In some cases ,analyst should work closely with the development team and programmers.

Skills of Successful System Analyst:

- Analytical skills
  - Understanding of organizations
  - Problem solving skills
  - System thinking
    - Ability to see organizations and information system as system
- Technical Skills
  - Understanding of potential and limitations of technology
  - Computer Network
  - Programming
  - Database
- Management skills
  - Ability to manage projects ,resources , risk and change
  - Resource management
  - Project management
  - Risk management
  - Change Management
- Interpersonal Skills
  - Communication skill
    - Interviewing and listening
    - Written and oral presentation
  - Working alone and with a team
  - Facilitating the groups
  - Managing the expectations

A Modern Approach to Systems Analysis and Design

- 1950s: focus on efficient automation of existing processes
- 1960s: advent of procedural third generation languages (3GL) faster and more reliable computers
- 1970s: system development becomes more like an engineering discipline
- 1980s: major breakthrough with 4GL, CASE tools, object-oriented methods
- 1990s: focus on system integration, GUI applications, client/server platforms, Internet
- The new century: Web application development, wireless PDAs and smart phones, component-based applications, application service providers (ASP)

System Development Methodology

System Development Methodology is a standard process followed in an organization to conduct all the steps necessary to analyze, design, implement, and maintain information systems

System Development Life Cycle (SDLC)

- SDLC is a methodology used to develop, maintain and replace information system.
- SDLC is a collective framework for activities performed to create information system.
- Also SDLC is Series of steps used to manage the phases of development for an information system.
- SDLC is Software Development life cycle or also called, as System Development Life Cycle a process for creating or modifying information system.
- It is also referred to as Application development life cycle it represents the model and methodologies that people use to develop the systems.

System Development Life Cycle (SDLC)

- Traditional methodology used to develop, maintain, and replace information systems.
- Phases in SDLC:
  - Planning
  - Analysis
  - Design
  - Implementation
  - Maintenance

Standard and Evolutionary Views of SDLC

Figure 1-3 The systems development life cycle

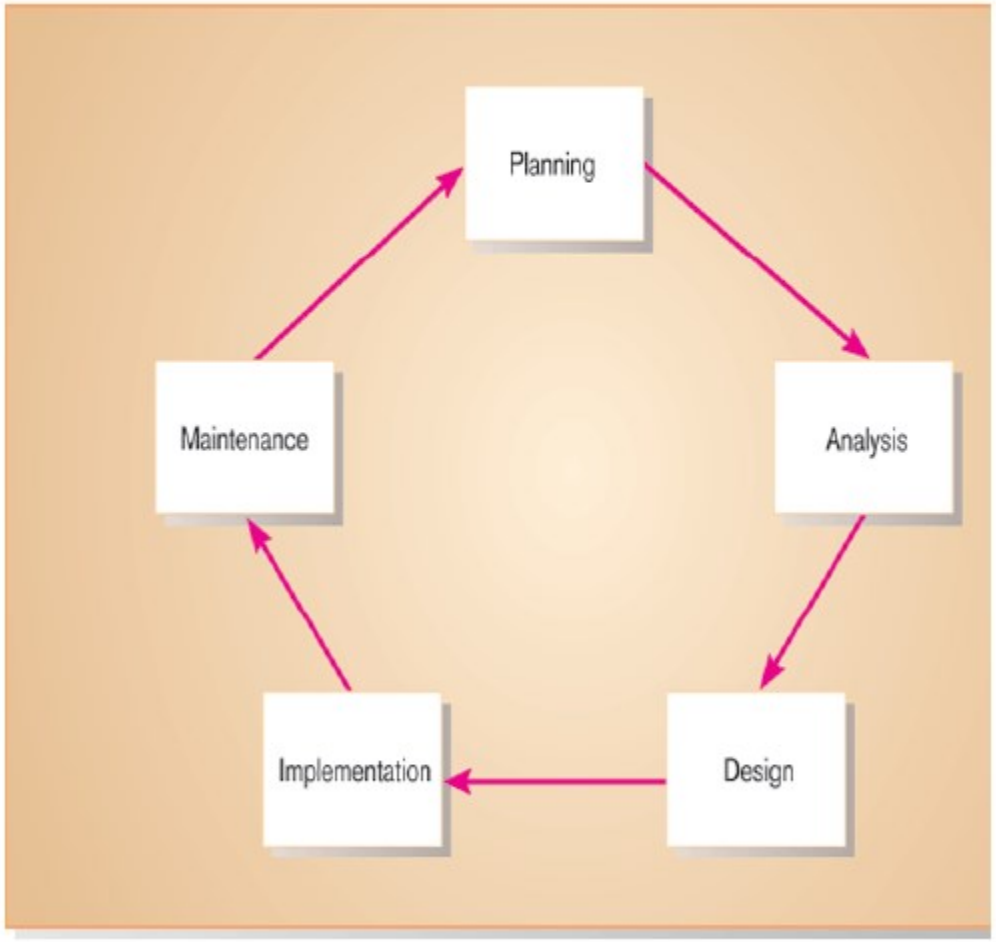
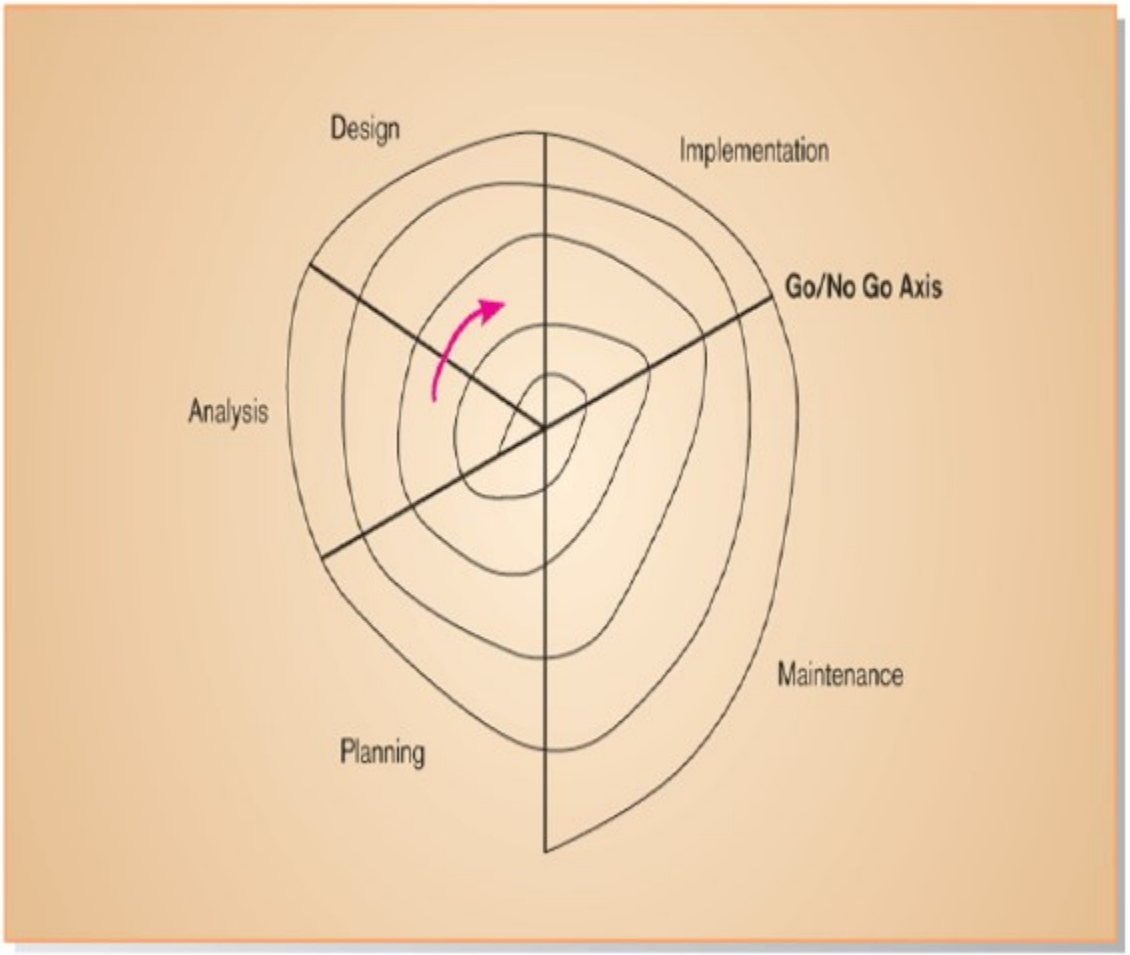


Figure 1-4 Evolutionary model SDLC



System Development Life Cycle (SDLC)

- **Planning** – an organization's total information system needs are identified, analyzed, prioritized, and arranged
- **Analysis** – system requirements are studied and structured
- **Design** – a description of the recommended solution is converted into logical and then physical system specifications
- **Logical design** – all functional features of the system chosen for development in analysis are described independently of any computer platform
- **Physical design** – the logical specifications of the system from logical design are transformed into the technology-specific details from which all programming and system construction can be accomplished
- **Implementation** – the information system is coded, tested, installed and supported in the organization
- **Maintenance** – an information system is systematically repaired and improved

Products of SDLC Phases

TABLE 1-1 Products of SDLC Phases

Phase	Products, Outputs, or Deliverables
Planning	Priorities for systems and projects; an architecture for data, networks, and selection hardware, and information systems management are the result of associated systems
	Detailed steps, or work plan, for project
	Specification of system scope and planning and high-level system requirements or features
	Assignment of team members and other resources
	System justification or business case
Analysis	Description of current system and where problems or opportunities are with a general recommendation on how to fix, enhance, or replace current system
	Explanation of alternative systems and justification for chosen alternative
Design	Functional, detailed specifications of all system elements (data, processes, inputs, and outputs)
	Technical, detailed specifications of all system elements (programs, files, network, system software, etc.)
	Acquisition plan for new technology
Implementation	Code, documentation, training procedures, and support capabilities
Maintenance	New versions or releases of software with associated updates to documentation, training, and support

The Heart of the Systems Development Process

FIGURE 1-8
Analysis–design–code–test loop

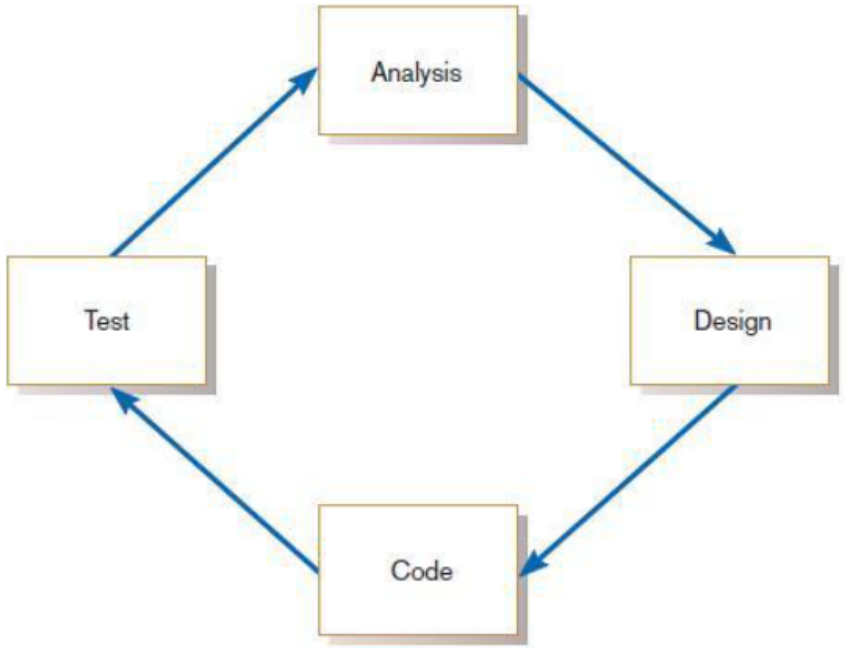
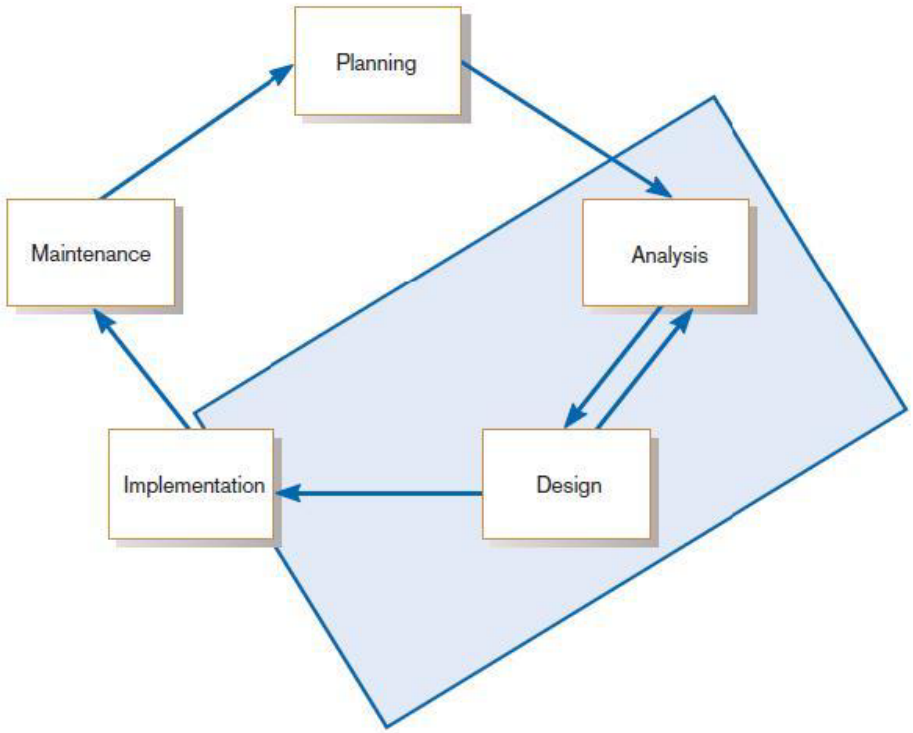


FIGURE 1-9
The heart of systems development



Current practice combines analysis, design, and implementation into a single iterative and parallel process of activities.

Different Lifecycle Models

- A software life cycle model (also called process model) is a descriptive and diagrammatic representation of the software life cycle.
- A life cycle model represents all the activities required to make a software product transit through its life cycle phases. It also captures the order in which these activities are to be undertaken.
- In other words, a life cycle model maps the different activities performed on a software product from its inception to retirement.
- Different life cycle models may map the basic development activities to phases in different ways.

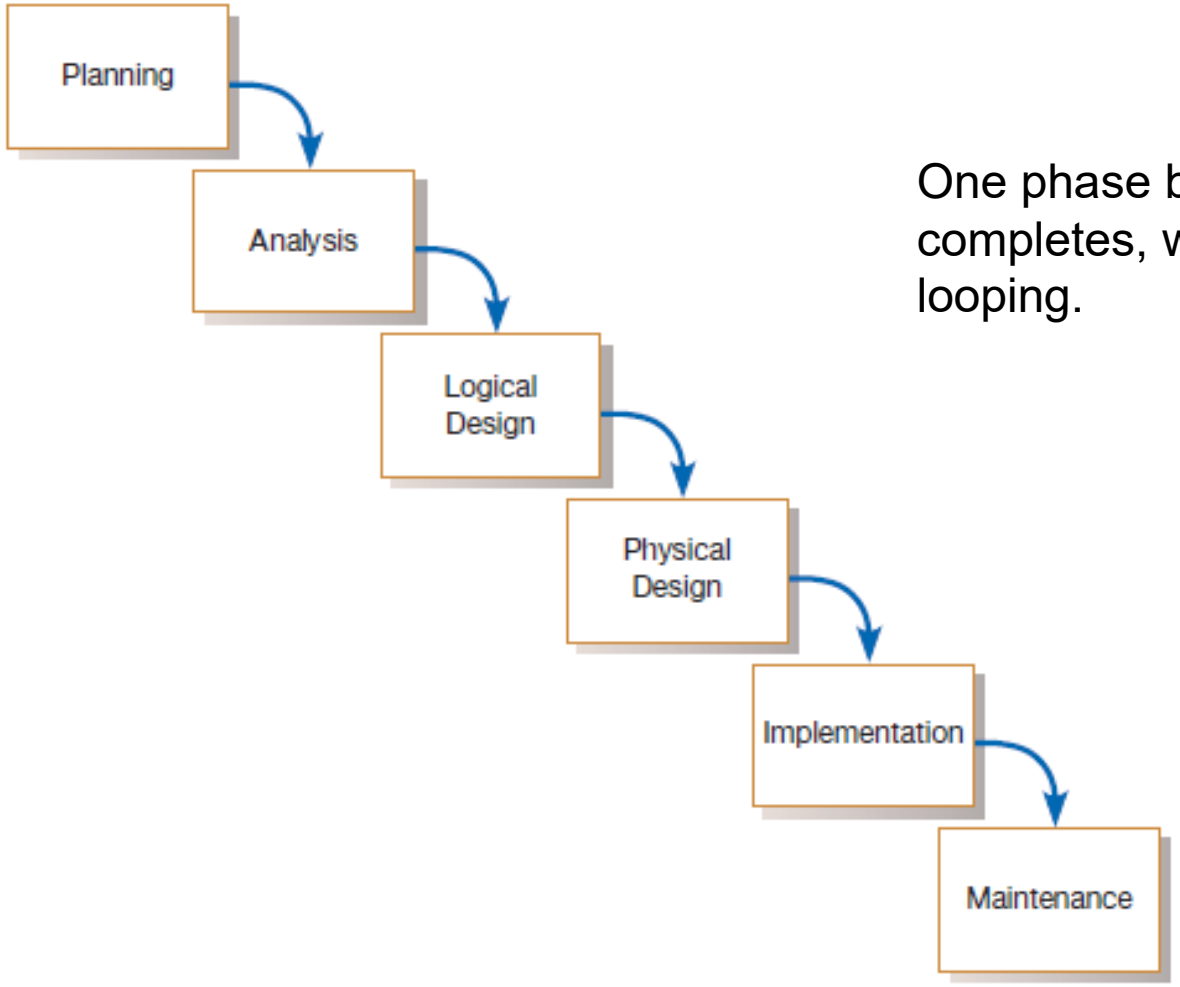
Different Lifecycle Models

- A few important and commonly used life cycle models are as follows:
  - Traditional Waterfall model
  - Iterative Waterfall Model
  - V-Shaped Model
  - Incremental/Evolutionary model
  - Prototyping model
  - Spiral model
  - Rapid Application Development(RAD) Model
  - Object Oriented Based : Rational Unified Process (RUP)

Different Approaches to Improving Development

- Attempts to make systems development less of an art and more of a science are usually referred to as **systems engineering or software engineering**
  - CASE TOOLS
  - Rapid Application Development
  - Service-Oriented Architecture
  - Agile Methodologies
  - eXtreme Programming
  - Object-Oriented Analysis And Design

Traditional Waterfall Model



One phase begins when another completes, with little backtracking and looping.

FIGURE 1-10
Traditional waterfall SDLC

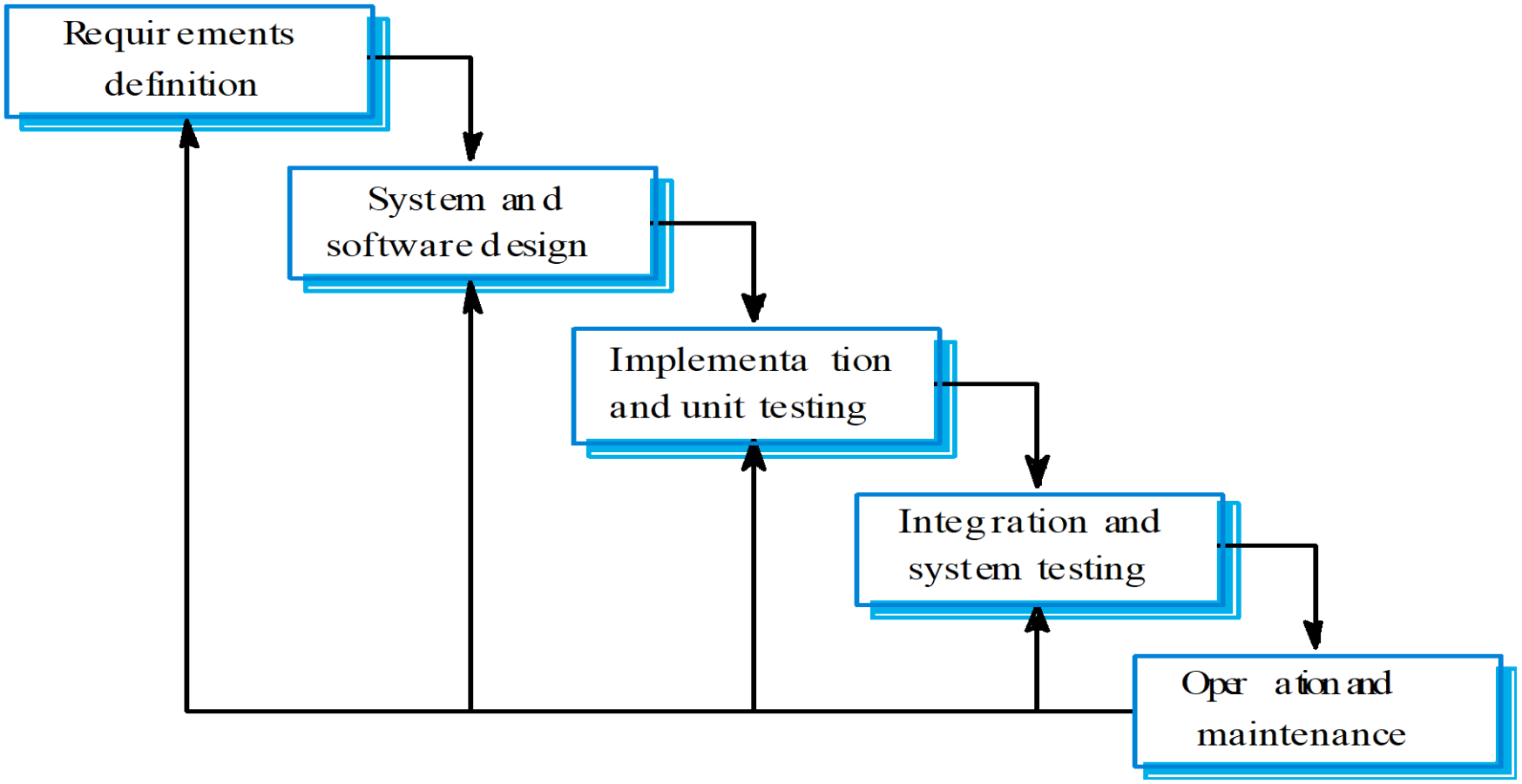
Traditional Waterfall Model

- This model can be considered to be a *theoretical way of developing software*. But all other life cycle models are essentially derived from the classical waterfall model. So, in order to be able to appreciate other life cycle models it is necessary to learn the classical waterfall model.
- The Waterfall Model was first Process Model to be introduced.
- It is also referred to as a **linear-sequential life cycle model**. It is very simple to understand and use.
- In a waterfall model, each phase must be completed fully before the next phase can begin.
- At the end of each phase, a review takes place to determine if the project is on the right path and whether or not to continue or discard the project. In waterfall model phases do not overlap.
- **Advantages of waterfall model:**
- Simple and easy to understand and use.
- Easy to manage due to the rigidity of the model – each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.

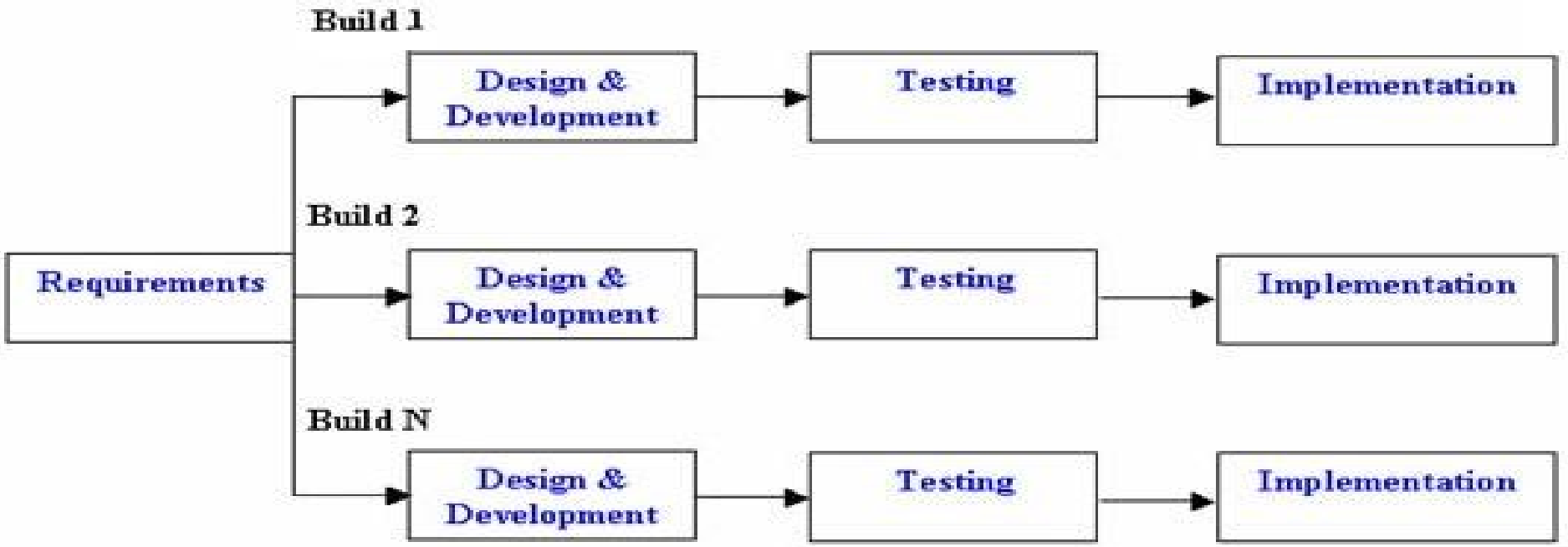
Traditional Waterfall Model

- **Problems with Waterfall Approach:**
  - Feedback ignored, milestones lock in design specs even when conditions change
  - Limited user involvement (only in requirements phase)
  - Too much focus on milestone deadlines of SDLC phases to the detriment of sound development practices
  - Once an application is in the testing stage, it is very difficult to go back and change something that was not well-thought out in the concept stage.
  - No working software is produced until late during the life cycle.
  - High amounts of risk and uncertainty.
  - Not a good model for complex and object-oriented projects.
  - Poor model for long and ongoing projects.
  - Not suitable for the projects where requirements are at a moderate to high risk of changing.
- **When to use the waterfall model:**
  - Requirements are very well known, clear and fixed.
  - Product definition is stable.
  - Technology is understood.
  - There are no ambiguous requirements
  - Ample resources with required expertise are available freely
  - The project is short.

Iterative Waterfall



Incremental Model



Incremental Life Cycle Model

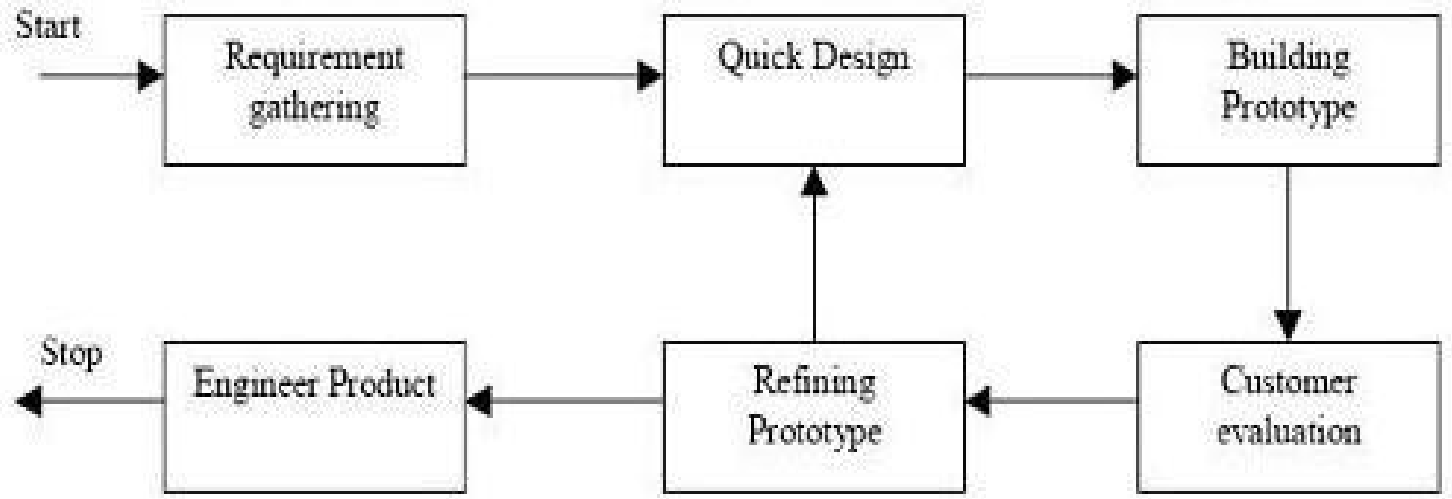
Incremental Model

- In incremental model the whole requirement is divided into various builds. Multiple development cycles take place here, making the life cycle a “multi-waterfall” cycle.
- Cycles are divided up into smaller, more easily managed modules.
- Each module passes through the requirements, design, implementation and testing phases.
- A working version of software is produced during the first module, so you have working software early on during the software life cycle.
- Each subsequent release of the module adds function to the previous release.
- The process continues till the complete system is achieved.

- **Advantages of Incremental model:**
  - Generates working software quickly and early during the software life cycle.
  - More flexible – less costly to change scope and requirements.
  - Easier to test and debug during a smaller iteration.
  - Customer can respond to each built.
  - Lowers initial delivery cost.
  - Easier to manage risk because risky pieces are identified and handled during it'd iteration.

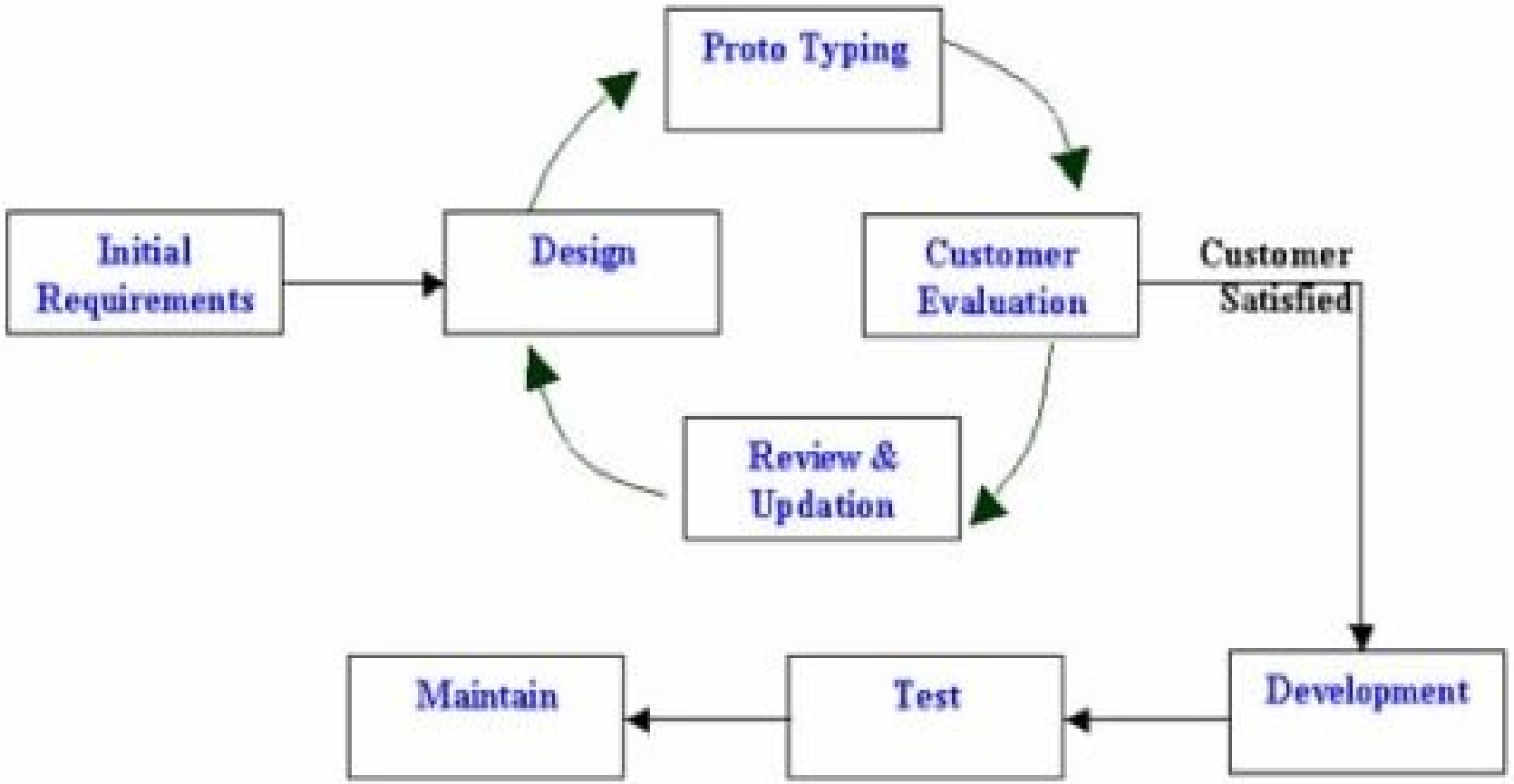
- **Disadvantages of Incremental model:**
  - Needs good planning and design.
  - Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
  - Total cost is higher than waterfall.
- **When to use the Incremental model:**
  - Requirements of the complete system are clearly defined and understood.
  - Major requirements must be defined; however, some details can evolve with time.
  - There is a need to get a product to the market early.
  - A new technology is being used
  - Resources with needed skill set are not available
  - There are some high risk features and goals.

Prototyping Model



Prototyping Model

Prototyping Model



Proto Type Model

Prototyping

- **Prototyping:** An iterative process of systems development in which requirements are converted to a working system that is continually revised through close collaboration between an analyst and users.
- Prototyping will enable you to quickly convert basic requirements into a working, though limited, version of the desired information system. The prototype will then be viewed and tested by the user.
- Typically, seeing verbal descriptions of requirements converted into a physical system will prompt the user to modify existing requirements and generate new ones.
- As the prototype changes through each iteration, more and more of the design specifications for the system are captured in the prototype.
- The prototype can then serve as the basis for the production system in a process called **evolutionary prototyping**.

Prototyping

- Alternatively, the prototype can serve only as a model, which is then used as a reference for the construction of the actual system. In this process, **called throwaway prototyping**, the prototype is discarded after it has been used.

Prototyping Model

A prototype usually turns out to be a very crude version of the actual system.

A prototype is a toy implementation of the system. A prototype usually exhibits limited functional capabilities, low reliability, and inefficient performance compared to the actual software

- A prototype of the actual product is preferred in situations such as:
  - user requirements are not complete
  - technical issues are not clear

- **Need for a prototype in system development**
- There are several uses of a prototype. An important purpose is to illustrate the input data formats, messages, reports, and the interactive dialogues to the customer.
- This is a valuable mechanism for gaining better understanding of the customer's needs:
  - how the screens might look like
  - how the user interface would behave
  - how the system would produce outputs

Prototyping Model

A prototyping model can be used when technical solutions are unclear to the development team.

A developed prototype can help engineers to critically examine the technical issues associated with the product development

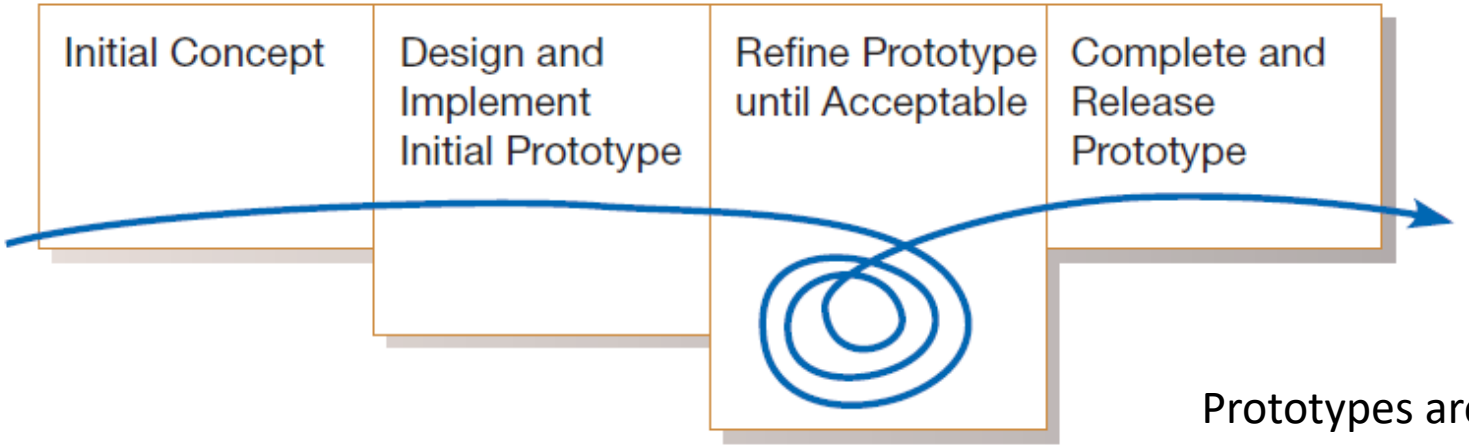
- **Advantages of Prototype model:**
  - Users are actively involved in the development
  - Since in this methodology a working model of the system is provided, the users get a better understanding of the system being developed.
  - Errors can be detected much earlier.
  - Quicker user feedback is available leading to better solutions.
  - Missing functionality can be identified easily
  - Confusing or difficult functions can be identified Requirements validation, Quick implementation of, incomplete, but functional, application.

- **Disadvantages of Prototype model:**
  - Leads to implementing and then repairing way of building systems.
  - Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
  - Incomplete application may cause application not to be used as the full system was designed
  - Incomplete or inadequate problem analysis.

- **Need for a prototype in system development**
- There are several uses of a prototype. An important purpose is to illustrate the input data formats, messages, reports, and the interactive dialogues to the customer.
- This is a valuable mechanism for gaining better understanding of the customer’s needs:
  - how the screens might look like
  - how the user interface would behave
  - how the system would produce outputs

Evolutionary Prototyping

- In evolutionary prototyping, you begin by modeling parts of the target system and, if the prototyping process is successful, you evolve the rest of the system from those parts .
- A life-cycle model of evolutionary prototyping illustrates the iterative nature of the process and the tendency to refine the prototype until it is ready to release



Prototypes are designed to handle only the typical cases, so exception handling must be added to the prototype as it is converted to the production system. Clearly, the prototype captures only part of the system requirements.

Throwaway Prototyping

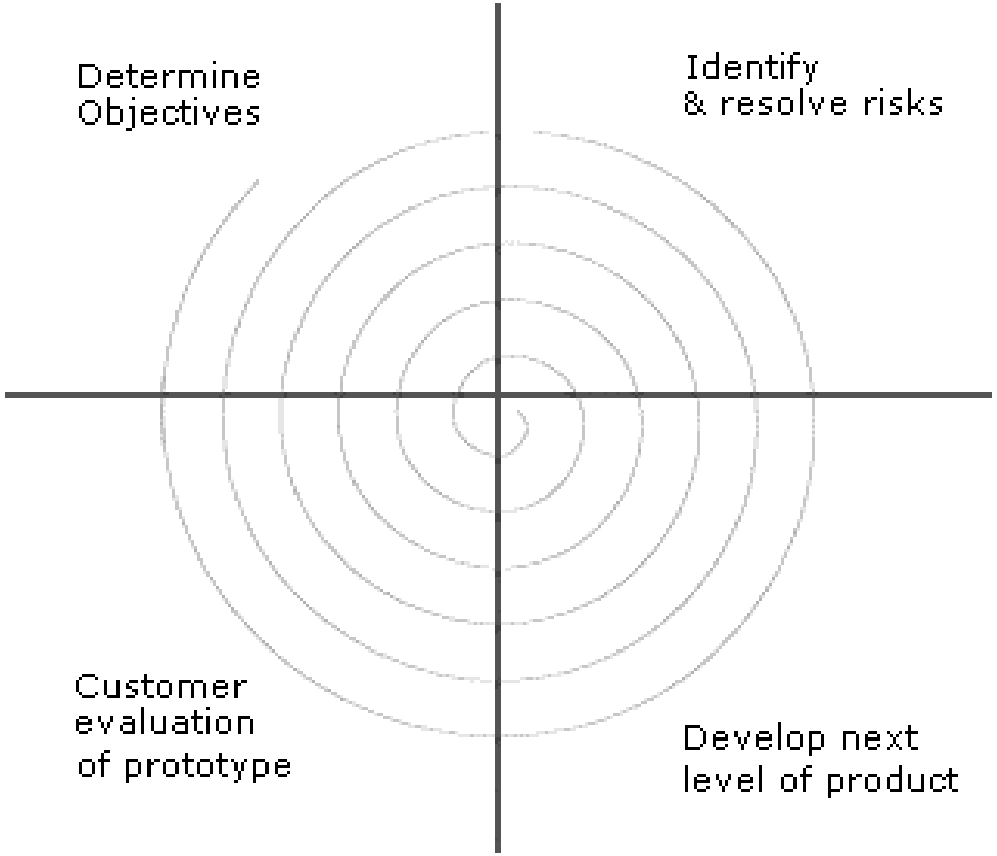
- Unlike evolutionary prototyping, throwaway prototyping does not preserve the prototype that has been developed. With throwaway prototyping, there is never any intention to convert the prototype into a working system.
- Instead, the prototype is developed quickly to demonstrate some aspect of a system design that is unclear or to help users decide among different features or interface characteristics.
- Once the uncertainty the prototype was created to address has been reduced, the prototype can be discarded, and the principles learned from its creation and testing can then become part of the requirements determination.

Spiral Model

- The Spiral model of software development is shown in fig. below.
- The diagrammatic representation of this model appears like a spiral with many loops. The exact number of loops in the spiral is not fixed. Each loop of the spiral represents a phase of the software process.
- For example, the innermost loop might be concerned with feasibility study.
- The next loop with requirements specification, the next one with design, and so on. Each phase in this model is split into four sectors (or quadrants) as shown in fig..

- The spiral model is called a meta model since it encompasses all other life cycle models.
- Risk handling is inherently built into this model.
- The spiral model is suitable for development of technically challenging software products that are prone to several kinds of risks. However, this model is much more complex than the other models – this is probably a factor deterring its use in ordinary projects.
- The following activities are carried out during each phase of a spiral model

Spiral Model



- **First quadrant (Objective Setting)**
  - During the first quadrant, it is needed to identify the objectives of the phase.
  - Examine the risks associated with these objectives.
- **Second Quadrant (Risk Assessment and Reduction)**
  - A detailed analysis is carried out for each identified project risk.
  - Steps are taken to reduce the risks. For example, if there is a risk that the requirement are inappropriate, a prototype system may be developed.
- **Third Quadrant (Development and Validation)**
  - Develop and validate the next level of the product after resolving the identified risks.
- **-Fourth Quadrant (Review and Planning)**
  - Review the results achieved so far with the customer and plan the next iteration around the spiral.
  - Progressively more complete version of the software gets built with each iteration around the spiral.

- **Advantages of Spiral model:**
  - High amount of risk analysis hence, avoidance of Risk is enhanced.
  - Good for large and mission-critical projects.
  - Strong approval and documentation control.
  - Additional Functionality can be added at a later date.
  - Software is produced early in the software life cycle.
- **Disadvantages of Spiral model:**
  - Can be a costly model to use.
  - Risk analysis requires highly specific expertise.
  - Project's success is highly dependent on the risk analysis phase.
  - Doesn't work well for smaller projects.

- **When to use Spiral model:**
  - When costs and risk evaluation is important
  - For medium to high-risk projects
  - Long-term project commitment unwise because of potential changes to economic priorities
  - Users are unsure of their needs
  - Requirements are complex
  - New product line
  - Significant changes are expected (research and exploration)

Approaches of Improving Development

- CASE Tools
- Rapid Application Development;
- Service Oriented Architecture(SOA),
- Agile Methodology,
- eXtreme Programming,
- Object Oriented Analysis and Design

Rapid Application Development (RAD)

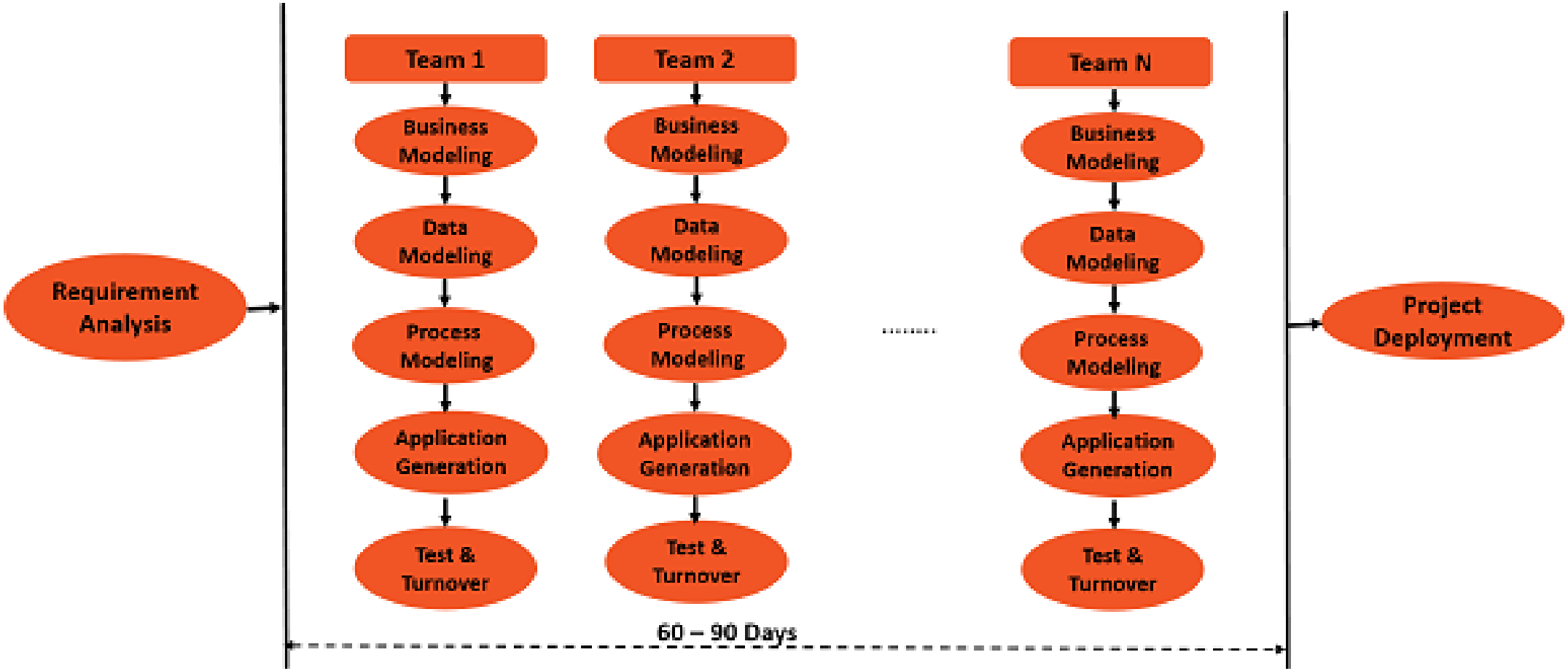
- is a form of agile software development methodology that prioritizes rapid prototype releases and iterations.
- Unlike the Waterfall method, RAD emphasizes the use of software and user feedback over strict planning and requirements recording.
- software development methodology that uses **minimal planning** in favor of **rapid prototyping**
- is **based on prototyping and iterative development** with no specific planning involved.
- The process of writing the software itself involves the planning required for developing the product.
- RAD (Rapid Application Development) is a concept that products can be developed faster and of higher quality through:
  - Gathering requirements using workshops or focus groups
  - Prototyping and early, reiterative user testing of designs
  - The re-use of software components
  - A rigidly paced schedule that refers design improvements to the next product version
  - Less formality in reviews and other team communication

Rapid Application Development (RAD)

- Decreases design and implementation time
- Involves: extensive user involvement, prototyping, integrated CASE tools, code generators
- More focus on user interface and system function, less on detailed business analysis and system performance

- Rapid Application Development (RAD) model has the following phases
  - **Requirements Planning phase** − In the requirements planning phase, a workshop needs to be conducted to discuss business problems in a structured manner.
  - **User Description phase** − In the User Description phase, automated tools are used to capture information from users.
  - **Construction phase** − In the Construction phase, productivity tools, such as code generators, screen generators, etc. are used inside a time-box, with a “Do until Done” approach.
  - **Cut Over phase** − In the Cut over phase, installation of the system, user acceptance testing and user training are performed.

Rapid Application Development Model



Rapid Application Development (RAD)

- RAD (Rapid Application Development) is a concept that products can be developed faster and of higher quality through:
- Rapid Application Development focuses on
  - gathering customer requirements through
    - workshops or
    - focus groups,
  - early testing of the prototypes by the customer using iterative concept,
  - reuse of the existing prototypes (components),
  - continuous integration and
  - rapid delivery.
  - Less formality in reviews and other team communication
  - A rigidly paced schedule that refers design improvements to the next product version

Rapid Application Development (RAD)

- **A prototype** is a working model that is functionally equivalent to a component of the product.
- In the RAD model, the functional modules are developed in parallel as prototypes and are integrated to make the complete product for faster product delivery.
- Since there is no detailed preplanning, it makes it easier to incorporate the changes within the development process.
- The most important aspect for this model to be successful is to make sure that **the prototypes developed are reusable**

Rapid Application Development Model

- RAD model is Rapid Application Development model.
- It is a type of incremental model.
- In RAD model the components or functions are developed in parallel as if they were mini projects.
- The developments are time boxed, delivered and then assembled into a working prototype.
- This can quickly give the customer something to see and use and to provide feedback regarding the delivery and their requirements.

- The phases in the rapid application development (RAD) model are:
- **Business modeling:** The information flow is identified between various business functions.
- **Data modeling:** Information gathered from business modeling is used to define data objects that are needed for the business.
- **Process modeling:** Data objects defined in data modeling are converted to achieve the business information flow to achieve some specific business objective. Description are identified and created for CRUD of data objects.
- **Application generation:** Automated tools are used to convert process models into code and the actual system.
- **Testing and turnover:** Test new components and all the interfaces.

- **Advantages of the RAD model:**
  - Reduced development time.
  - Increases reusability of components
  - Quick initial reviews occur
  - Encourages customer feedback
  - Integration from very beginning solves a lot of integration issues.
- **Disadvantages of RAD model:**
  - Depends on strong team and individual performances for identifying business requirements.
  - Only system that can be modularized can be built using RAD
  - Requires highly skilled developers/designers.
  - High dependency on modeling skills
  - Inapplicable to cheaper projects as cost of modeling and automated code generation is very high.
- **When to use RAD model:**
  - RAD should be used when there is a need to create a system that can be modularized in 2-3 months of time.
  - It should be used if there's high availability of designers for modeling and the budget is high enough to afford their cost along with the cost of automated code generating tools.
  - RAD SDLC model should be chosen only if resources with high business knowledge are available and there is a need to produce the system in a short span of time (2-3 months).

Joint Application Design (JAD)

- **A structured process in which users, managers, and analysts work together for several days in a series of intensive meetings to specify or review system requirements**.
- Joint Application Design (**JAD**) started in the late 1970s at IBM, and since then the practice of JAD has spread throughout many companies and industries.
- The main idea behind JAD is to bring together the **key users**, **managers**, and **systems analysts** involved in the analysis of a current system.
- The primary purpose of using JAD in the analysis phase is to collect systems requirements simultaneously from the key people involved with the system. The result is an intense (extreme and forceful or (of a feeling) very strong) and structured, but highly effective, process.
- As with a group interview, having all the key people together in one place at one time allows analysts to see where there are areas of agreement and where there are conflicts.
- Meeting with all of these important people for over a week of intense **sessions** allows you the opportunity to resolve conflicts, or at least to understand why a conflict may not be simple to resolve.

Joint Application Design (JAD)

- **JAD sessions** are usually conducted at a location other than the place where the people involved normally work. The idea behind such a practice is **to keep participants away from as many distractions** as possible so that they can concentrate on systems analysis.
- A JAD may last anywhere from **four hours to an entire week** and may consist of several sessions. **A JAD employs thousands of dollars of corporate resources**,
- the most expensive of which is the time of the people involved. Other expenses include the costs associated with flying people to a remote site and putting them up in hotels and feeding them for several days.

Joint Application Design (JAD)

- The typical participants in a JAD are listed below:
- **JAD session leader**: The JAD session leader organizes and runs the JAD. This person has been trained in group management and facilitation as well as in systems analysis. The JAD leader sets the agenda and sees that it is met; he or she remains neutral on issues and does not contribute ideas or opinions, but rather concentrates on keeping the group on the agenda, resolving conflicts and disagreements, and soliciting (manage) all ideas.
- **Users:** The key users of the system under consideration are vital participants in a JAD. They are the only ones who have a clear understanding of what it means to use the system on a daily basis.

Joint Application Design (JAD)

- **Managers**: Managers of the work groups who use the system in question provide insight into new organizational directions, motivations for and organizational impacts of systems, and support for requirements determined during the JAD.
- **Sponsor**: As a major undertaking due to its expense, a JAD must be sponsored by someone at a relatively high level in the company. If the sponsor attends any sessions, it is usually only at the very beginning or the end.
- **Systems analysts**: Members of the systems analysis team attend the JAD, although their actual participation may be limited. Analysts are there to learn from users and managers, not to run or dominate the process.

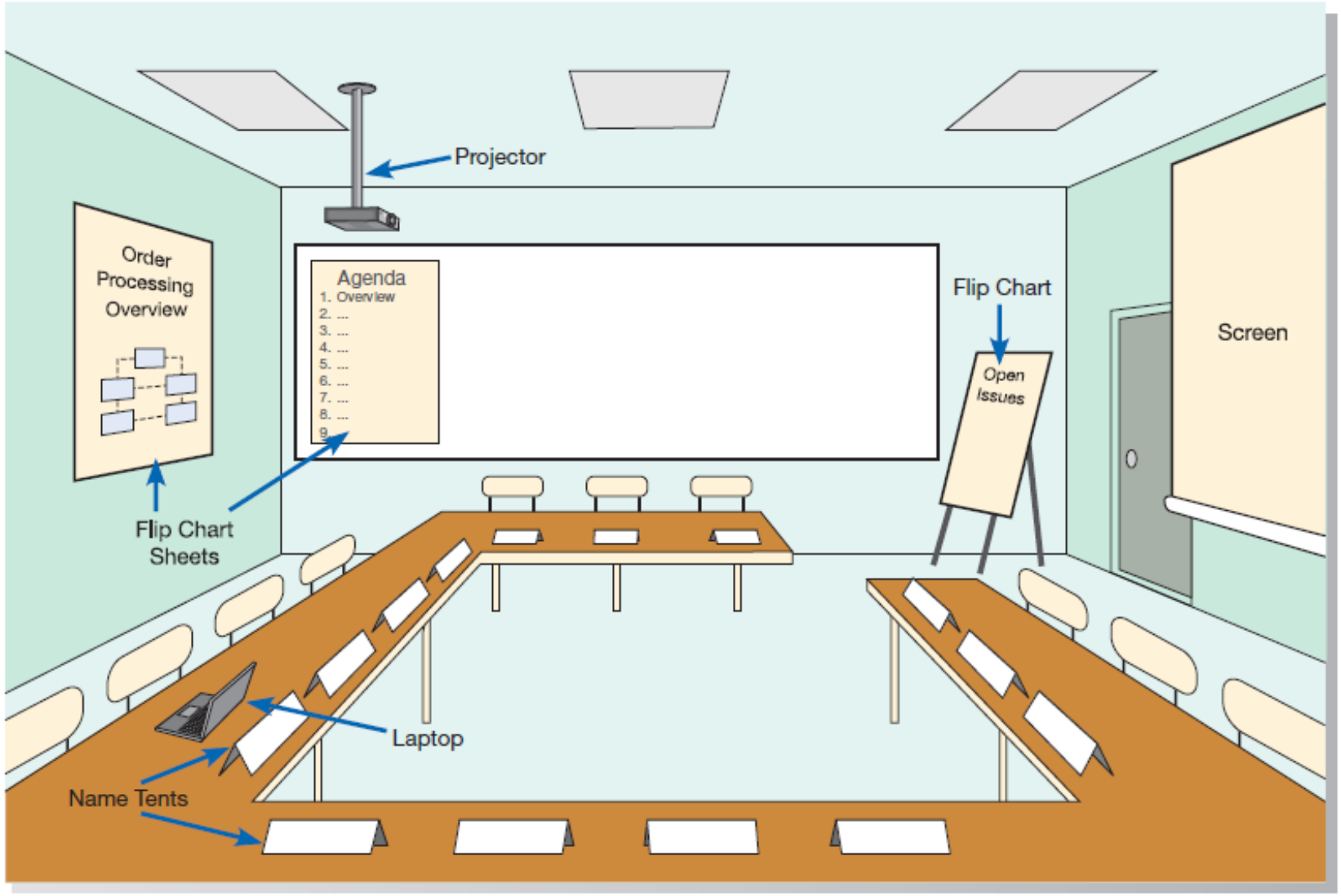
Joint Application Design (JAD)

- **Scribe**: The scribe **takes notes during the JAD sessions**. This is usually done on a laptop. Notes may be taken using a word processor, or notes and diagrams may be entered directly into a **CASE tool**.
- **IS staff**: Besides systems analysts, other **information systems** (IS) staff, such as **programmers**, **database analysts**, **IS planners**, and **data center personnel,** may attend to learn from the discussion and possibly contribute their ideas on the technical feasibility of proposed ideas or the technical limitations of current systems.

Joint Application Design (JAD)

- **JAD sessions** are usually held in special-purpose rooms where participants sit around **horseshoe-shaped** tables
- These rooms are typically equipped with whiteboards. Other audiovisual tools may be used, such as **magnetic symbols** that can be easily **rearranged on a whiteboard**, flip charts, and computer-generated displays. **Flip-chart paper is typically used for keeping track of issues that cannot be resolved** during the JAD or for those issues requiring additional information that can be gathered during breaks in the proceedings. Computers may be used to create and display form or
- report designs, diagram existing or replacement systems, or create prototypes.

Joint Application Design (JAD)



Joint Application Design (JAD)

- When a JAD is completed, the end result is a set of documents that detail the workings of the current system related to the study of a replacement system.
- Depending on the exact purpose of the JAD, analysts may also walk away from the JAD with some detailed information on what is desired of the replacement system.
- **CASE Tools During JAD:** For requirements determination and structuring, the most useful CASE tools are used for diagramming and form and report generation.

- Some observers advocate using CASE tools during JADs.
- Running a CASE tool during a JAD enables analysts to enter system models directly into a CASE tool, providing consistency and reliability in the joint model-building process.

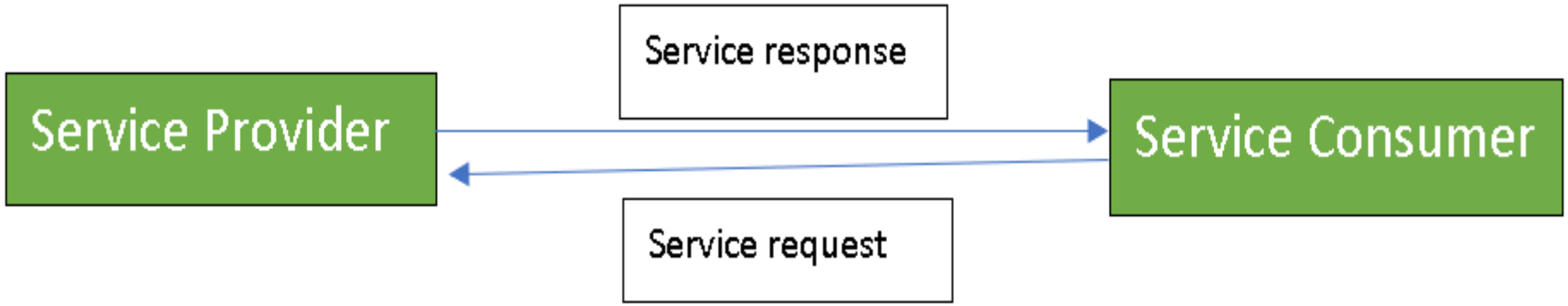
- The CASE tool captures system requirements in a **more flexible** and **useful** way than can a scribe or an analysis team **making notes**. Further, the CASE tool can be used to project **menu**, **display**, and **report designs**, so users can directly observe **old** and new **designs** and evaluate their usefulness for the analysis team.

- **Advantage of JAD**
- JAD allows you to resolve difficulties more simply and produce better, error-free software.
- The joint collaboration between the company and the clients lowers all risks.
- JAD reduces costs and time needed for project development.
- Well-defined requirements improve system quality.
- Due to the close communication, progress is faster.
- JAD encourages the team to push each other to work faster and deliver on time.

- **Disadvantage of JAD**
- Different opinions within the team make it difficult to align goals and maintain focus
- Depending on the size of the project , JAD may require a significant time commitment.

Service Oriented Architecture(SOA)

- Service-Oriented Architecture (SOA) is a stage in the evolution of application development and/or integration.
- It defines a way to make software components **reusable** using the interfaces.
- Formally, SOA is an architectural approach in which
  - applications make use of services available in the network.
  - services are provided to form applications, through a network call over the internet.
  - uses common communication standards to speed up and streamline the service integrations in applications.
  - Each service in SOA is a complete business function in itself.
  - The services are published in such a way that it makes it easy for the developers to assemble their apps using those services.
- SOA allows users to combine a large number of facilities from existing services to form applications.
- SOA encompasses a set of design principles that structure system development and provide means for integrating components into a coherent and decentralized system.
- SOA-based computing packages functionalities into a set of interoperable services, which can be integrated into different software systems belonging to separate business domains.



Service provider:

- The service provider is the maintainer of the service and the organization that makes available one or more services for others to use.
- To advertise services, the provider can publish them in a registry, together with a service contract that specifies the nature of the service, how to use it, the requirements for the service, and the fees charged.

Service consumer:

- The service consumer can locate the service metadata in the registry and develop the required client components to bind and use the service.

•Advantages of SOA

•**Service reusability:** In SOA, applications are made from existing services. Thus, services can be reused to make many applications.

•**Easy maintenance:** As services are independent of each other they can be updated and modified easily without affecting other services.

•**Platform independent:** SOA allows making a complex application by combining services picked from different sources, independent of the platform.

•**Availability:** SOA facilities are easily available to anyone on request.

•**Reliability:** SOA applications are more reliable because it is easy to debug small services rather than huge codes

•**Scalability:** Services can run on different servers within an environment, this increases scalability

•Disadvantages of SOA

•**High overhead:** A validation of input parameters of services is done whenever services interact this decreases performance as it increases load and response time.

•**High investment:** A huge initial investment is required for SOA.

•**Complex service management:** When services interact they exchange messages to tasks. the number of messages may go in millions. It becomes a cumbersome task to handle a large number of messages.

Practical Application of SOA

1.SOA infrastructure is used by many armies and air forces to deploy situational awareness systems.

2.SOA is used to improve healthcare delivery.

3.Nowadays many apps are games and they use inbuilt functions to run. For example, an app might need GPS so it uses the inbuilt GPS functions of the device. This is SOA in mobile solutions.

4.SOA helps maintain museums a virtualized storage pool for their information and content.

Agile Methodologies

- Motivated by recognition of software development as fluid, unpredictable, and dynamic
- Three key principles
  - Adaptive rather than predictive
  - Emphasize people rather than roles
  - Self-adaptive processes

TABLE 1-3 The Agile Manifesto

The Manifesto for Agile Software Development

Seventeen anarchists agree:

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

- *Individuals and interactions* over processes and tools.
- *Working software* over comprehensive documentation.
- *Customer collaboration* over contract negotiation.
- *Responding to change* over following a plan.

That is, while we value the items on the right, we value the items on the left more.
We follow the following principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Businesspeople and developers work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Continuous attention to technical excellence and good design enhances agility.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

—Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, Ron Jeffries, Jon Kern, Brian Marick, Robert C. Martin, Steve Mellor, Ken Schwaber, Jeff Sutherland, Dave Thomas (*www.agileAlliance.org*)

(*Source: http://agilemanifesto.org/* © 2001, the above authors this declaration may be freely copied in any form, but only in its entirety through this notice.)

The Agile Methodologies group argues that software development methodologies adapted from engineering generally do not fit with real-world software development.

The Manifesto for Agile Software Development

Seventeen anarchists agree:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

That is, while we value the items on the right, we value the items on the left more.

We follow the following principles:

- Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
- Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
- Business people and developers work together daily throughout the project.
- Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
- The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity—the art of maximizing the amount of work not done—is essential.
- The best architectures, requirements, and designs emerge from self-organizing teams.
- At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Advantages of Agile Model:

- Is a very realistic approach to software development.
- Promotes teamwork and cross training.
- Functionality can be developed rapidly and demonstrated.
- Resource requirements are minimum.
- Suitable for fixed or changing requirements
- Enables concurrent development and delivery within an overall planned context.
- Frequent Delivery
- Face-to-Face Communication with clients.

Disadvantages:

- Not suitable for handling complex dependencies.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- lack of documentation.

When to use Agile Methodologies

- If your project involves:
  - Unpredictable or dynamic requirements
  - Responsible and motivated developers
  - Customers who understand the process and will get involved

TABLE 1-4 Five Critical Factors That Distinguish Agile and Traditional Approaches to Systems Development

Factor	Agile Methods	Traditional Methods
Size	Well matched to small products and teams. Reliance on tacit knowledge limits scalability.	Methods evolved to handle large products and teams. Hard to tailor down to small projects.
Criticality	Untested on safety-critical products. Potential difficulties with simple design and lack of documentation.	Methods evolved to handle highly critical products. Hard to tailor down to products that are not critical.
Dynamism	Simple design and continuous refactoring are excellent for highly dynamic environments but a source of potentially expensive rework for highly stable environments.	Detailed plans and Big Design Up Front, excellent for highly stable environment but a source of expensive rework for highly dynamic environments.
Personnel	Requires continuous presence of a critical mass of scarce experts. Risky to use no-agile people.	Needs a critical mass of scarce experts during project definition but can work with fewer later in the project, unless the environment is highly dynamic.
Culture	Thrives in a culture where people feel comfortable and empowered by having many degrees of freedom (thriving on chaos).	Thrives in a culture where people feel comfortable and empowered by having their roles defined by clear practices and procedures (thriving on order).

eXtreme Programming

- Extreme Programming (XP) is an agile software development framework that aims to produce higher quality software, and higher quality of life for the development team. XP is the most specific of the agile frameworks regarding appropriate engineering practices for software development.
- XP is a lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop a software

Major Features :

- Short, incremental development cycles
- Automated tests
- Two-person programming teams
- Coding, testing, listening, designing
- Coding and testing operate together

Advantages:

- Communication between developers
- High level of productivity
- High-quality code

Extreme Programming(Xps)

- **When XP is applicable?**
  - Dynamically changing software requirements
  - Risks caused by fixed time projects using new technology
  - Small, co-located extended development team
  - The technology you are using allows for automated unit and functional tests
- **What are the values of XP?**
  - Communication
    - face to face discussion with the aid of a white board or other drawing mechanism;
    - must be in communication with each other at all times
  - Simplicity
    - address only the requirements that you know about; don't try to predict the future
  - Feedback
    - builds something, gathers feedback on your design and implementation, and then adjust your product going forward.
  - Courage
    - effective action in the face of fear
  - Respect
    - respect each other in order to communicate with each other, provide and accept feedback that honors your relationship, and to work together to identify simple designs and solutions.

Advantages:

- Lightweight methods suit small-medium size projects.
- Produces good team cohesion.
- Emphasizes final product.
- Test based approach to requirements and quality assurance.
- Close contact with the customer
- Communication between developers
- High level of productivity
- High-quality code

Disadvantages

- Difficult to scale up to large projects where documentation is essential.
- Needs experience and skill if not to degenerate into code-and-fix.

Object-Oriented Analysis and Design (OOAD)

- Based on objects rather than data or processes
- **Object**: a structure encapsulating attributes and behaviors of a real-world entity
- **Object class**: a logical grouping of objects sharing the same attributes and behaviors
- **Inheritance**: hierarchical arrangement of classes enable subclasses to inherit properties of superclasses

Object-Oriented Analysis and Design (OOAD)

- The **object model** visualizes the elements in a software application in terms of objects.
- Object-Oriented Modelling (OOM) technique visualizes things in an application by using models organized around objects.
- Any software development approach goes through the following stages −
  - Analysis,
  - Design, and
  - Implementation.
- In **object-oriented software engineering**, the software developer identifies and organizes the application in terms of object-oriented concepts, prior to their final representation in any specific programming language or software tools.

Rational Unified Process(RUP)

- **A Process model of implementing the concept of Object oriented development.**
- **Rational Unified Process (RUP)** is a software development process for object-oriented models. It is also known as the **Unified Process Model**.
- Developed by I.Jacobson, G.Booch and J.Rumbaugh.
- Software engineering process with the goal of producing good quality maintainable software within specified time and budget.
- Developed through a series of fixed length mini projects called iterations.
- Maintained and enhanced by Rational Software Corporation and thus referred to as **Rational Unified Process (RUP).**
- The Rational Unified Process (RUP) is **iterative**, meaning repeating; and **agile**. Iterative because all of the process's core activities repeat throughout the project. The process is agile because various components can be adjusted, and phases of the cycle can be repeated until the software meets requirements and objectives.
- RUP reduces unexpected development costs and prevents wastage of resources.
- RUP has the following key characteristics:
  - Use-case driven from inception to deployment
  - Architecture-centric, where architecture is a function of user needs
  - Iterative and incremental, where large projects are divided into smaller projects

Rational Unified Process (RUP)

- An object-oriented systems development methodology
- Establishes four phase of development: inception, elaboration, construction, and transition
- Each phase is organized into a number of separate iterations

Rational Unified Process (RUP)

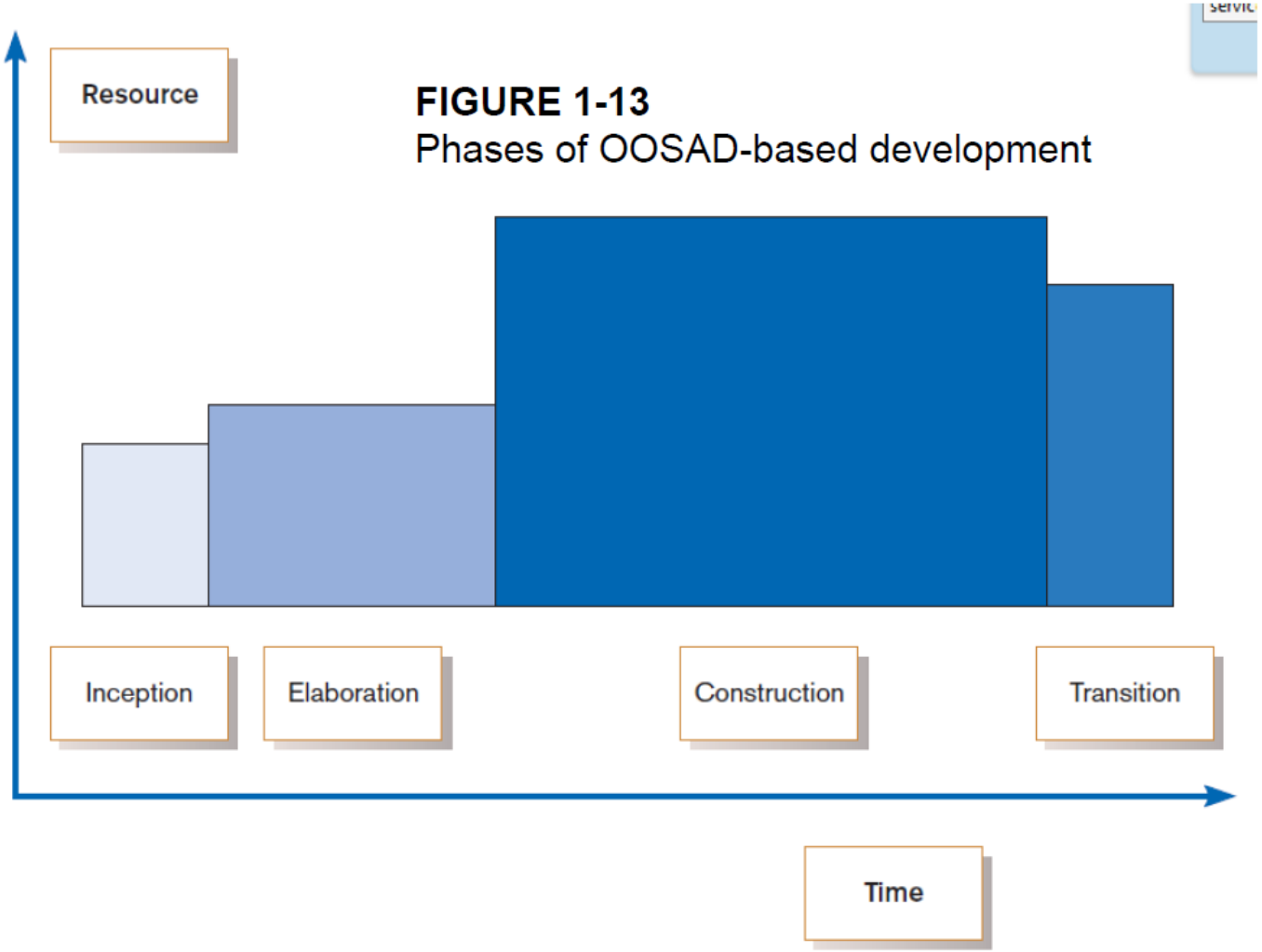
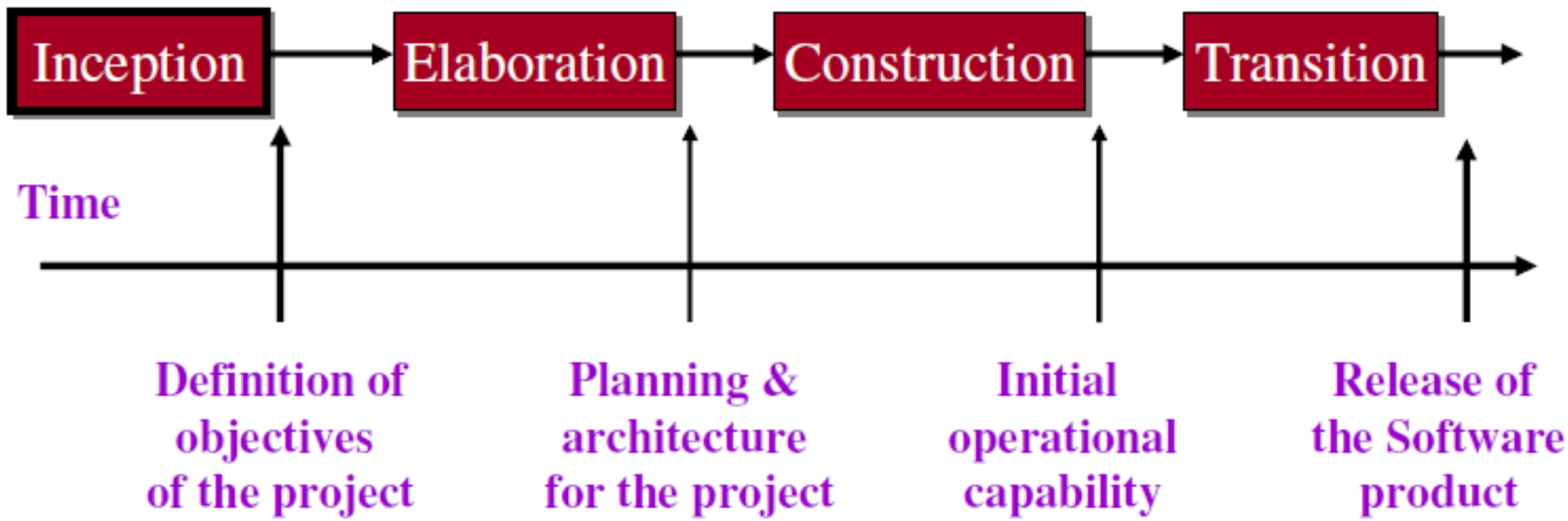


FIGURE 1-13
Phases of OOSAD-based development

The Unified Process

- Developed by I.Jacobson, G.Booch and J.Rumbaugh.
- Software engineering process with the goal of producing good quality maintainable software within specified time and budget.
- Developed through a series of fixed length mini projects called iterations.
- Maintained and enhanced by Rational Software Corporation and thus referred to as Rational Unified Process (RUP).

Phases of the Unified Process



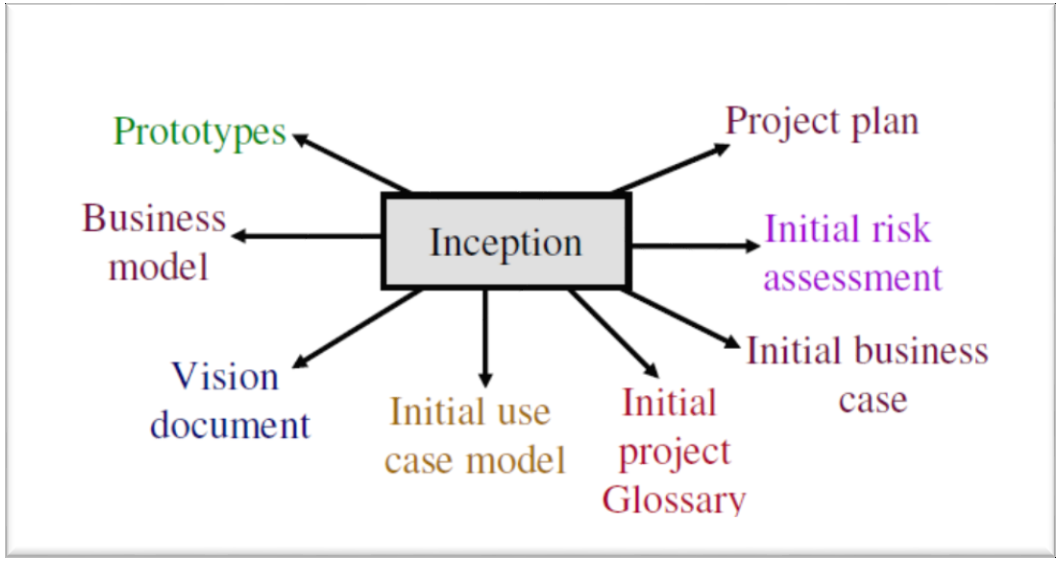
Phases of the Unified Process

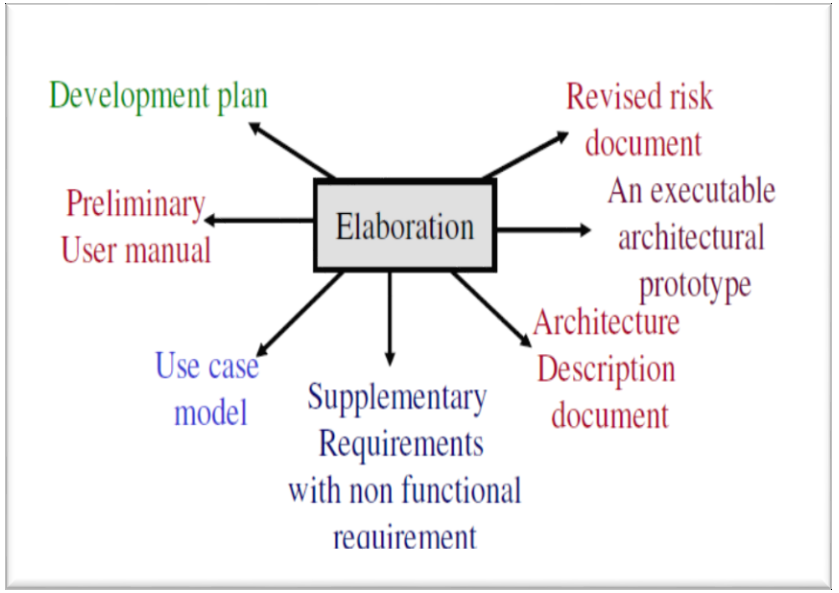
- Inception: defines scope of the project.
- Elaboration

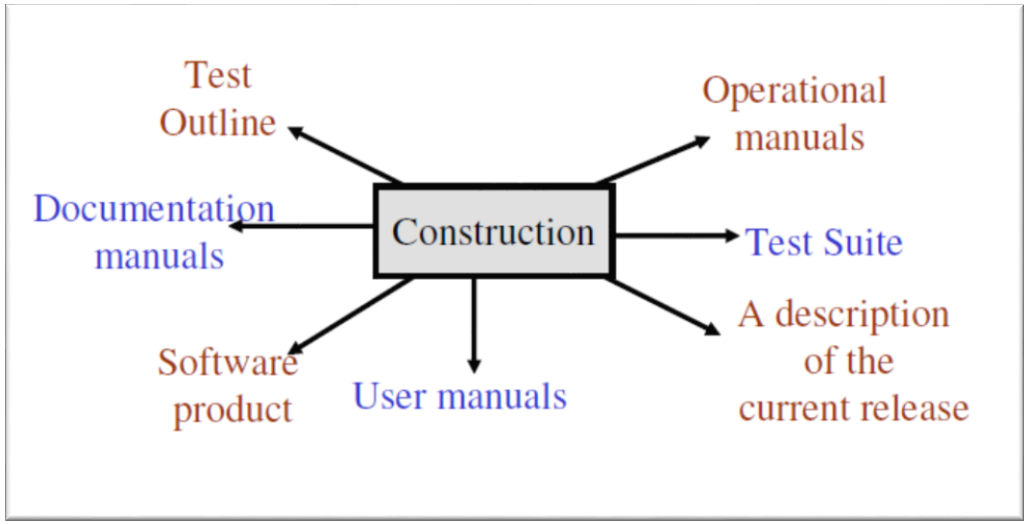
  - How do we plan & design the project?

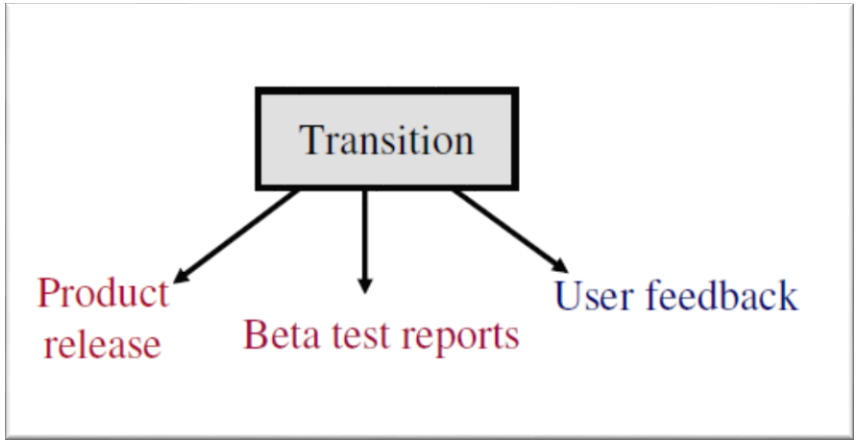
  - What resources are required?

  - What type of architecture may be suitable?
- Construction: the objectives are translated in design & architecture documents.
- Transition : involves many activities like delivering, training, supporting, and maintaining the product.


Prototypes
Project plan
Business model
Inception
Initial risk assessment
Initial business case
Vision document
Initial use case model
Initial project Glossary

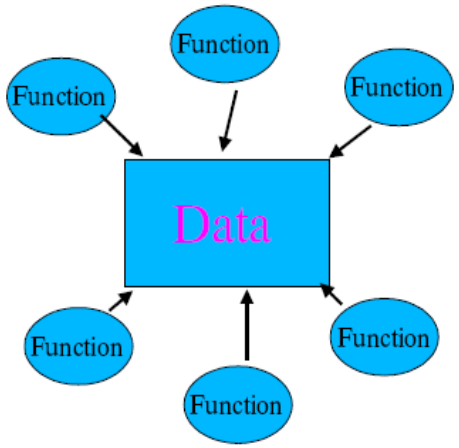

Development plan
Revised risk document
Preliminary User manual
Elaboration
An executable architectural prototype
Architecture Description document
Use case model
Supplementary Requirements with non functional requirement


Test Outline
Operational manuals
Documentation manuals
Construction
Test Suite
A description of the current release
Software product
User manuals

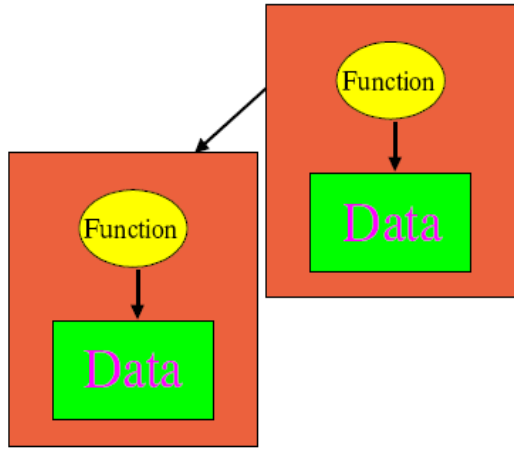

Transition
Product release
Beta test reports
User feedback

OO Approach vs Structured approach

- Structured approach focuses on major **functions** within a system and on the **data** used by the functions.
- OO approach focuses on the **objects** in a system and on the relationships between those objects.
- Unlike functional decomposition, OO views a complex problem as a meaningful collection of objects that collaborate to achieve some higher level behaviour => closely **mirrors how people view complex problems** => using OO should make the job of developing large, complex systems more manageable.



Top-down hierarchies of function calls and dependencies



Bottom-up hierarchy of dependencies

OO Vs Structured approach

OO:	**Structured:**
• Systems decomposed into collections of data objects; • function + data in one place • System components more independent : more resilient to requirements and maintenance changes. • Inheritance and polymorphism are possible: reuse, extension, and tailoring of software/designs is possible. • Closely mirrors how humans decompose and solve complex. • Process allows for **iterative** and **incremental** development : Integration of programs is series of incremental prototypes. • Users and developers get important feedback throughout development. • Testing resources distributed more evenly. • If time is short, coding and testing can begin before the design is finished.	• Systems decomposed into functions; • functions and data modelled separately : • System components are more dependent on each other : requirements and maintenance changes more difficult • Inheritance and polymorphism not possible: limited reuse possible. • System components do not map closely to real-world entities : difficult to manage complexity. • Process **less flexible** and largely **linear** : Integration of programs is 'big bang' effect. • Users or developers provided with little or no feedback; see system only when it has been completed. • Testing resources are concentrated in the implementation stage only. • Coding and testing cannot begin until all previous stages are complete.

Why Object-Oriented:

- A New Paradigm with Evolving Object Orientation
- Software complexity rises exponentially
- Need for tools to deal with complexity: OOAD provides these tools
- Helps to Manage large projects professionally
- OOAD creates models by defining objects and their rules of interaction
- Modeling is simplified with OO because we have objects and relations

Object Oriented Analysis

- Grady Booch has defined OOA as, “Object-oriented analysis is a method of analysis that examines requirements from the perspective of the classes and objects found in the vocabulary of the problem domain”.
- an investigation of the problem and requirements, rather than finding a solution to the problem
- Real-world concepts are modeled as classes with corresponding **state, behavior, and collaborations**.
- OOA, instead of mapping them indirectly into functions or data **directly maps the problem domain directly into a model** flows.
- The same model is used throughout the software engineering process during analysis, design and implementation.
- **Minimizes volatility and maximizes resiliency.**
- The focus is on classes, not function and data.
- **Reuse is enhanced**.
  - Classification - Classes are relatively stable over time.
  - Encapsulation - Classes with well-defined state and behavior.
  - Inheritance - Specializing classes for specific applications.

OO Analysis

- Examines requirements from the **perspective of the classes and objects found in the vocabulary of the problem domain**.
- In other words, the world (of the system) is modelled in terms of objects and classes.
- OOA is concerned with developing an object model of the application domain
- **Object Oriented Analysis-Steps/Activities**
  - Analyze the domain problem
  - Identify the objects
  - Organizing the objects by creating object model diagram
  - Defining operations of the objects
  - Describing how the object interact
  - Specify attributes/Defining the object internally

OO Design

- OO decomposition and a notation for depicting models of the system under development.
- Structures are developed whereby sets of objects collaborate to provide the behaviours that satisfy the requirements of the problem.
- OOD is concerned with developing an object-oriented system model to implement requirements
- How will classes in the design communicate ?” . The OOD phase deals with finding a conceptual solution to the problem – it is about fulfilling the requirements, but not about implementing the solution
- During object-oriented design (OOD) (or simply, object design) there is an emphasis on defining software objects and how they collaborate to fulfill the requirements

OO Design

- Grady Booch has defined object-oriented design as "a method of design encompassing the process of object-oriented decomposition and a notation for depicting both logical and physical as well as static and dynamic models of the system under design".

- OOD phase, the question typically starts with How...? like
  - "How will this class handle it's responsibilities?",
  - "How to ensure that this class knows all the information it needs?" and
  - "How will classes in the design communicate?"

- For example, in the library system, a Book software object may have a title attribute and a getChapter method

OO Design

- In OOD, concepts in the analysis model, which are technology−independent, are mapped onto implementing classes, constraints are identified and interfaces are designed, resulting in a model for the solution domain, i.e., a detailed description of how the system is to be built on concrete technologies.
- The implementation details generally include −
  - Restructuring the class data (if necessary),
  - Implementation of methods, i.e., internal data structures and algorithms,
  - Implementation of control, and
  - Implementation of associations.

Object-Oriented Programming

- Object-oriented programming (OOP) is a programming paradigm based upon objects (having both data and methods) that aims to incorporate the advantages of modularity and reusability.
- Objects, which are usually instances of classes, are used to interact with one another to design applications and computer programs.
- The important features of object–oriented programming are −
  - Bottom–up approach in program design
  - Programs organized around objects, grouped in classes
  - Focus on data with methods to operate upon object's data
  - Interaction between objects through functions
  - Reusability of design through creation of new classes by adding features to existing classes

Object-Oriented Programming

- Some examples of object-oriented programming languages are C++, Java, Smalltalk, Delphi, C#, Perl, Python, Ruby, and PHP.
- Grady Booch has defined object–oriented programming as

  "a method of implementation in which programs are organized as cooperative collections of objects, each of which represents an instance of some class, and whose classes are all members of a hierarchy of classes united via inheritance relationships".

☐*Object oriented programming satisfies the following requirements:*

- *It supports objects that are data abstractions with an interface of named operations and a hidden local state.*
- *Objects have associated type (class).*
- *Classes may inherit attributes from supertype to subtype.*

CASE Tools

- Computer-aided software engineering (CASE) tools
  - Facilitate creation of a central repository for system descriptions and specifications
  - Computer-aided software engineering (CASE) is the scientific application of a set of tools and methods to a software system which is mean to result in high-quality, defect-free, and maintainable software products.
  - Software tools providing automated support for systems development
  - Project dictionary/workbook: system description and specifications
  - Upper CASE Tools: Planning, analysis, and designing of different stages of the software development life cycle can be performed using upper case tools.
  - Lower Case Tools: Implementation, testing, and maintenance can be performed using lower case.
  - Diagramming tools
  - Example :Oracle Designer, Rational Rose

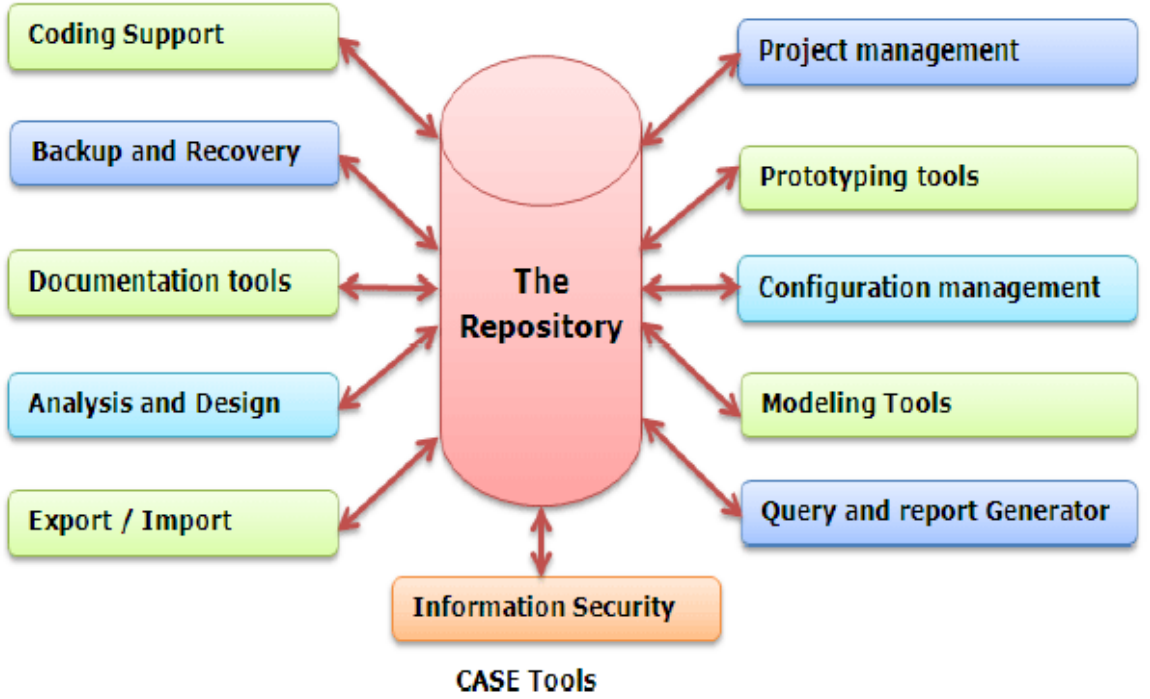
Different IDEs like Eclipse, configuration tools like Git ,word processor, excel sheet etc.

Today's CASE tools provide two distinct ways to develop system models **– forward engineering and reverse engineering**.

- **Forward engineering** requires the system analyst to draw system models, either from scratch or from templates. The resulting models are subsequently transformed into program code.
- **Reverse engineering**, on the other hand, allows a CASE tool to read existing program code and transform that code into a representative system model that can be edited and refined by the systems analyst.
- CASE tools that allow for bi-directional, forward and reverse engineering are said to provide for

"Round -trip Engineering."

- **Components of CASE:**
- At the center of any CASE tool's architecture is a developer's database called a **CASE repository**.
- **CASE repository** is a system developer's database where developers can store system models, detailed description and specification, and other products of system development.
- It is also called dictionary or encyclopedia. Around the CASE repository is a collection of tools or facilities for creating system models and documentation.
- These facilities generally include:
  - **Diagramming tools** –These tools are used to draw system models.
  - **Dictionary tools** – These tools are used to record, delete, edit, and output detailed documentation and specification
  - **Design tools** – These tools are used to construct system components including system inputs and outputs. These are also called prototyping tools.
  - **Documentation tools** – These tools are used to assemble, organize, and report on system models, descriptions and specifications, and prototypes.
  - **Quality management tools** – These tools are used to analyze system models, descriptions and specifications, and prototypes for completeness, consistency, and conformance to accepted rules of methodologies.
  - **Design and code generator tools** – These tools automatically generate database designs and application programs or significant portions of those programs.



CASE Tools

•Advantages of CASE

•As special emphasis is placed on redesign as well as testing, the servicing cost of a product over its expected lifetime is considerably reduced.

•The overall quality of the product is improved as an organized approach is undertaken during the process of development.

•Chances to meet real-world requirements are more likely and easier with a computer-aided software engineering approach.

CASE Tools

- CASE stands for **C**omputer **A**ided **S**oftware **E**ngineering. It means, development and maintenance of software projects with help of various automated software tools.
- CASE Tools
- CASE tools are set of software application programs, which are used to automate SDLC activities. CASE tools are used by software project managers, analysts and engineers to develop software system.
- There are number of CASE tools available to simplify various stages of Software
- Development Life Cycle such as Analysis tools, Design tools, Project management tools, Database Management tools, Documentation tools are to name a few.
- Use of CASE tools accelerates the development of project to produce desired result and helps to uncover flaws before moving ahead with next stage in software development

CASE Tools

- Diagramming tools enable graphical representation.
- Computer displays and report generators help prototype how systems “look and feel”.
- IBM’s Rational products are the best known CASE tools.
- Analysis tools automatically check for consistency in diagrams, forms, and reports.
- A central repository provides integrated storage of diagrams, reports, and project management specifications.
- Documentation generators standardize technical and user documentation.
- Code generators enable automatic generation of programs and database code directly from design documents, diagrams, forms, and reports.

CASE Tools

TABLE 1-2 Examples of CASE Usage within the SDLC

SDLC Phase	Key Activities	CASE Tool Usage
Project identification and selection	Display and structure high-level organizational information	Diagramming and matrix tools to create and structure information
Project initiation and planning	Develop project scope and feasibility	Repository and documentation generators to develop project plans
Analysis	Determine and structure system requirements	Diagramming to create process, logic, and data models
Logical and physical design	Create new system designs	Form and report generators to prototype designs; analysis and documentation generators to define specifications
Implementation	Translate designs into an information system	Code generators and analysis, form and report generators to develop system; documentation generators to develop system and user documentation
Maintenance	Evolve information system	All tools are used (repeat life cycle)

1.2 The Origins of Software

SYSTEMS ACQUISITION: Internal corporate information systems departments now spend a smaller and smaller proportion of their time and effort on developing systems from scratch. Companies continue to spend relatively little time and money on traditional software development and maintenance. Instead, they invest in packaged software, open-source software, and outsourced services.

Systems Acquisition: Outsourcing

- **Outsourcing**:
  - Turning over responsibility of some or all of an organization's information systems applications and operations to an outside firm
  - If one organization develops or runs a computer application for another organization, that practice is called outsourcing.
- Outsourcing Examples
  - A company that runs payroll applications for clients
  - A company that runs your applications at your site
- There are various sources of software for organizations.
- There are criteria to evaluate software from different sources.
- **Reasons to outsource**
  - Cost-effective
  - Take advantage of economies of scale
  - Free up internal resources
  - Reduce time to market
  - Increase process efficiencies
  - System development is a non-core activity for the organization

Sources of Software

- Information technology services firm
- Packaged software producers
- Enterprise-wide solutions
- Application service providers (ASPs)- Cloud computing vendors
- Open source software
- In-house developers

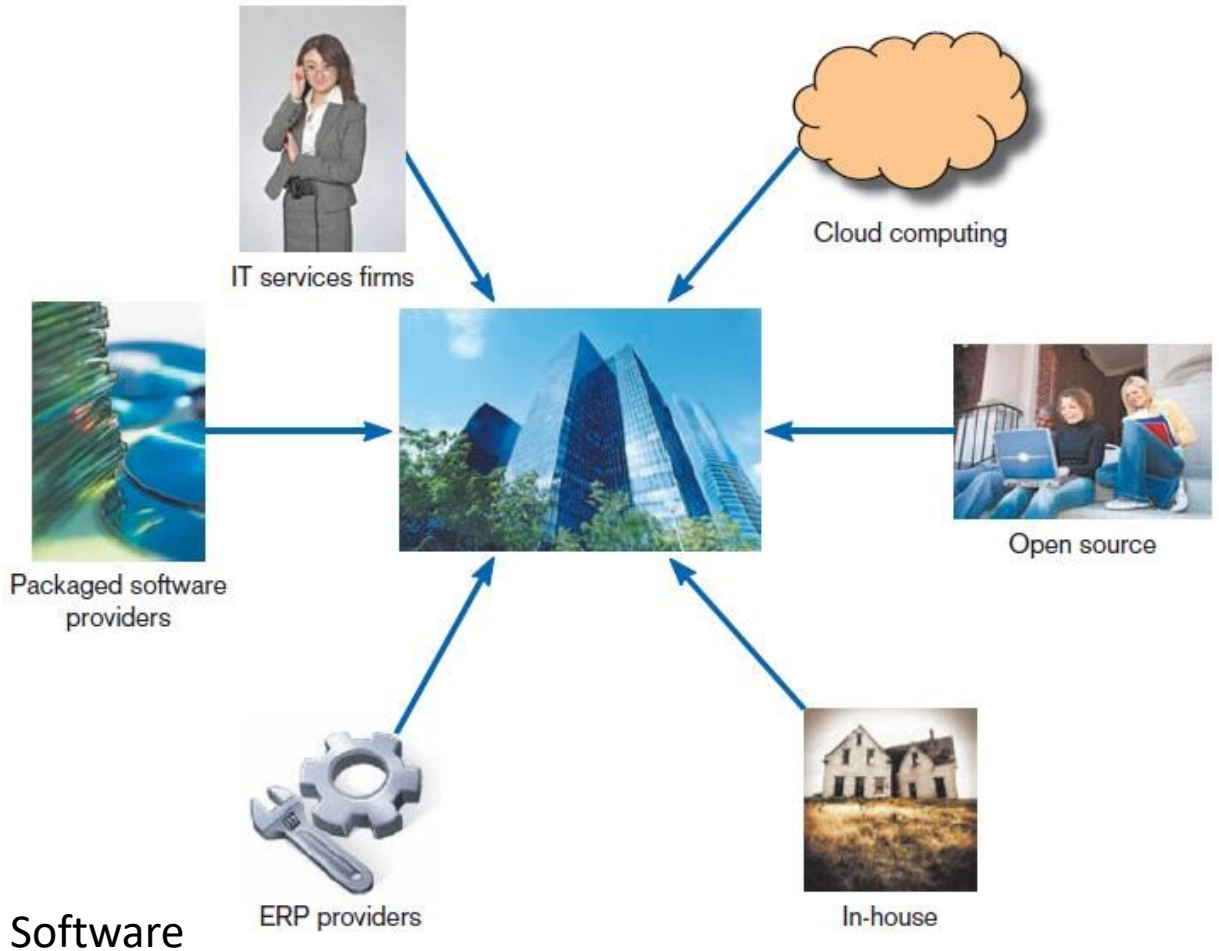


Fig: Sources of Application Software

Sources of Software

TABLE 2-1 Leading Software Firms and Their Development Specializations

Specialization	Example Firms or Websites
IT Services	Accenture
	Computer Sciences Corporation (CSC)
	IBM
	HP
Packaged Software Providers	Intuit
	Microsoft
	Oracle
	SAP AG
	Symantec
Enterprise Software Solutions	Oracle
	SAP AG
Cloud Computing	Amazon.com
	Google
	IBM
	Microsoft
	Salesforce.com
Open Source	SourceForge.net

Sources of Software

- **Information Technology (IT) Services Firms**
  - IT services firms Help companies develop custom information systems for internal use or
  - Develop, host, and run applications for customers or
  - Provide other services
  - If a company needs an information system but does not have the expertise or the personnel to develop the system in-house the company will likely consult an information technology services firm
- **Packaged Software Producers**
  - Serve many market segments.
  - Provide software ranging from broad-based packages (i.e. general ledger) to niche packages (i.e. day care management). Software runs on all size computers, from microcomputers to large mainframes.
  - Prepackaged software is off-the-shelf, turnkey software (i.e. not customizable).
  - Off-the-shelf software at best meets 70 percent of organizations' needs.

Criteria for Selecting Off-the-Shelf Software

- **Cost**: comparing the cost of developing the same system in-house with the cost of purchasing or licensing the software package
- **Functionality**: the tasks that the software can perform and the mandatory, essential, and desired system features
- **Vendor support**: whether or how much support the vendor can provide and at what cost
- **Viability of vendor**: can the software adapt to changes in systems software and hardware
- **Flexibility**: how easy it is to customize the software
- **Documentation**: is the user's manual and technical documentation understandable and up-to-date
- **Response time**: how long it takes the software package to respond to the user's requests in an interactive session
- **Ease of installation**: a measure of the difficulty of loading the software and making it operational

Packaged Software Producers (Cont.)

TABLE 2-1 The 2007 Top Global Software Companies

Rank	Company	2007 Software/Services Revenue ($US million)	Software Business Sector
1	IBM	$74,126	Middleware/Application Server/Web Server
2	Microsoft	$44,846	Operating Systems
3	EDS	$22,134	Outsourcing Services
4	Accenture	$19,696	Systems Integration Services/IT Consulting
5	HP	$18,971	Systems Integration Services/IT Consulting
6	Oracle	$17,996	Database
7	SAP	$14,980	Enterprise Application/Data Integration
8	Computer Sciences Corporation (CSC)	$14,857	Systems Integration Services/IT Consulting
9	Capgemini	$12,818	Systems Integration Services/IT Consulting
10	Lockheed Martin Corporation	$10,213	Vertical Industry Applications

- A good example is **Microsoft**, probably the best-known software company in the world . Almost 87 percent of Microsoft's revenue comes from its software sales, mostly for its Windows operating systems and its personal productivity software, the Microsoft Office Suite.

- Also listed in Table 2-1, Oracle is exclusively a software company known primarily for its database software, but Oracle also makes enterprise systems.

• Another company on the list, SAP, is also a software-focused company that develops enterprise-wide system solutions

Prepackaged Software

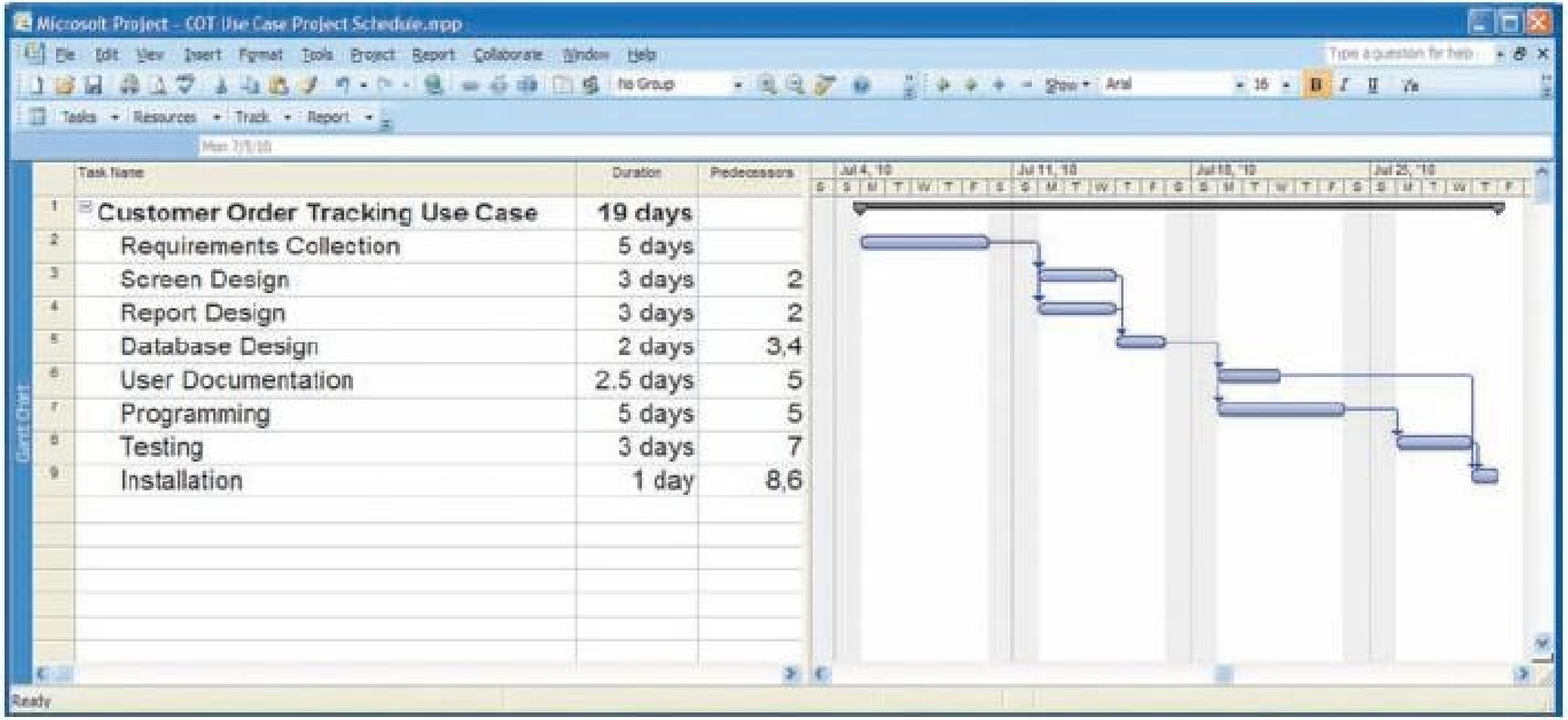


Figure 2-2 Microsoft Project

Enterprise Solutions Software

- ***Enterprise Resource Planning (ERP)*** systems integrate individual traditional business functions into modules enabling a single seamless transaction to cut across functional boundaries.
- Many firms have chosen complete software solutions, called enterprise solutions *or* **enterprise resource planning (ERP) systems, to support their operations and business processes.**
- **These ERP** software solutions consist of a series of integrated modules. Each module supports an individual, traditional business function, such as accounting, distribution, manufacturing, or human resources
- For example, a series of modules will support the entire order entry process, from receiving an order, to adjusting inventory, to shipping to billing, to after-the-sale service
- SAP AG is the leading vendor of ERP systems.

Cloud Computing

- **Cloud computing refers to t**he provision of computing resources, including applications, over the Internet, so customers do not have to invest in the computing infrastructure needed to run and maintain the resources
- It is a another method for organizations to obtain applications is to rent them or license them from third-party providers who run the applications at remote sites. Users have access to the applications through the Internet or through virtual private networks.
- The application provider buys, installs, maintains, and upgrades the applications. Users pay on a per-use basis or they license the software,
- A well-known example of cloud computing is **Google Apps**, where users can **share** and **create documents, spreadsheets, and presentations**
- Another well-known example is **Salesforce.com**, which provides customer relationship management software online

Cloud Computing

- Cloud computing encompasses many areas of technology, including software as a service (often referred to as *SaaS),* which includes Salesforce.com, and hardware as a service, which includes Amazon Web Services and allows companies to order server capacity and storage on demand.

Open Source Software

- Freely available including source code
- Developed by a community of interested people
- Performs the same functions as commercial software
- Examples: Linux, mySQL, Firefox

In-House Development

- If sufficient system development expertise with the chosen platform exists in-house, then some or all of the system can be developed by the organization’s own staff.
- Hybrid solutions involving some purchased and some in-house components are common.
- Of course, in-house development need not entail (to make something necessary) development of all of the software that will constitute the total system.
- **Hybrid** solutions involving some purchased and some inhouse software components are common. If you choose to acquire software from outside sources, this choice is made at the end of the analysis phase.

Sources of Software Components

TABLE 2-2 Comparison of Six Different Sources of Software Components

Producers	When to Go to This Type of Organization for Software	Internal Staffing Requirements
IT services firms	When task requires custom support and system can't be built internally or system needs to be sourced	Internal staff may be needed, depending on application
Packaged software producers	When supported task is generic	Some IS and user staff to define requirements and evaluate packages
Enterprise-wide solutions vendors	For complete systems that cross functional boundaries	Some internal staff necessary but mostly need consultants
Cloud computing	For instant access to an application; when supported task is generic	Few; frees up staff for other IT work
Open source software	When supported task is generic but cost is an issue	Some IS and user staff to define requirements and evaluate packages
In-house developers	When resources and staff are available and system must be built from scratch	Internal staff necessary though staff size may vary

Validating Purchased Software Information

- Use a variety of information sources:
  - Collect information from vendor
  - Software documentation
  - Technical marketing literature

Request For Proposal (RFP)

- **Request for proposal** (**RFP**) is a document provided to vendors to ask them to propose hardware and system software that will meet the requirements of a new system.
- Sometimes called a **Request For Quote** (**RFQ**)
- Use a variety of information sources
- Based on vendor bids, analyst selects best candidates.

Information Sources For RFP

- Vendor’s proposal
- Running software through a series of tests
- Feedback from other users of the vendor’s product
- Independent software testing services
- Articles in trade publications

Reuse

- The use of previously written software resources, especially objects and components, in new applications
- **Advantages:** Reuse should also decrease development time, minimizing schedule overruns. Because existing pieces of software have already been tested, reusing them should also result in higher-quality software with lower defect rates, decreasing maintenance costs
  - Less effort,
  - Time-saving,
  - Reduce cost,
  - Increase software productivity,
  - Utilize fewer resources,
  - Leeds to a better quality software

Reuse (Cont.)

- Commonly applied to two different development technologies:
  - Object-oriented development
  - Component-based development
- **Object-oriented development**
  - Object class encapsulates data and behavior of common organizational entities (e.g. employees)
- **Component-based development**
  - Components can be as small as objects or as large as pieces of software that handle single business functions.

- Object-oriented development reuse is the use of object classes in more than one application (e.g. Employee).
- Component-based development reuse is the assembly of an application from many different components at many different levels of complexity and size (e.g. Currency conversion).

Costs and Benefits of Reuse

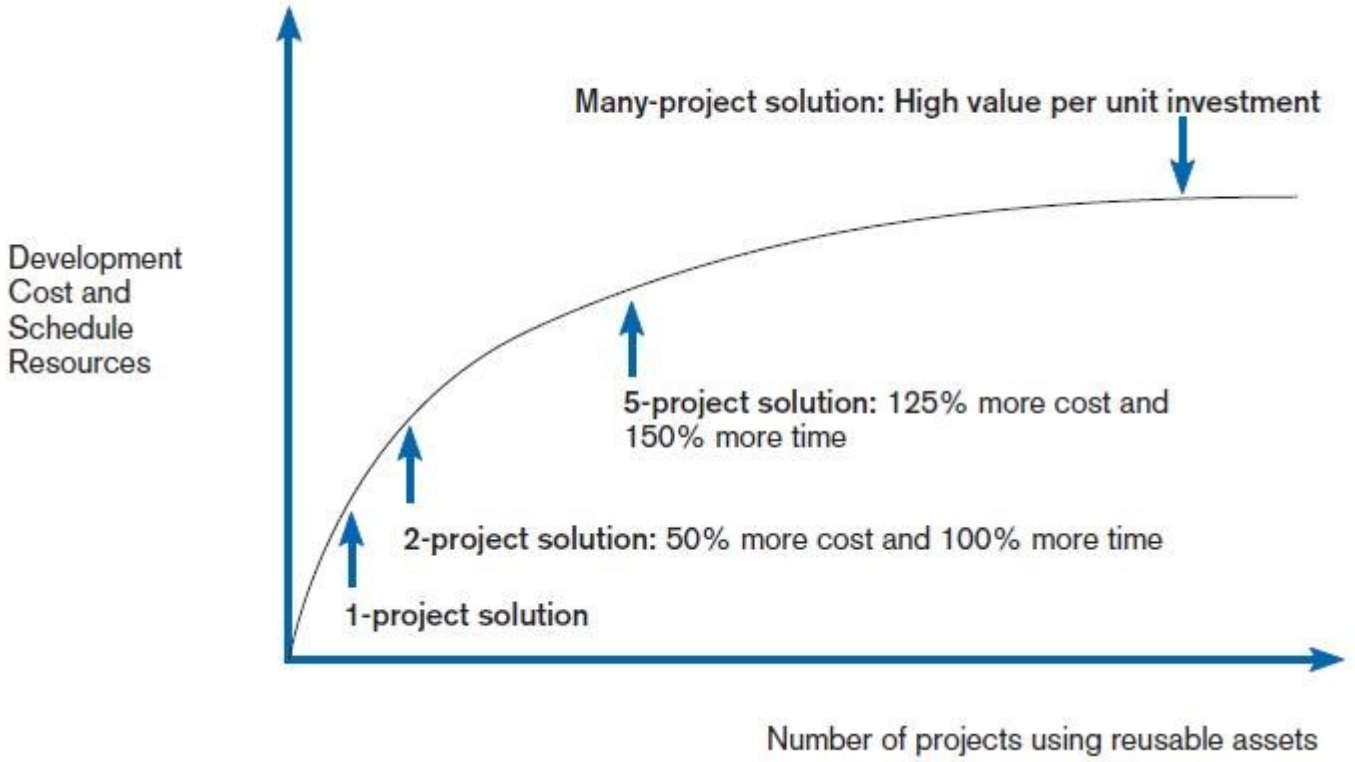


FIGURE
Investments necessary to achieve reusable components

Approaches to Reuse

- **Ad-hoc**: individuals are free to find or develop reusable assets on their own.
- **Facilitated**: developers are encouraged to practice reuse.
- **Managed**: the development, sharing, and adoption of reusable assets is mandated.
- **Designed**: assets mandated for reuse as they are being designed for specific applications.

Approaches to Reuse (Cont.)

TABLE 2-3 Approaches to Reuse

Approach	Reuse level	Cost	Policies & Procedures
Ad hoc	None to low	Low	None
Facilitated	Low	Low	Developers are encouraged to reuse but are not required to do so.
Managed	Moderate	Moderate	Development, sharing and adoption of reusable assets are mandated; Organizational policies are established for documentation, packaging, and certification.
Designed	High	High	Reuse is mandated; Policies are put in place so that reuse effectiveness can be measured; Code must be designed for reuse during initial development, regardless of the application it is originally designed for; There may be a corporate office for reuse.

(*Source:* Adapted from Griss, 2003.)

1.3 Managing the Information Systems Project

- Introduction;
- Managing the Information Systems Project;
- Representing and Scheduling Project Plans;
- Using Project Management Software

Introduction

- **Project** is a planned undertaking of a series of related activities to reach an objective that has a beginning and an end
- **Objective of a project**
  - Solve a business problem (develop a MIS)
  - Take advantage of a business opportunities (develop BIS)
  - Other non rational reason: spend existing available resources, training and enhancing skills of employees
- **Where do projects come from?**
  - There is no standard and answer varies from organization to organization
    - Several projects may be submitted and need selection by filling a “Systems Service Requests”

Project management

- **Project management**
  - A controlled process of initiating, planning, executing, and closing down a project
- **Project management** is an important aspect of system analyst
  - Need to know project management skills more details about resources management, project management, change management, risk management
  - Focus of project management
    - To ensure that information system projects meet customer expectations
    - Delivered in a timely manner
    - Meet time constraints and requirements

Project manager

- **Project manager** is an individual with a diverse set of skills (management, leadership, technical, conflict management, and customer relationship) who is responsible for managing the project process when the project is accepted
- **Responsibilities of project manager include**: initiating, planning, executing and closing-down the project

Skills of project manager includes:

- Leadership
- Management (resources, materials, funding)
- Customer relationships
- Technical problem solving
- Conflict management
- Customer relationship Management
- Team management
- Risk and change management

- **Project manager** refers to system analyst's role in managing information system project
  - Project manager works in an environment of change.
    - He identifies and solves problems
    - He is responsible of all aspect of system development project (time, cost, progress, etc. see next slides)
    - He is responsible either as a part or the whole project
  - System analyst has specific tasks (identify requirements, allocate budget and keep timing constraints)

Project Management Process

- Project management is the controlled process of initiating, planning, executing and closing-down a project.
- Project Management Process Consists four Phases:
  - Initiation
  - Planning
  - Execution
  - Closing down

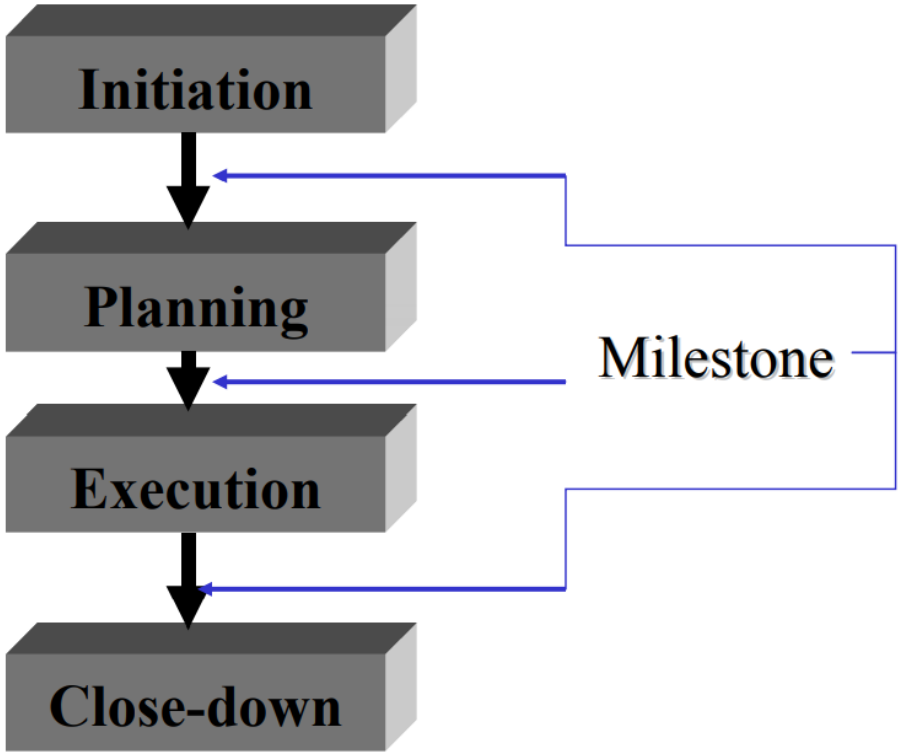


Fig:Project Management Phases

Project Management Process

- **Phase 1:Project Initiation**
  - It is the phase where activities are performed to assess the size, scope and the complexity of the project & to establish procedures to support later project activities.

  - **Activities in Project Initiations are:**
    - Establish project initiation team
    - Establish relationship with customer
    - Establish project initiation plan
    - Establish management procedures
    - Establish project management environment and workbook

Workbook:

- Workbook can be on line (a web site as SIMNET) or a hard copy repository that contains :
  - All project correspondence: minutes, deliverable, input, output
  - Deliverables and reports
  - Standard to performing audits
  - Management procedures
  - Post project review meeting (future– Workbook are mainly on line in order to allows different partners to access the content through different locations)

- Establishing the project initiation team
  - Size: identify project team member who will work within the project
- Establishing relationship with the customer
  - Develop good work relationship & trust between customer or users of the project & the IS development group (project team) before the project
  - Helps customers to understand their problems they might face & propose improvement
- Establishing the project initiation plan
  - Define the scope (objectives) of the project (even objectives are fuzzy)
  - Assign objectives to project team members
  - Define roles of each member in the project
  - Might lead to the creation of a System Service Request (SSR)
- Establishing management procedure
  - Concern procedures to manage the project such as communication and reporting procedures,
  - Communication procedure
  - Funding & billing procedure
  - Conflict management
  - Regulatory procedure
  - Procedures exist or need to be created
- Establishing project management environment & project workbook
  - establish an environment that includes data related project, called workbook
  - Performing the workbook is one important task of project manager

Phase 2-Project Planning

- It consists to define clear discrete activities and the work needed to complete the activities within a single project (identify time, input and output)
- Project planning is different than Information System Planning (ISP)
- ISP focuses on assessing the information system needs of the entire organization
- Following activities involves in project Planning Phases:
  1. Describing project scope, alternatives and feasibility
  2. Dividing the project into manageable tasks and logical order
  3. Estimating resources and creating a resources plan
  4. Developing a preliminary schedule
  5. Developing a communication plan
  6. Determine project standards and procedures
  7. Identifying and assessing risks
  8. Creating a preliminary budget
  9. Developing a statement of work (for customer)
  10. Setting a baseline project plan

1. Describing project scope, alternatives and feasibility

- Identify project scope through answering question such as
  - What problem or opportunity does the project address?
  - What quantifiable results to be achieved
  - How success will be measured
  - What criteria to be used in order to ensure the project is completed?
- Identify alternatives solution for current business problem
- Assess feasibility of solutions
- Make decision about the planed solution

2. Dividing the project into manageable tasks and logical order

– Is called “work breakdown” structure

– Require to decompose SDLC phases into activities and activities in to tasks

– Each activities involves many tasks. For example:

- Activities = develop data and process flow.
- Tasks = interviewing manager, identifying process and data inflow, outflow, and transformation, etc.

3: Estimating resources and creating a resources plan

– Estimate resource requirements :

- how many manpower, money, software tools, are required to complete the project?

– Resources planning is the estimation of the resource, within each activity, needed to complete the project

– Time allocated to tasks depend on people assignment to tasks

– Remark: a person could be assigned to more than one tasks in his own area of expertise

4: Developing a preliminary schedule

- Preliminary schedule = tasks + time + people within each activity of work breakdown structure
- This schedule may be drawn using the Grant and the PERT chart

5: Developing a communication plan

- When & how different roles will communicate
- When and how written and oral presentation will be provided by the team
- How many deliverables (official reports) and when should be written (set deadlines)
- Define agenda for meetings and set deadlines (small & big meetings)

6:Determine project standards and procedures

- How standard SDLC must be modified?
- What case tools to use?
- What approaches will be used (JAD, prototyping, etc.)?
- How different team will report (horizontal or vertical)?

7: Identifying and assessing risks

- Identify sources of risks and their expected impact
- Source: use of new technology, resistance to change, availability of insufficient resources, team member inexperience

8:Creating a preliminary budget

- Estimate project cost of the and some times the revenue of the project (show revenue is great than cost)

9:Developing a statement of work (for customer)

- Is a short description of all work to be done & expected deliverables
- Give clear idea to all project team and customer about the project size

10:Setting a baseline project plan and set Milestone with a review meeting

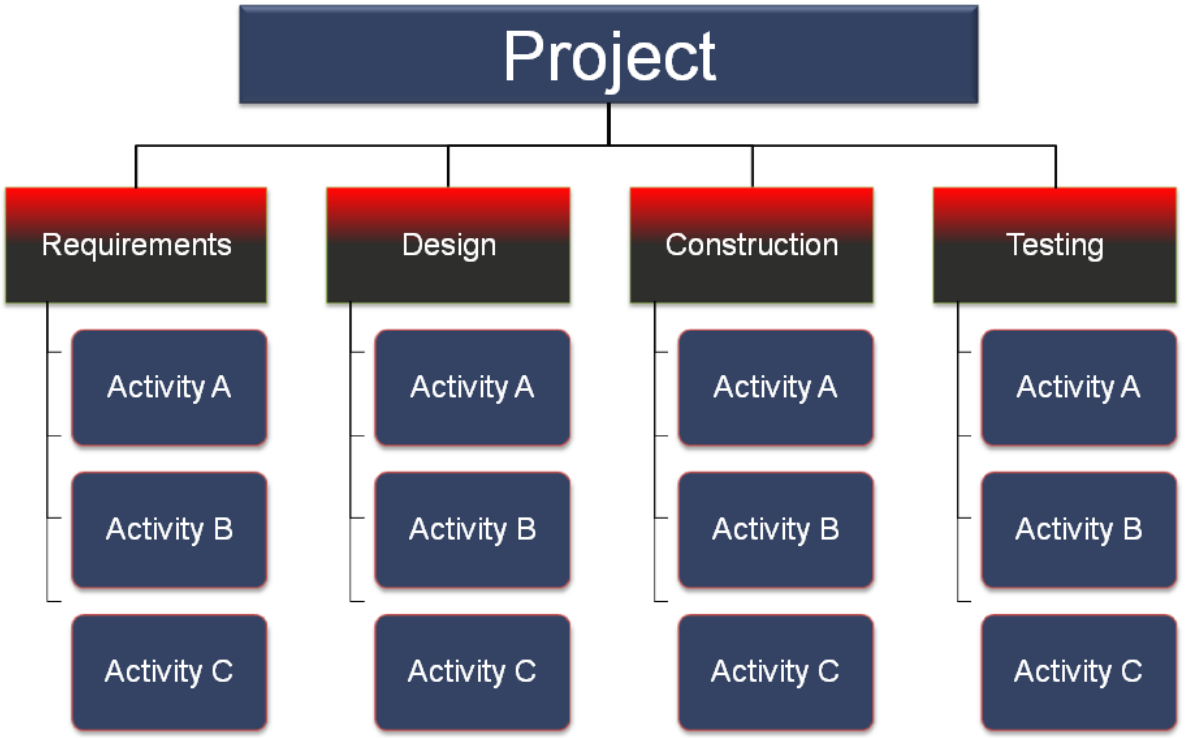
- Contains resources, times and manpower (resource requirement)
- Is used to guide the executing phase or to update it when change happen
- Propose modification and update if needed ->back to previous activities

- Some components of Project Planning
- Statement of Work (SOW)
  - “Contract” between the IS staff and the customer regarding deliverables and time estimates for a system development project
- The Baseline Project Plan (BPP)
  - Contains estimates of scope, benefits, schedules, costs, risks, and resource requirements
- Preliminary Budget
  - Cost-benefit analysis outlining planned expenses and revenues
- Work Breakdown Structure (WBS)
  - Division of project into manageable and logically ordered tasks and subtasks
- Scheduling Diagrams
  - Gantt chart: horizontal bars represent task durations
  - Network diagram: boxes and links represent task dependencies

WBS:Work Breakdown Structure

- A work breakdown structure (WBS), in project management and systems engineering, is a deliverable-oriented decomposition of a project into smaller components. A work breakdown structure is a key project deliverable that organizes the team's work into manageable sections.
- Dividing complex projects to simpler and manageable tasks is the process identified as Work Breakdown Structure (WBS).
- Usually, the project managers use this method for simplifying the project execution.
- In WBS, much larger tasks are broken down to manageable chunks of work. These chunks can be easily supervised and estimated.
- Following are a few reasons for creating a WBS in a project:
  - Accurate and readable project organization and structure.
  - Accurate assignment of responsibilities to the project team.
  - Indicates the project milestones and control points.
  - Helps to estimate the cost, time and risk.
  - Illustrate the project scope, so the stakeholders can have a better understanding of the same.
- **Construction of a WBS**
  - The root of the tree is labelled by the problem name.
  - Each node of the tree is broken down into smaller activities that are made the children of the node.
  - Each activities is recursively decomposed into smaller sub-activities until at the leaf level.

WBS Structure of MIS Problem



Phase-3 : Executing the project

- It consists to put the planned baseline project into action
- Activities during executing phase:

1. Executing the base line project plan

- Keep the project schedule
- Ensure the quality of product deliverable
- Motivate people and increase the work team; think as one member

2. Monitoring the project progress against the actual progress work

- Enable modifications to current plans when needed
- Adjust resources, budget, and time to activities
- Evaluate efficiency of project team member
- E.g. if one dependence activity is changed, you have to undertake the impact on the other related activities

3. Managing change to the baseline project plan

- Manage the change if it has to occur
  - Slipped completion dates
  - Changes in personnel
  - New activities
  - spoiled activities
- Managing change requires three steps
  - Request a change
  - Accept change
  - Apply change order
- PERT and Gantt charts can be used to undertaking changes

4.Maintaining the project workbook

- Update the workbook as the project progress

5.Communicating the project status

- Keep involved roles informed about the latest development of the project
- There are three types of communication (communication is useful for)
  - Solving issues through oral presentation
  - Informing others through e-mails
  - Keeping permanent storage of records (of written documents)
- **Communication methods example:**
  - Project workbook
  - Meetings
  - Seminars and workshops
  - Newsletters and status reports
  - Specification documents
  - Minutes of meetings
  - Bulletin boards
  - Memos
  - Brown Bag launches and hallway discussions

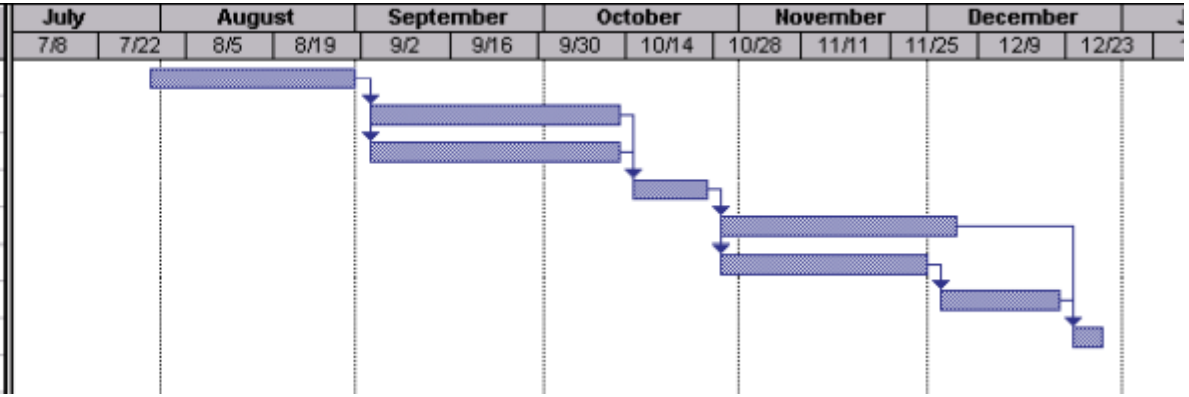
Phase-4 : Closing down the project

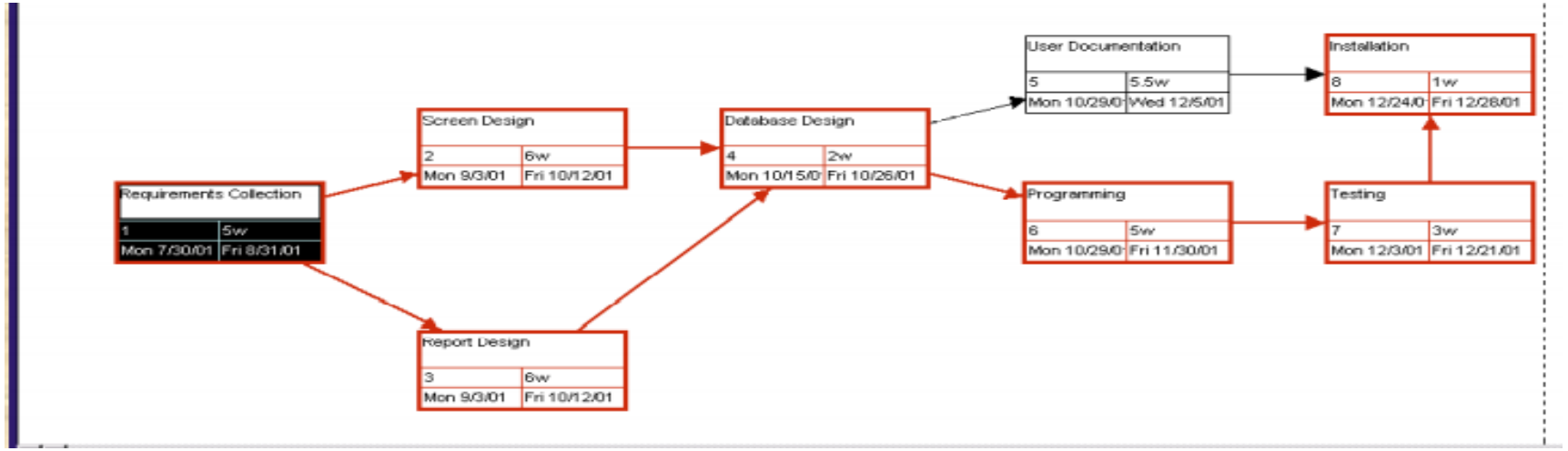
- It is consists to bring the project to an end
- When does a project end?
  - If requirements have been all met (normal or natural end) or project stop(unnatural end)
  - If all objectives have been successfully achieved
  - Customers' need are not any more valid in the customer business environment; state-of-the-art technology is available on the market)
  - Running out of money
- Closing-down the project
  - Inform all members about the project end during a review meeting
  - Personnel Appraisal
  - Conducting post-project review
    - Set a review meeting with management and customers to assess project' strengths and weakness
    - Develop new idea for new projects
  - Closing the customer contract
    - Stop funding and further new projects

• Representing and Scheduling Project Plans

- Gantt chart is a graphical representation of a project that shows each task activity as a horizontal bar who is proportional to its time for completion.
- Activity Network is a graphical representation of sequence of project activities and the dependencies among them.
- PERT chart is a diagram that represents project activities & their dependencies
- There are several tools to support Gantt and PERT charts

	Task Name	Duration	Start	Finish
1	Requirements Collection	5w	Mon 7/30/01	Fri 8/31/01
2	Screen Design	6w	Mon 9/3/01	Fri 10/12/01
3	Report Design	6w	Mon 9/3/01	Fri 10/12/01
4	Database Design	2w	Mon 10/15/01	Fri 10/26/01
5	User Documentation	5.5w	Mon 10/29/01	Wed 12/5/01
6	Programming	5w	Mon 10/29/01	Fri 11/30/01
7	Testing	3w	Mon 12/3/01	Fri 12/21/01
8	Installation	1w	Mon 12/24/01	Fri 12/28/01
9				

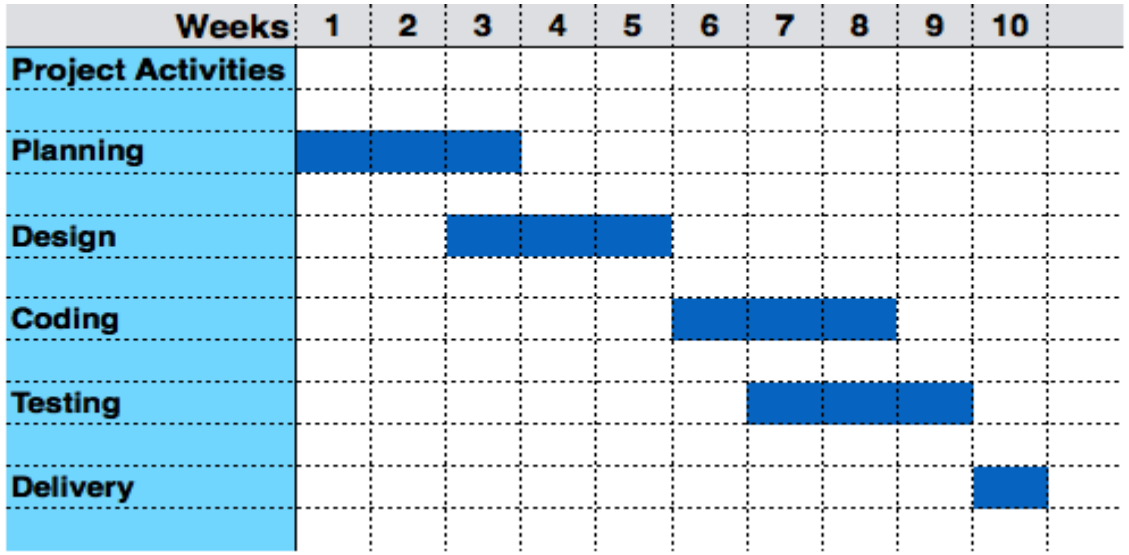


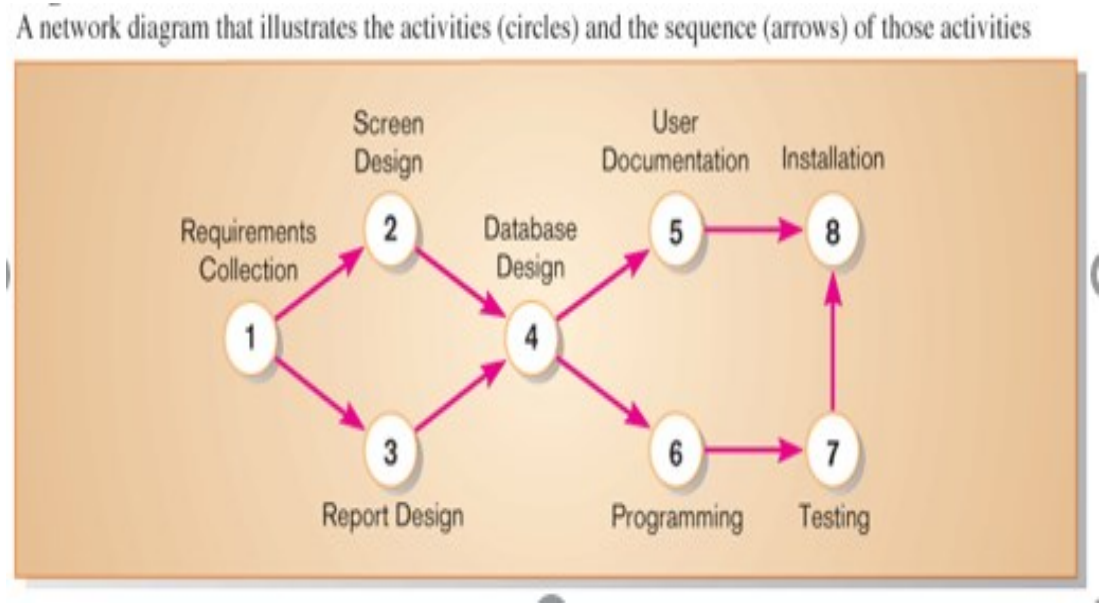


Gantt Charts

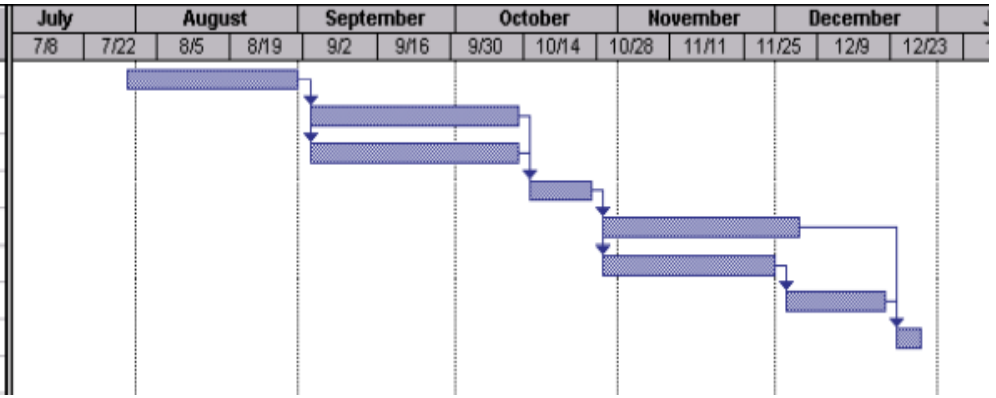
- Gantt chart was invented by a mechanical engineer named Henry Gantt in 1910. Since the invention, Gantt chart has come a long way.
- Gantt chart is a type of a bar chart that is used to allocate resources to activities including staff, hardware and software.
- Gantt charts allow project managers to track the progress of the entire project. Through Gantt charts, the project manager can keep a track of the individual tasks as well as of the overall project progression.
- In addition to tracking the progression of the tasks, Gantt charts can also be used for tracking the utilization of the resources in the project. These resources can be human resources as well as materials used.
- The bars in the Gantt are drawn along a time line and the length of each bar is proportional to the duration of time planned for the corresponding activity.
- The White Part in Gantt chart shows the slack time(latest time by which the task must be finished) and the shaded part shows the length of the time each task is estimated to take.
- Advantages and Disadvantages:
  - ability to grasp the overall status of a project and its tasks at once
  - helps to identify and maintain the critical path of a project schedule
  - For large projects, the information displayed in Gantt charts may not be sufficient for decision making.
  - it does not elaborate on the project size or size of the work elements

Gantt Chart :





	Task Name	Duration	Start	Finish
1	Requirements Collection	5w	Mon 7/30/01	Fri 8/31/01
2	Screen Design	6w	Mon 9/3/01	Fri 10/12/01
3	Report Design	6w	Mon 9/3/01	Fri 10/12/01
4	Database Design	2w	Mon 10/15/01	Fri 10/26/01
5	User Documentation	5.5w	Mon 10/29/01	Wed 12/5/01
6	Programming	5w	Mon 10/29/01	Fri 11/30/01
7	Testing	3w	Mon 12/3/01	Fri 12/21/01
8	Installation	1w	Mon 12/24/01	Fri 12/28/01
9				



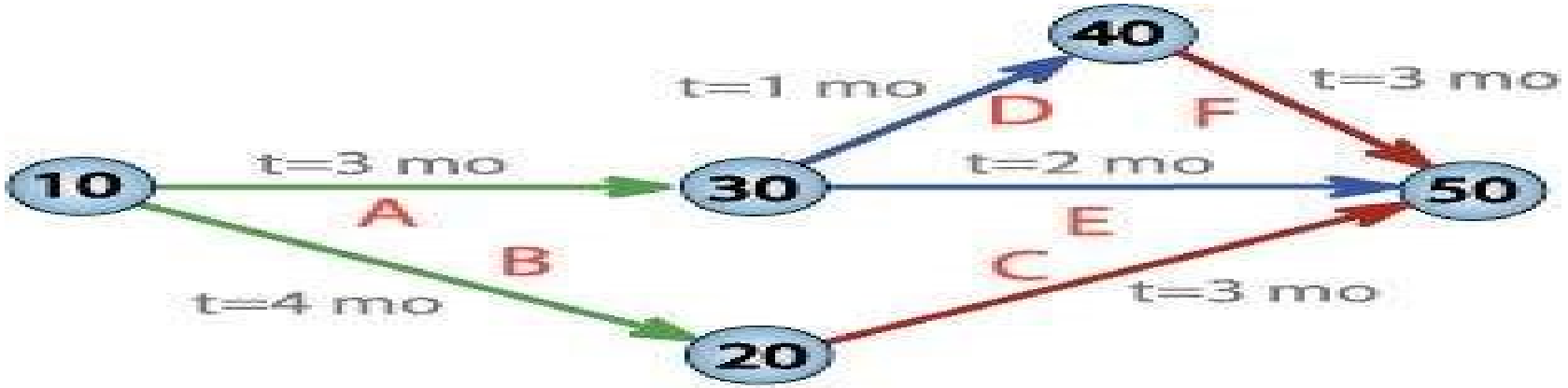
Difference between Gantt Chart and Network Diagram

Gantt Chart	Network Diagram
Gantt Chart is a Pictorial Representation of Project Schedules	Network Diagram is a Pictorial Representation of Project Work-flow
Focus is on Tasks and Time Management	Focus is on Milestones Work Sequential Order
In-depth Plan to understand of Project Tasks	High-level Plan to Understand the Project Work-flow
Project Time lines are Clearly Defined. For example, 1-Jan-2017 to 25-July-2017 for Task A	Estimated Time frames are defined. For example, 4 days, 2 weeks, 1 month for Milestone A.
Effective Resource Allocation is can be done using Gantt Format	It is not possible using Network Diagram
Project Progress is captured	Can not Capture the % Progress
Project Status is updated. Like New, Open, In progress, Completed, On hold	We can not Updated Project Status
Project Complexity Details recorded. For Example, Level 2, Level 2, Level 3.	We will not record project complexity.
We can add Task description and Remarks for each task and update any time if required	We can not enter remarks or task description here
Easy to modify and edit the information	Difficult to change the information
Stacked Bar Chart is used to Create Gantt Chart	Process Flow Chart diagrams are used to create Network diagram

PERT(Program Evaluation and Review Techniques)

- PERT (Program Evaluation and Review Technique) is a project management tools used to schedule, organize and coordinate tasks within a project.
- PERT was initially created by the US Navy in the late 1950s. The pilot project was for developing Ballistic Missiles and there have been thousands of contractors involved.
- PERT chart consists of networks of boxes and arrows. The boxes represent activities and the arrows represents task dependencies.
- PERT chart represents the statistical variations in the project estimates assuming normal distribution. So instead of making single estimates for each task ,pessimistic(p),optimistic(o) and most likelihood(m) estimates are made. That is
  - **Estimates(te)=(o+4*m+p)/6.**
  - ***Where optimistic time(o) means the minimum possible period of time for an activity to be completed***
  - ***Pessimistic time(p) means the maximum possible period of time for an activity to be completed***
  - ***Most likelihood or realistic time(m) is the project manager's best guess of the amount of time that the activity actually will require for the completion***
- Same as most of other estimation techniques, PERT also breaks down the tasks into detailed activities.
- PERT consists of Milestones and Activities.

- Advantages:
  - Simple graphical representations helps to show the task interrelationships.
  - It has the ability to highlights the project's critical path and slack time.
  - It also shows which tasks must be completed before other are begun.
  - It exposes all possible parallelism in the activities and thus help in allocating resources .
- Limitations:
  - Project task has to be clearly defined as well as their relationships
  - It does not deal very well with task overlapping case.
  - It does not help in deciding which activities are necessary or how long each will take.



PERT network chart for a seven-month project with five milestones (10 through 50) and six activities (A through F)

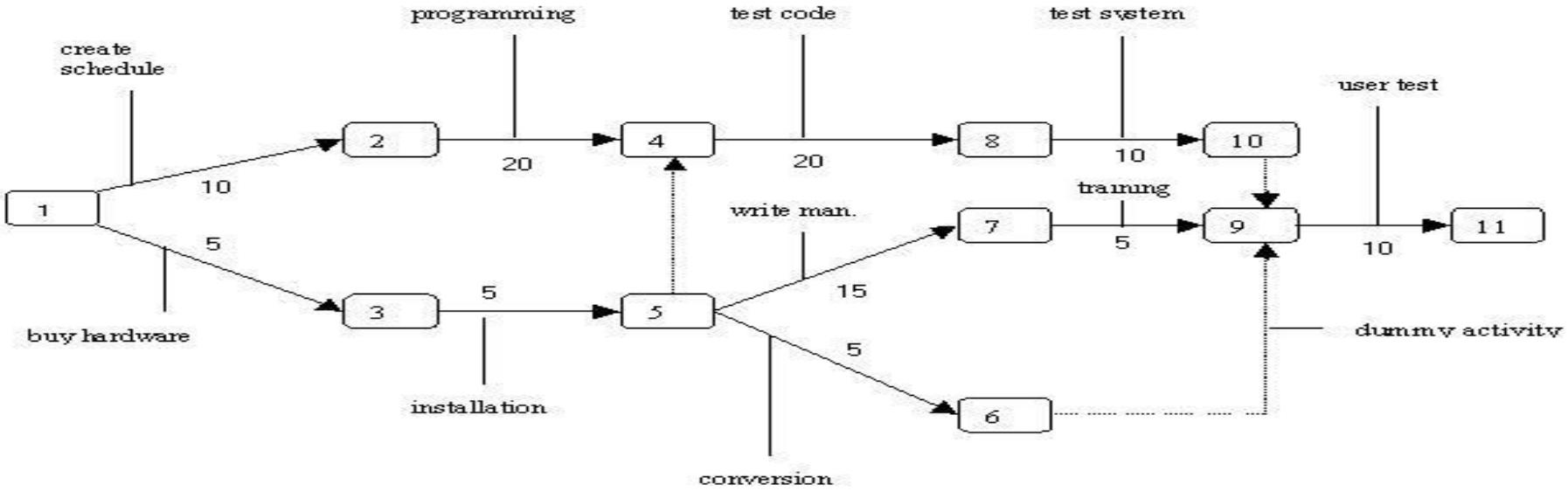


Fig. 1:
PERT Chart

* Numbered rectangles are nodes and represent events or milestones.
* Directional arrows represent dependent tasks that must be completed sequentially.
* Diverging arrow directions (e.g. 1-2 & 1-3) indicate possibly concurrent tasks
* Dotted lines indicate dependent tasks that do not require resources.

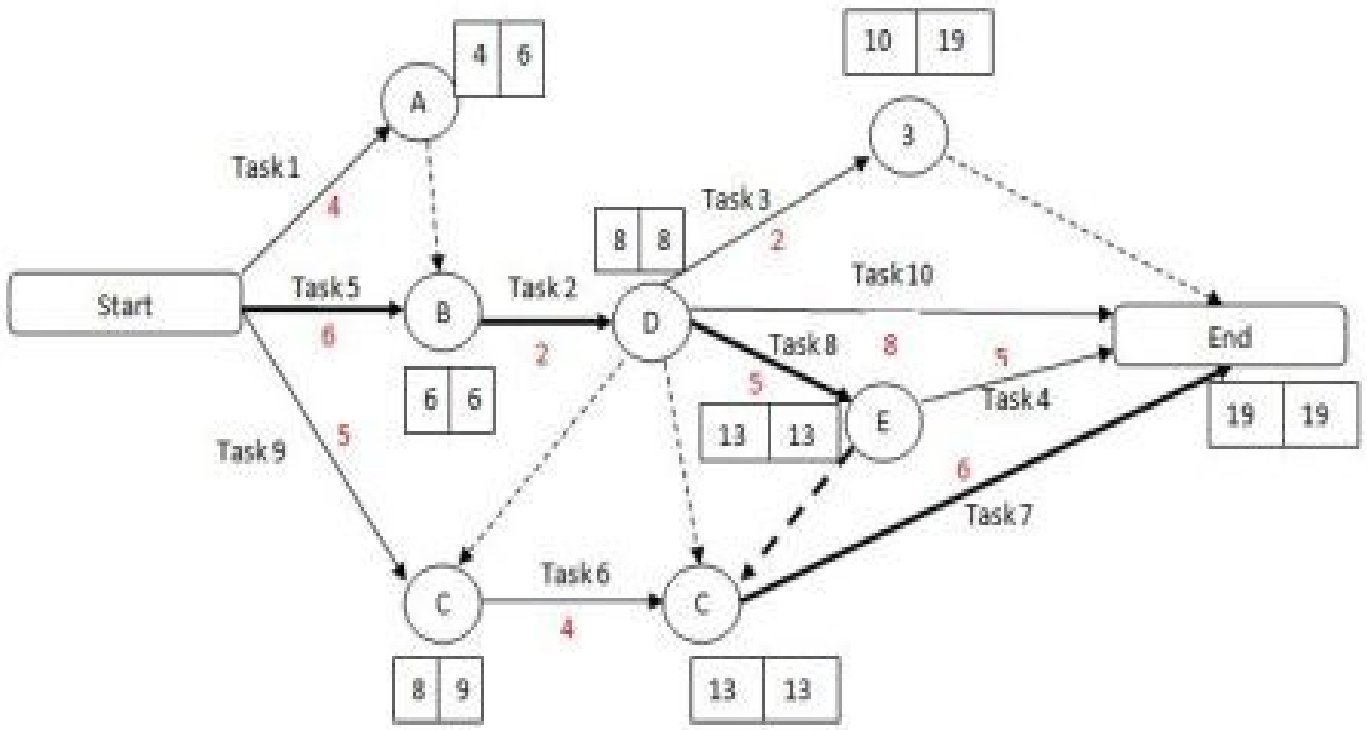
CPM(Critical Path Methods)

- CPM allows to monitor the achievements of the project goals and help to find the remedial actions need to be taken to get the project back on course.
- Critical path is the sequential activities from start to the end of a project ie the path of longest duration as determined on the project network diagram(total duration of the project).
- Although many projects have only one critical path, some projects may have more than one critical paths depending on the flow logic used in the project.
- If there is a delay in any of the activities under the critical path, there will be a delay of the project deliverables.
- The path is critical because the task that followed the critical path can not be started until all of the previous task on the critical paths are completed.
- Most of the times, if such delay is occurred, project acceleration or re-sequencing is done in order to achieve the deadlines.
- Critical path method is based on mathematical calculations and it is used for scheduling project activities. This method was first introduced in 1950s as a joint venture between Remington Rand Corporation and DuPont Corporation.
- In the critical path method, the critical activities of a program or a project are identified. These are the activities that have a direct impact on the completion date of the project.
- The critical task will have starting time and finishing time and task not in critical path can have flexibility of starting and finishing the task known as slack time or float time.
- Critical Path have four key elements:
  - i.Critical path analysis:find the path with longest duration among different activities from start to finish.
  - Ii.Float determiniation:float is an amount of time an activity can delayed before it causes whole project delayed that is slack.(no slack in CP)
  - Iii.Early start and early finish time calculation.ES=**Ef+activity duration and EF=ES+duration-1;for first activity ES=1**
  - **Iv.late start time and late finish time calculation.LS=LF-duration+1 and LF of last activity is EF of last activity**

ES:Earliest start time =EF +activity duration for task 1 (0+4)=4
EF:Earliest finish time=Es+duration-1(1+4-1)=4 or (1+6-1)=6 for task2
LS:late start time=LF+duration-1
LF:late finished time=last EF=19
ES=Ef+activity duration
Ls=LF –activity duration
float =LS-Es or LF-EF
dummy=having no duration

Advantages/Limitations of Critical Path Method

Offers a visual representation of the project activities.
Presents the time to complete the tasks and the overall project.
Tracking of critical activities.
Difficult to understand and manage.



Earliest Completion Time	Latest Completion Time

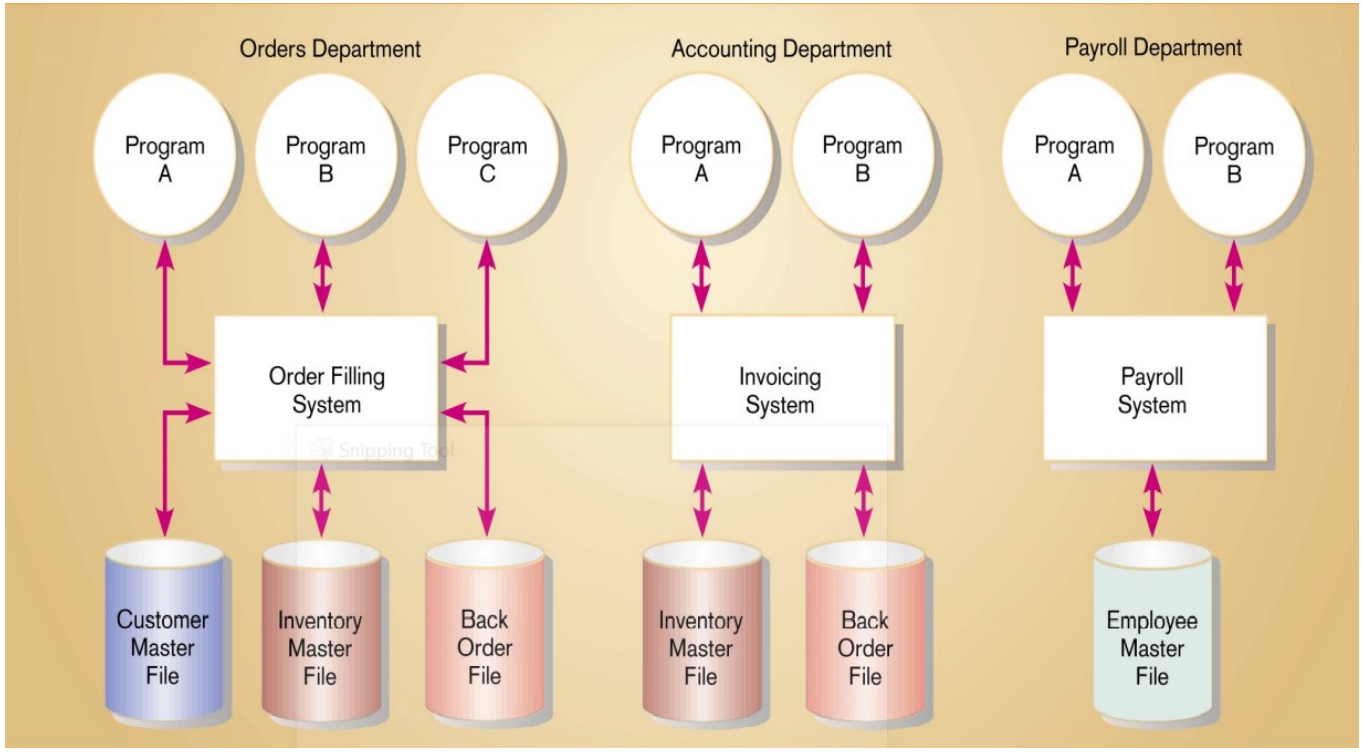
Basis for Comparison	PERT	CPM
Meaning	PERT is a project management technique, used to manage uncertain activities of a project.	CPM is a statistical technique of project management that manages well defined activities of a project.
What is it?	A technique of planning and control of time.	A method to control cost and time.
Orientation	Event-oriented	Activity-oriented
Evolution	Evolved as Research & Development project	Evolved as Construction project
Model	Probabilistic Model	Deterministic Model
Focuses on	Time	Time-cost trade-off
Estimates	Three time estimates	One time estimate
Appropriate for	High precision time estimate	Reasonable time estimate
Management of	Unpredictable Activities	Predictable activities
Nature of jobs	Non-repetitive nature	Repetitive nature
Critical and Non-critical activities	No differentiation	Differentiated
Suitable for	Research and Development Project	Non-research projects like civil construction, ship building etc.
Crashing concept	Not Applicable	Applicable

- **Using Project management software**
- There are several tools to support the Gantt and Pert charts
  - Allow to facilitate complexity of a project
  - Complexity = number of tasks & relationship
  - Allow to define tasks, order tasks, assign resources to tasks, Differences between system
- There are many software – E.g. Microsoft's Project is a great tool for any one who oversees a team, plans a budget, juggles schedules, or has deadlines to meet
- Many systems are available

- Three activities required to use:
  - Establish project start or end date
  - Enter tasks and assign task relationships
  - Select scheduling method to review project reports

Three computer applications at Pine Valley Furniture: Order Filling, Invoicing, and Payroll (Source: Hoffer, Prescott, and McFadden, 2002)

- **Pine Valley Furniture Manufacturing Company**
- Product: Wood Furniture
- Market: US
- Organized into functional areas
  - Manufacturing
  - Sales
  - Orders
  - Accounting
  - Purchasing
- Three independent computer systems were converted to a database in 1990's



Gantt and PERT Charts for Pine Valley Furniture

Steps

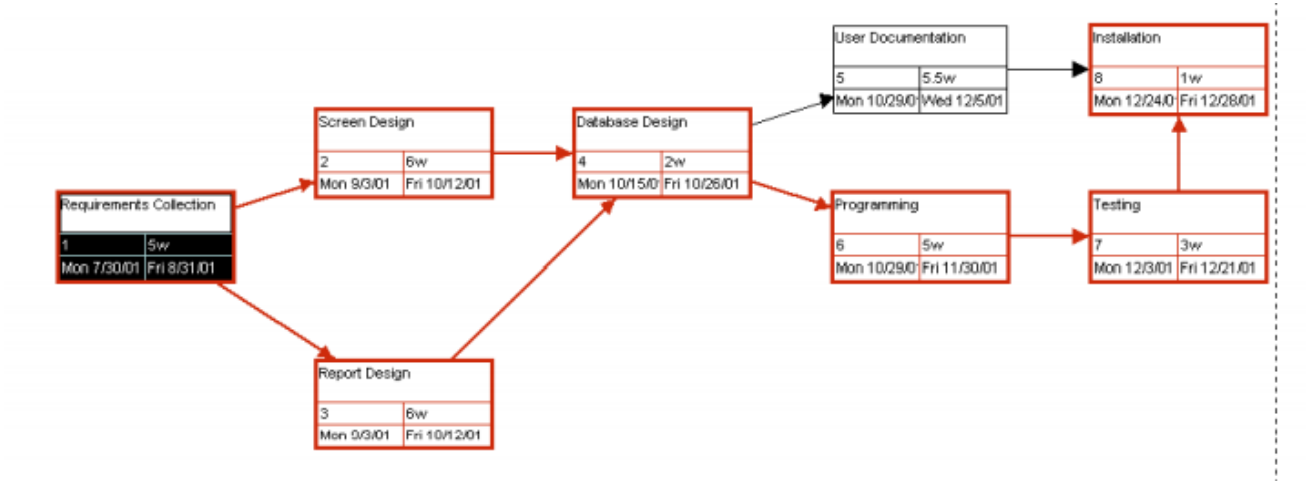
- Identify each activity
- Requirements Collection
- Screen Design
- Report Design
- Database Design
- User Documentation
- Software Programming
- Installation and Testing

- Determine time estimates and expected completion times for each activity
  - Determine sequence of activities
  - Determine critical path
  - Sequence of events that will affect the final project delivery date

- **Calculation of Earlier finish time (TE)i.e forward pass and Latest Finish time(TL) i.e. backward pass**
- **Forward Pass Calculation**
- Step 1: Begin from the start event and move towards the end event.
- Step 2: Put TE = 0 for the start event.
- Step3:Goto the next event(i.enode2)if there is an incoming activity for event2,
- Add calculate TE of previous event(i.eevent1)and activity time.
- Note:If there are more than one incoming activities, calculate TE for all incoming activities and take the maximum value. This value is the TE for event2.
- Step 4: Repeat the same procedure from step 3 till the end event.
- **Backward Pass Calculation**
- Step1:Begin from end event and move towards the start event. Assume that the direction of arrows is reversed.
- Step2:Latest Time TL for the last event is the earliest time. TE of the last event.
- Step3:Goto the next event, if there is an incoming activity, subtract the value of TL of previous event from the activity duration time. The arrived value is TL for that event. If there are more than one incoming activities, take them in minimum TE value.
- Step 4:Repeat the same procedure from step 2 till the start event.
- **An activity is said to be critical only if both the conditions are satisfied.**
- 1. TL –TE = 0
- 2. TLj –tij –TEj = 0

EXAMPLE :REPRESENTING AND SCHEDULING PROJECT PLAN

A Sales Promotion Tracking System(SPTS) has different activities with their time estimation .Calculate the Expected time For the completion of project ,draw the sequence of activities and based on these activities prepare the gantt chart and Network Diagram showing estimated times for each activity and also calculate the earliest and latest expected completion time for each activity.Calculate the slack time to show the activities of project are in critical path or not.
The set of activities and their estimation has been shown in below table:



ACTIVITY	TIME ESTIMATE (in weeks) o	r	p
1. Requirements Collection	1	5	9
2. Screen Design	5	6	7
3. Report Design	3	6	9
4. Database Design	1	2	3
5. User Documentation	3	6	7
6. Programming	4	5	6
7. Testing	1	3	5
8. Installation	1	1	1

step 1: calculation of Expected Time(ET) form PERT Analysis

ACTIVITY	TIME ESTIMATE (in weeks) o	r	p	EXPECTED TIME (ET) o + 4r + p / 6
1. Requirements Collection	1	5	9	5
2. Screen Design	5	6	7	6
3. Report Design	3	6	9	6
4. Database Design	1	2	3	2
5. User Documentation	3	6	7	5.5
6. Programming	4	5	6	5
7. Testing	1	3	5	3
8. Installation	1	1	1	1

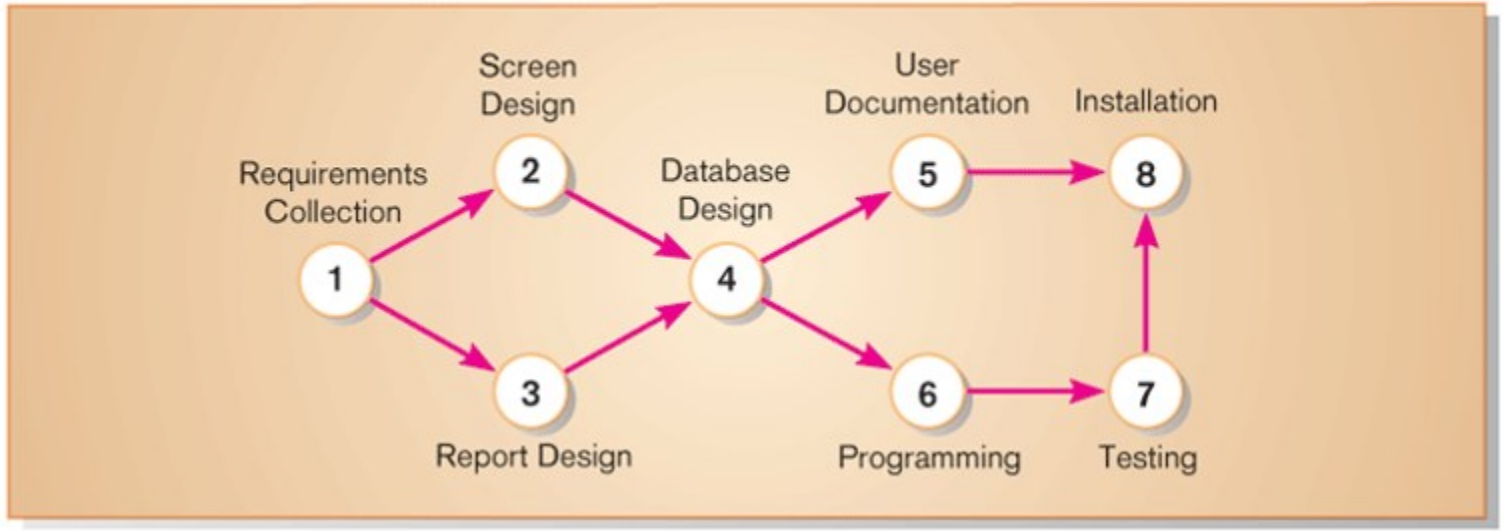
step 2: showing the sequence of activities

Sequence of Activities within the SPTS project

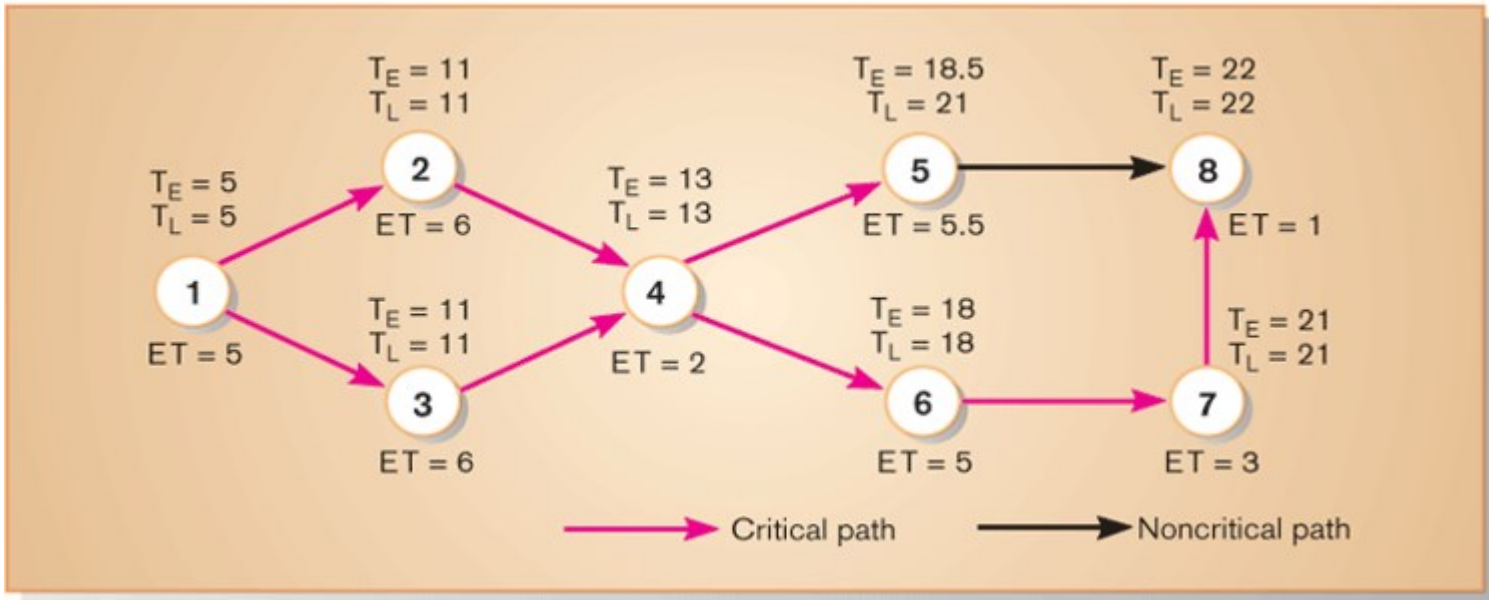
ACTIVITY	PRECEDING ACTIVITY
1. Requirements Collection	—
2. Screen Design	1
3. Report Design	1
4. Database Design	2,3
5. User Documentation	4
6. Programming	4
7. Testing	6
8. Installation	5,7

step 3:Network diagram illustrating different activities dependencies

A network diagram that illustrates the activities (circles) and the sequence (arrows) of those activities



Step 4:Network diagram showing the critical path calculation with Earliest Completion time (TE) and Latest completion time(TL)



Step 5:Critical Path Calculation

ACTIVITY	T_E	T_L	SLACK $T_L - T_E$	ON CRITICAL PATH
1	5	5	0	✓
2	11	11	0	✓
3	11	11	0	✓
4	13	13	0	✓
5	18.5	21	2.5	
6	18	18	0	✓
7	21	21	0	✓
8	22	22	0	✓

Example :

A project schedule has the following characteristics as shown in Table

Table 8.5: Project Schedule

Activity	Name	Time	Activity	Name	Time (days)
1-2	A	4	5-6	G	4
1-3	B	1	5-7	H	8
2-4	C	1	6-8	I	1
3-4	D	1	7-8	J	2
3-5	E	6	8-10	K	5
4-9	F	5	9-10	L	7

i. Construct PERT network.
ii. Compute TE and TL for each activity.
iii. Find the critical path.

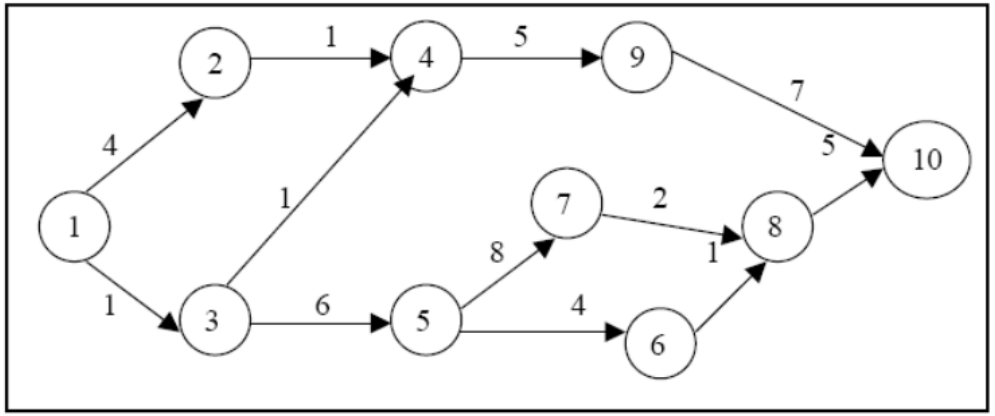


Figure 8.16: Activity Network Diagram

(ii) To determine the critical path, compute the earliest time TE and latest time TL for each of the activity of the project. The calculations of TE and TL are as follows:,

To calculate TE for all activities

$TE_1 = 0$

$TE_2 = TE_1 + t_{1,2} = 0 + 4 = 4$

$TE_3 = TE_1 + t_{1,3} = 0 + 1 = 1$

$TE_4 = \max (TE_2 + t_{2,4} \text{ and } TE_3 + t_{3,4})$
$= \max (4 + 1 \text{ and } 1 + 1) = \max (5, 2)$
$= 5$ days

$TE_5 = TE_3 + t_{3,6} = 1 + 6 = 7$

$TE_6 = TE_5 + t_{5,6} = 7 + 4 = 11$

$TE_7 = TE_5 + t_{5,7} = 7 + 8 = 15$

$TE_8 = \max (TE_6 + t_{6,8} \text{ and } TE_7 + t_{7,8})$
$= \max (11 + 1 \text{ and } 15 + 2) = \max (12, 17)$
$= 17$ days

$TE_9 = TE_4 + t_{4,9} = 5 + 5 = 10$

$TE_{10} = \max (TE_9 + t_{9,10} \text{ and } TE_8 + t_{8,10})$
$= \max (10 + 7 \text{ and } 17 + 5) = \max (17, 22)$
$= 22$ days

To calculate TL for all activities

$TL_{10} = TE_{10} = 22$

$TL_9 = TE_{10} - t_{9,10} = 22 - 7 = 15$

$TL_8 = TE_{10} - t_{8,10} = 22 - 5 = 17$

$TL_7 = TE_8 - t_{7,8} = 17 - 2 = 15$

$TL_6 = TE_8 - t_{6,8} = 17 - 1 = 16$

$TL_5 = \min (TE_6 - t_{5,6} \text{ and } TE_7 - t_{5,7})$
$= \min (16 - 4 \text{ and } 15 - 8) = \min (12, 7)$
$= 7$ days

$TL_4 = TL_9 - t_{4,9} = 15 - 5 = 10$

$TL_3 = \min (TL_4 - t_{3,4} \text{ and } TL_5 - t_{3,5})$
$= \min (10 - 1 \text{ and } 7 - 6) = \min (9, 1)$
$= 1$ day

$TL_2 = TL_4 - t_{2,4} = 10 - 1 = 9$

$TL_1 = \text{Min} (TL_2 - t_{1,2} \text{ and } TL_3 - t_{1,3})$
$= \text{Min} (9 - 4 \text{ and } 1 - 1) = 0$

Table 8.6: Various Activities and their Floats

Activity	Activity Name	Normal Time (tij)	Earliest Time (TE)		Latest Time (TL)		Total Float
			Start (ES)	Finish (EF)	Start (LS)	Finish (LF)	
1-2	A	4	0	4	5	9	5
1-3	B	1	0	1	0	1	0
2-4	C	1	4	5	9	10	5
3-4	D	1	1	2	9	10	8
3-5	E	6	1	7	1	7	0
4-9	F	5	5	10	10	15	5
5-6	G	4	7	11	12	16	5
5-7	H	8	7	15	7	15	0
6-8	I	1	11	12	16	17	5
7-8	J	2	15	17	15	17	0
8-10	K	5	17	22	17	22	0
9-10	L	7	10	17	15	22	5

(iii) From the Table , we observe that the activities 1 – 3, 3 – 5, 5 – 7,7 – 8 and 8 – 10 are critical activities as their floats are zero.